ESP32-S3 Embedded Rust Laboratory Manual

Programming the ESP32-S3 with esp-hal 1.1.1 on Linux Mint

Laboratory tutorial, API reference and integrated projects

November 2026

Table of Contents

About this manual **4**
How accuracy was ensured 4
Verification status legend 4
Findings that differ from common assumptions 5

Part I - Environment and project configuration **6**
Chapter 1 - Introduction to the ESP32-S3 architecture 6
Chapter 2 - Embedded Rust and no_std 8
Chapter 3 - Installing Rust and the Espressif Xtensa toolchain on Linux Mint 10
Chapter 4 - Generating and configuring an ESP32-S3 project 12
Chapter 5 - Every important file explained 12
Chapter 6 - Firmware entry point, panic handler and peripheral initialisation 17
Chapter 7 - The board pin map and GPIO conflicts 19
Chapter 8 - Build, flash and monitor 21

Part II - ESP32-S3 peripheral laboratories **23**
Lab 00 - Hello world, logging and the build pipeline 24
Lab 01 - GPIO digital output: blinking an LED 27
Lab 02 - GPIO digital input, pull-up / pull-down and debouncing 30
Lab 03 - CPU clock configuration 33
Lab 04 - ADC analog-to-digital conversion 36
Lab 05 - LEDC PWM: LED brightness control 39
Lab 06 - Servo motor control with PWM 42
Lab 07 - Hardware timers and periodic events 45
Lab 08 - GPIO interrupts and interrupt handlers 48
Lab 09 - UART serial communication 51
Lab 10 - I2C master: bus scan 54
Lab 11 - LCD1602 on I2C (PCF8574 backpack) 57
Lab 12 - SPI master: full-duplex loopback 60
Lab 13 - RC522 RFID reader over SPI 63
Lab 14 - DMA: memory-to-memory transfer 67
Lab 14B - DMA with SPI: full-duplex loopback 70
Lab 15 - Watchdog timer 73
Lab 16 - RMT peripheral: driving a WS2812 RGB LED 76
Lab 17 - PCNT pulse counter and rotary encoder 80
Lab 18 - MCPWM: motor-control PWM signals 84
Lab 19 - TWAI (CAN 2.0) communication 87
Lab 20 - Wi-Fi connectivity: station, DHCP and a TCP echo server 90
Lab 21 - Bluetooth Low Energy: GATT Battery Service 95
Lab 22A - USB Serial/JTAG: blocking echo 101
Lab 22B - USB OTG: CDC-ACM serial device with embassy-usb 104
Lab 23 - Asynchronous programming with Embassy 108
Lab 24 - Bonus: external PSRAM as heap 112

Part III - ESP-HAL 1.1.1 API reference **115**
1. Initialisation, clocks, time and delay 116
2. GPIO 121
3. ADC (analog input) 127
4. LEDC (PWM) and servo control 129
5. Hardware timers and watchdog 132
6. Interrupt handlers, entry point and macros 135
7. UART 136
8. I2C master 139

9. SPI master 141
10. DMA 144
11. RMT 146
12. PCNT (pulse counter) 148
13. MCPWM 152
14. TWAI (CAN 2.0) 155
15. USB Serial/JTAG and USB OTG 158
16. PSRAM, heap allocation and logging (companion crates) 159
17. esp-rtos, Embassy glue and wireless (companion crates) 160

Part IV - Integrated projects 163

Project 1 - Basic monitoring system 163
Project 2 - RFID access-control system 167
Project 3 - Advanced IoT monitoring system 172

Appendices 181

Appendix A - Project directory structure and Cargo.toml 181
Appendix B - Final checklist for compiling, flashing and testing each laboratory 186
Appendix C - Version pitfalls and migration notes 188
Appendix D - Verification record 188
Appendix E - Resolved dependency versions 190
Appendix F - References 191

About this manual

This manual teaches embedded Rust on the **ESP32-S3** (Xtensa LX7) using **esp-hal version 1.1.1** in `no_std` mode, on Linux Mint with Visual Studio Code, Cargo and `espflash`. It contains an environment guide (Part I), 27 hands-on laboratories (Part II), an API reference generated from the esp-hal 1.1.1 source code (Part III), and three integrated projects (Part IV). Every source listing in this document is a file from the accompanying project `esp32s3-labs`; the text and the code cannot drift apart because the listings were inserted by a script from the files that were compiled.

How accuracy was ensured

The brief asked for technical accuracy above length and for a clear separation of tested, untested and conceptual material. The following work was carried out while preparing this manual.

Check	What was done	Result
Exact version pin	Project depends on `esp-hal = "=1.1.1"`; `Cargo.lock` confirms 1.1.1	Verified
Source of truth	The git tag `esp-hal-v1.1.1` was cloned; examples, `lib.rs`, drivers and device metadata were read	Done
Compilation	Every laboratory and project was compiled with `cargo build --release` for `xtensa-esp32s3-none-elf` (Rust 1.95.0 Xtensa toolchain, Xtensa GCC 16.1.0)	30 binaries, see Appendix D
Stable-only claims	Labs labelled stable-only were compiled in a second project with the `unstable` feature **disabled**; two unstable labs were compiled there as negative controls and failed as expected	Verified
Ownership claims	A deliberate pin-conflict program was compiled and rejected by the compiler (`E0382`)	Verified
API reference	Each signature in Part III was extracted automatically from the 1.1.1 source; the generator aborts if a name does not exist. Stability flags come from `#[instability::unstable]` and the `unstable_module!` list in `lib.rs`	171 entries resolved
Hardware	**No code was executed on an ESP32-S3.** No board was available while preparing this manual; no timing, electrical or wireless behaviour was measured	Not hardware-tested

Verification status legend

Every laboratory states one of the following levels. The levels are deliberately narrow; nothing in this manual is labelled “tested” in the sense of having run on hardware.

Label	Meaning
BUILD-VERIFIED	The program compiled and linked with `cargo build --release` against esp-hal 1.1.1 in this project. Logic and electrical behaviour have not been observed on hardware.
+ STABLE-ONLY CHECKED	Additionally compiled with the esp-hal `unstable` feature disabled, so it uses only the stable API.
NOT hardware-tested	Added where behaviour depends on parts, timing or radios that could not be exercised: RC522, LCD, servo, WS2812, TWAI bus, Wi-Fi, BLE, USB.
Pattern / unverified	A fragment that was *not* compiled (for example the MCPWM dead-time helper mentioned in Lab 18). It is explicitly flagged where it appears.
Hardware-tested	Executed on a real board and observed. *No item in this manual has this label.*

Findings that differ from common assumptions

Reading the real 1.1.1 source and compiling against it exposed several points that a tutorial written from memory would get wrong. They are collected here because they affect how you should use this manual.

Finding	Consequence
The tag `v1.1.1` in the link of the brief does not exist. The tag is **`esp-hal-v1.1.1`**.	Use `https://github.com/esp-rs/esp-hal/tree/esp-hal-v1.1.1`.
1.1.1 is not the newest release: 1.1.2 and 1.2.x exist.	Pin `esp-hal = "=1.1.1"`. A requirement such as `~1.1` would silently accept 1.1.2.
Embassy support is no longer `esp-hal-embassy`; it is provided by **`esp-rtos`**, and Wi-Fi/BLE by **`esp-radio`**.	Labs 20-23 and Project 3 use `esp-rtos 0.3.0` and `esp-radio 0.18.0`.
Most peripherals needed here (ADC, LEDC, MCPWM, RMT, PCNT, TWAI, timers, DMA, USB) are behind the esp-hal **`unstable`** feature. Stable: GPIO, I2C, SPI, UART, clock, time, system, interrupt, RNG, eFuse.	Each lab states its level. Unstable APIs may change in minor releases.
Inside stable modules some items are still unstable: `Spi::with_cs`, `Io::new`, `Input::listen`, `system::reset_reason`, `Uart::split`. The `#[handler]` attribute macro is unstable too.	Lab 12 uses a software chip select so that it is stable-only.
`esp_hal::init` **disables** all watchdogs.	Lab 15 enables one explicitly.
`esp_hal::tsens` (on-die temperature sensor) has **no driver for the ESP32-S3** in 1.1.1 (only C6, H2, P4). The 1.1.1 doc example also contains a typo (`to_celcius`; the method is `to_celsius`).	Project 3 does not use it.
`Mem2Mem::new` on the ESP32-S3 consumes the **`SPI2`** token as well as a DMA channel.	SPI2 cannot be used in the same program as Lab 14.
trouble-host 0.6.0 needs `embassy-sync` **0.7**; version 0.8 in your own `Cargo.toml` breaks `#[gatt_server]`.	`Cargo.toml` pins `embassy-sync = "0.7.2"` (real error shown in Lab 21).
`Input::wait_for_falling_edge` is a **stable** method, although the generic `wait_for` and the `Flex` variants are unstable.	Lab 23 labelled accordingly.
Building `espflash` from source requires **rustc 1.95.0 or newer** (README of espflash).	Use `cargo binstall espflash` or a recent toolchain (chapter 3).

Part I - Environment and project configuration

Chapter 1 - Introduction to the ESP32-S3 architecture

1.1 What the chip is

The ESP32-S3 is a dual-core, 32-bit **Xtensa LX7** microcontroller from Espressif with integrated 2.4 GHz Wi-Fi (IEEE 802.11 b/g/n) and Bluetooth Low Energy 5. The esp-hal device description for the chip lists two cores, 45 GPIO pins (GPIO0-GPIO21 and GPIO26-GPIO48), two 12-bit SAR ADC units with 20 channels in total, 14 capacitive-touch inputs, and five DMA channels. The CPU runs at 80, 160 or 240 MHz and has a single-precision floating-point unit. It has no DAC, no Bluetooth Classic and no 5 GHz radio, and its CAN controller (called TWAI) implements CAN 2.0, not CAN FD.

Resource	ESP32-S3	Where it appears in this manual
CPU	2 x Xtensa LX7, up to 240 MHz	Lab 03
Internal memory	512 KB SRAM, ROM, small RTC memory	Chapter 6
External memory	Flash (quad/octal SPI) and optional PSRAM, both cached	Lab 24
GPIO	45 pins, routed through the GPIO matrix / IO MUX	Labs 01, 02, 08
ADC	ADC1 (GPIO1-10), ADC2 (GPIO11-20), 12-bit	Lab 04
PWM	LEDC (8 channels), 2 x MCPWM	Labs 05, 06, 18
Timers	2 timer groups (2 timers + watchdog each), SYSTIMER	Labs 07, 15
Serial buses	3 x UART, 2 x I2C, SPI2/SPI3 (SPI0/1 reserved for flash), 2 x I2S	Labs 09-14
DMA	GDMA, 5 channels	Lab 14
Special	RMT (4 TX + 4 RX), PCNT (4 units), TWAI, LCD_CAM	Labs 16, 17, 19
USB	USB OTG full-speed and USB Serial/JTAG (shared pins GPIO19/20)	Labs 22a, 22b
Radio	Wi-Fi 802.11 b/g/n, Bluetooth LE 5	Labs 20, 21
Security	AES, SHA, RSA, HMAC accelerators, RNG	not covered
Coprocessor	ULP RISC-V core	not covered

Source of the peripheral list: `esp-metadata/devices/esp32s3.toml` *and the module list of esp-hal 1.1.1.*

1.2 Memory and caches in one paragraph

Instructions and data live in a unified internal SRAM, and the CPU also executes code from external flash through an instruction cache. This is why interrupt handlers that must run while the flash cache is busy are marked `#[ram]` (Labs 07, 08). External PSRAM is mapped through a data cache; it is larger but slower than internal RAM and cannot be used by DMA engines in the same way as internal memory (Lab 24).

1.3 The GPIO matrix and the IO MUX

Most peripheral signals (UART TX/RX, SPI lines, PWM outputs, I2C, …) are connected to pins through the **GPIO matrix**, which can route a signal to almost any pin. A few functions are fixed to particular pads: the analog inputs (ADC channels, touch), the USB D+/D- pads (GPIO20/GPIO19), the flash/PSRAM lines (GPIO26-GPIO37), and the strapping pins. In esp-hal this flexibility appears as drivers that accept *any* suitable pin token (`.with_sda(peripherals.GPIO41)`), while pin-specific restrictions are enforced by the types of the tokens or, for pads with board-level meaning, documented in chapter 7.

1.4 Which esp-hal drivers exist for the ESP32-S3

The esp-hal 1.1.1 documentation marks drivers that are not yet covered by semantic-versioning guarantees as `unstable`. The table is derived from the `unstable_module!` and `unstable_driver!` lists in `esp-hal/src/lib.rs` of the tag `esp-hal-v1.1.1` and matches the module list on docs.espressif.com.

Module	Status	Provides	Used in
`clock`	**Stable**	CPU clock presets and queries	Lab 03
`efuse`	**Stable**	eFuse access	-
`gpio`	**Stable**	Digital I/O, interrupts, async pins	Labs 01, 02, 08
`i2c`	**Stable**	I2C master	Labs 10, 11
`interrupt`	**Stable**	Interrupt handler types	Labs 07, 08
`peripherals`	**Stable**	Peripherals singleton struct	all
`rng`	**Stable**	Random numbers	Labs 20, Project 3
`spi`	**Stable**	SPI master	Labs 12, 13, 14b
`system`	**Stable**	Reset, reset reason	Lab 15
`time`	**Stable**	Instant, Duration, Rate	all
`uart`	**Stable**	UART	Lab 09
`aes`	UNSTABLE	AES accelerator	-
`analog`	UNSTABLE	ADC (and DAC on other chips)	Lab 04, Projects 1, 3
`delay`	UNSTABLE	Blocking Delay	Labs 05, 06, 11
`dma`	UNSTABLE	GDMA, buffers, Mem2Mem	Lab 14
`ecc`	UNSTABLE	ECC accelerator	-
`etm`	UNSTABLE	Event task matrix	-
`hmac`	UNSTABLE	HMAC accelerator	-
`i2s`	UNSTABLE	I2S audio bus	-
`lcd_cam`	UNSTABLE	LCD and camera interface	camera (not covered)
`ledc`	UNSTABLE	LED PWM controller	Labs 05, 06, Projects 1, 2
`mcpwm`	UNSTABLE	Motor-control PWM	Lab 18
`otg_fs`	UNSTABLE	USB OTG full speed	Lab 22b
`parl_io`	UNSTABLE	Parallel IO	-
`pcnt`	UNSTABLE	Pulse counter	Lab 17
`psram`	UNSTABLE	External PSRAM	Lab 24
`rmt`	UNSTABLE	Remote-control transceiver	Lab 16
`rsa`	UNSTABLE	RSA accelerator	-
`rtc_cntl`	UNSTABLE	RTC control, sleep	-
`sha`	UNSTABLE	SHA accelerator	-
`timer`	UNSTABLE	Timer groups, SYSTIMER, watchdog	Labs 07, 15
`touch`	UNSTABLE	Capacitive touch	-
`tsens`	UNSTABLE	Temperature sensor: no ESP32-S3 driver in 1.1.1	Project 3 note
`twai`	UNSTABLE	CAN 2.0 controller	Lab 19
`usb_serial_jtag`	UNSTABLE	Built-in USB serial/JTAG	Lab 22a

Other unstable internals (`asynch`, `debugger`, `assist_debug`, `rom`, `sync`, `trace`) are not used. Unstable modules need `esp-hal = { features = ["unstable"] }`.

Wi-Fi and Bluetooth are **not** in esp-hal: they come from the separate `esp-radio` crate (0.18.0), scheduled by `esp-rtos` (0.3.0). The on-die temperature sensor (`tsens`) has no driver for the ESP32-S3 in this release.

Chapter 2 - Embedded Rust and `no_std`

2.1 `core`, `alloc` and `std`

Rust's standard library is layered. **`core`** has no dependency on an operating system: primitive types, `Option`, `Result`, iterators, formatting, atomics. **`alloc`** adds heap collections (`Vec`, `String`, `Box`) and needs a global allocator. **`std`** adds files, threads, networking and requires an OS. Firmware for the ESP32-S3 in this manual is `#![no_std]`: it links `core` always and `alloc` only when a heap is provided (`esp-alloc`, needed by Wi-Fi and BLE). Because the Xtensa target is not distributed as a pre-built standard library, `cargo` compiles `core` (and `alloc`) from source with `-Zbuild-std`, which is enabled in `.cargo/config.toml` and is the reason a special Xtensa toolchain (`channel = "esp"`) is required.

`#![no_main]` tells the compiler not to expect the normal `main` entry shim; the `#[esp_hal::main]` attribute (or `#[esp_rtos::main]` for async programs) generates the entry point that the runtime calls.

2.2 The crate stack

```
            your firmware  (src/bin/*.rs)
                  |
   esp-hal  (drivers: gpio, uart, spi, i2c, ... on a register-access layer)
      |           |                    |
 embedded-hal   esp-rtos  ------>  esp-radio (Wi-Fi, BLE)
 traits (1.0)   (scheduler, Embassy       |
                 time driver)         embassy-net / trouble-host
                  |
   esp-println (console)   esp-backtrace (panic handler)   esp-alloc (heap)
```

2.3 Ownership of peripheral tokens

`esp_hal::init` returns a `Peripherals` struct whose fields (`GPIO21`, `UART1`, `I2C0`, `SPI2`, …) are *singletons*: there is exactly one value for each piece of hardware, and the type system tracks who owns it. This is Rust's ownership rule applied to hardware.

- **Move**: `Output::new(peripherals.GPIO21, ...)` *moves* the token into the driver. Afterwards `peripherals.GPIO21` is unusable; a second use is a compile-time error, not a run-time surprise.
- **Lifetimes**: tokens carry a lifetime (`GPIO21<'d>`). Tokens from `init` are `'static`. The `reborrow()` method creates a shorter-lived handle so you can lend a peripheral to a driver and take it back after the driver is dropped (esp-hal documentation, "Peripheral singletons").
- **Partial moves**: moving some fields out of `Peripherals` leaves the rest usable (`peripherals.GPIO21` and `peripherals.UART1` can be moved independently).
- **Shared state with interrupts**: a driver that an interrupt handler must touch is moved into a `static` wrapped in `critical_section::Mutex<RefCell<Option<_>>>` (Labs 07, 08, 17). The `Option` allows the static to start empty; the `Mutex` disables interrupts while the `RefCell` is borrowed.
- **DMA buffers** are moved into the transfer and returned by `wait()`, so you cannot touch memory the DMA engine is still using (Lab 14b).
- **Never `core::mem::forget` an esp-hal type**: many drivers rely on `Drop` to leave the hardware in a defined state (esp-hal documentation). The generated project denies `clippy::mem_forget`.

Demonstration: the compiler protects the camera pins

The helper `reserve_camera_pins!` (in `src/board.rs`) moves the fourteen camera pin tokens into a struct. The following program (kept in `docs/compile_fail/pin_conflict.rs`) then tries to use the camera pin GPIO15 for an LED. The compiler rejects it. This is the **real, unedited output** of `cargo build` from this project:

```
error[E0382]: use of moved value: `peripherals.GPIO15`
  --> src/bin/zz_conflict_demo.rs:12:28
   |
10 |     let _camera = reserve_camera_pins!(peripherals);
   |                   ------------------------------- value moved here
11 |     // GPIO15 is the camera XCLK pin, already moved into `_camera`:
12 |     let _bad = Output::new(peripherals.GPIO15, Level::Low, OutputConfig::default());
   |                            ^^^^^^^^^^^^^^^^^^ value used here after move
   |
   = note: move occurs because `peripherals.GPIO15` has type `GPIO15<'_>`, which does not
implement the `Copy` trait

For more information about this error, try `rustc --explain E0382`.
error: could not compile `esp32s3-labs` (bin "zz_conflict_demo") due to 1 previous error
```

2.4 Error handling and return types

- Most configuration functions return `Result<T, E>` (`Uart::new`, `I2c::new`, `Spi::new`, `Rmt::new`, LEDC `configure`). In the labs `.unwrap()` is used for brevity where the configuration is a constant; production code should handle or log the error.
- Hardware operations that may not be ready return **`nb::Result<T, E>`** (non-blocking): `Err(nb::Error::WouldBlock)` means "try again". The `nb::block!` macro spins until the operation completes (ADC `read_oneshot`, TWAI `transmit`/`receive`, USB Serial/JTAG `read_byte`).
- Async methods return futures and are used with `.await`.
- `main` returns `!` (never): on an embedded target there is nowhere to return to.
- A panic calls the panic handler of `esp-backtrace`, which prints the message and a backtrace and halts. `unwrap()` and `expect()` are therefore *controlled crashes*; they are acceptable in laboratories but should be replaced by explicit handling in products.

2.5 Stable and unstable APIs

esp-hal 1.x promises semantic-versioning stability only for the **stable** surface. Everything else is compiled only when the crate feature `unstable` is enabled, and may change in any minor release. The generated project enables it (`features = ["esp32s3", "unstable", "log-04"]`).

To check this claim empirically, the labs labelled "stable-only" were compiled in a second project with `unstable` *off*. The two unstable labs placed there as negative controls fail with the following genuine compiler output (excerpt):

```
error[E0432]: unresolved imports `esp_hal::analog::adc::Adc`,
`esp_hal::analog::adc::AdcCalCurve`, `esp_hal::analog::adc::AdcConfig`
  --> src/bin/neg_lab04_adc_needs_unstable.rs:13:19
   |
13 |     analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
   |                   ^^^  ^^^^^^^^^^^  ^^^^^^^^^ no `AdcConfig` in `analog::adc`
   |                   |    |
   |                   |    no `AdcCalCurve` in `analog::adc`
   |                   no `Adc` in `analog::adc`
   |
help: a similar name exists in the module
   |
13 -     analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
13 +     analog::adc::{Adc, AdcCalSource, AdcConfig, Attenuation},
   |
error[E0603]: module `analog` is private
   --> src/bin/neg_lab04_adc_needs_unstable.rs:13:5
    |
 13 |     analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
    |     ^^^^^^  --- module `adc` is not publicly re-exported
    |     |
    |     private module
    |
note: the module `analog` is defined here
   --> /home/claude/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/esp-hal-
```

```
1.1.1/src/lib.rs:236:13
    |
236 |                 pub(crate) mod $module;
    |                 ^^^^^^^^^^^^^^^^^^^^^^
...
```

Some items are unstable even inside stable modules: they carry `#[instability::unstable]` in the source. Part III flags each API accordingly. Libraries that depend on esp-hal must use the feature `requires-unstable`, never `unstable` itself.

2.6 Language and library features you will meet

Feature	Where	Why
`static` + `critical_section::Mutex`	Labs 07, 08, 17	Share data between `main` and interrupts
`core::sync::atomic`	Labs 07, 08, 17	Lock-free counters/flags
`heapless::String`	Lab 09, Project 3	Formatting without a heap
`core::fmt::Write` / `write!`	Lab 11, projects	Printing numbers to the LCD or sockets
`async`/`await`	Labs 20-23, Project 3	Cooperative multitasking
Attribute macros `#[main]`, `#[handler]`, `#[ram]`	everywhere	Entry point, ISR, RAM placement

Chapter 3 - Installing Rust and the Espressif Xtensa toolchain on Linux Mint

> **Honesty note.** The commands in this chapter are the documented procedure. In the sandbox used to prepare this manual the *same components* were installed manually from the same release assets that `espup` downloads (Rust 1.95.0.0 for Xtensa from `esp-rs/rust-build`, and the Xtensa GCC from `espressif/crosstool-NG`), because the sandbox could not reach all of the servers `espup` uses. `espup` itself, `espflash` and VS Code were **not run** in the sandbox. Only `esp-generate 1.3.0` was built and run. Flags below were checked against the `espup` 0.19.0 and `espflash` README sources.

3.1 System packages

```
sudo apt update
sudo apt install -y build-essential curl git pkg-config libssl-dev libudev-dev
```

`libudev-dev` is needed to compile `espflash` (serial-port enumeration); `pkg-config` and `libssl-dev` are used by several `cargo install` builds.

3.2 Rust with rustup

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
. "$HOME/.cargo/env"
rustc --version
```

3.3 espup: the Xtensa toolchain

The ESP32-S3 uses the Xtensa instruction set, which upstream Rust does not support. Espressif maintains a fork of the compiler (`rustc` with an Xtensa LLVM back end) distributed as the rustup toolchain **`esp`**. `espup` installs it, plus the Xtensa GCC that is used as the *linker*, and writes an environment file.

```
cargo install espup --locked
espup install --targets esp32s3          # latest toolchain
. "$HOME/export-esp.sh"                  # puts the Xtensa GCC on PATH
```

To reproduce the toolchain used for verification here, pin it: `espup install --targets esp32s3 --toolchain-version 1.95.0.0`. Useful options verified in `espup` 0.19.0: `-t/--targets`, `-v/--toolchain-version`, `-f/--export-file` (default `~/export-esp.sh`), `-c/--crosstool-toolchain-version`, `-s/--std`.

The environment file must be **sourced in every shell** that runs `cargo`, or add this line to `~/.bashrc`:

```
echo '. "$HOME/export-esp.sh"' >> ~/.bashrc
```

Verify: `rustc +esp --version` should print a line such as `rustc 1.95.0-nightly (95e5bda86 2026-04-15) (1.95.0.0)` (this is the output of the toolchain release used to verify this manual), and `xtensa-esp32s3-elf-gcc --version` should print the Xtensa GCC banner.

The minimum supported Rust version of esp-hal 1.1.1 and all companion crates used here is **1.88.0** (from their `Cargo.toml`).

3.4 espflash and esp-generate

```
cargo install espflash --locked          # run OUTSIDE a project directory
cargo install esp-generate --locked
```

- The `espflash` README states that compiling it from source needs `rustc >= 1.95.0`. If your `stable` toolchain is older, use `cargo binstall espflash` (pre-built binaries) instead.
- **Version caution for `esp-generate`**: it generates a project for the esp-hal release that is current when *it* was released. This manual needs the esp-hal 1.1.x templates, which came with **esp-generate 1.3.0** (April 2026). To obtain exactly those templates use `cargo install esp-generate --version 1.3.0 --locked`. The generator banner of that version lists the matching crates: esp-hal ~1.1.0, esp-rtos 0.3.0, esp-bootloader-esp-idf 0.5.0, esp-println 0.17.0, esp-backtrace 0.19.0, esp-alloc 0.10.0, esp-radio 0.18.0.
- To use `cargo binstall`, first install it with `cargo install cargo-binstall`.

3.5 Permission to use the serial port

On Linux the serial device (`/dev/ttyUSB0` for a CH340/CP210x bridge, `/dev/ttyACM0` for the native USB Serial/JTAG) belongs to the group `dialout`. The `espflash` README recommends:

```
sudo usermod -a -G dialout $USER
su - $USER          # or log out and in again
espflash list-ports
```

3.6 Visual Studio Code

Install the *rust-analyzer* extension (and optionally *Even Better TOML* and *CodeLLDB* for debugging). The generator's `vscode` option writes `.vscode/settings.json` so that rust-analyzer analyses the project for the right target with the right toolchain (explained in chapter 5).

Chapter 4 - Generating and configuring an ESP32-S3 project

4.1 Generate

`esp-generate` is Espressif's project generator. Interactively, `esp-generate --chip esp32s3 my-project` opens a menu in which options are ticked; non-interactively the options are passed with `-o`. The basic project used as the reference in this chapter was generated with:

```
esp-generate --chip esp32s3 --headless -o unstable-hal -o log -o esp-backtrace -o vscode
basic
```

Option	Effect
`unstable-hal`	Enables the esp-hal `unstable` feature (needed for most peripherals in this manual)
`log`	Adds the `log` crate and the esp-println logger, with `log-04` features in esp-hal and esp-println
`esp-backtrace`	Adds the panic handler with backtrace printing
`vscode`	Writes `.vscode/` settings for rust-analyzer, tasks and debugging
`alloc`, `wifi`, `embassy`, `ble-trouble`	(used for the second reference project) heap, Wi-Fi, esp-rtos/Embassy executor, BLE with the trouBLE host stack

The generated tree of the `basic` project (produced by running the generator in the sandbox, `target/` omitted):

```
basic/
|-- .cargo/
|   `-- config.toml
|-- .vscode/
|   |-- extensions.json
|   |-- launch.json
|   |-- settings.json
|   `-- tasks.json
|-- src/
|   |-- bin/
|   |   `-- main.rs
|   `-- lib.rs
|-- .clippy.toml
|-- .gitignore
|-- build.rs
|-- Cargo.toml
`-- rust-toolchain.toml
```

4.2 From the generated project to the laboratory project

This manual's code is one Cargo package, `esp32s3-labs`, with one binary per laboratory in `src/bin/`. It is the generated project plus: (a) `esp-hal` pinned with `=1.1.1`; (b) additional dependencies for the labs; (c) a small library (`src/lib.rs`) with the camera-pin reservation and the LCD driver; (d) `build-std = ["alloc", "core"]` because `esp-alloc` needs the `alloc` crate. Everything else is unchanged.

Chapter 5 - Every important file explained

5.1 `Cargo.toml`

The generated file (esp-hal `~1.1.0` means "1.1.x, at least 1.1.0"):

```toml
[package]
name = "basic"
version = "0.1.0"
edition = "2024"
rust-version = "1.88"

[[bin]]
```

```toml
name = "basic"
path = "./src/bin/main.rs"

[dependencies]
esp-hal = { version = "~1.1.0", features = [
    "esp32s3",
    "unstable",
    "log-04",
] }


esp-bootloader-esp-idf = { version = "0.5.0", features = [
    "log-04",
    "esp32s3",
] }
log = "0.4.27"

esp-println = { version = "0.17.0", features = [
    "esp32s3",
    "log-04",
] }
esp-backtrace = { version = "0.19.0", features = [
    "esp32s3",
    "panic-handler",
    "println",
] }
critical-section = "1.2.0"


# For fine tuning these settings, please refer to https://doc.rust-
lang.org/cargo/reference/profiles.html
[profile.dev]
# The default debug profile is too slow and too big for resource-constrained devices.
# Always build with some optimizations enabled.
opt-level = "s"

[profile.release]
codegen-units = 1        # LLVM can perform better optimizations using a single thread
debug = 2                # prefer slower builds but better debugging experience
lto = 'fat'
opt-level = 's'
```

Entry	Meaning
`edition = "2024"`, `rust-version = "1.88"`	Rust 2024 edition; minimum compiler required by the esp crates
`[[bin]] name/path`	A single binary at `src/bin/main.rs`. The lab project omits this and relies on automatic discovery of every file in `src/bin/`
`esp-hal` features `esp32s3`	**Chip selection** (exactly one chip feature must be enabled)
`esp-hal` feature `unstable`	Enables the APIs without stability guarantee (ADC, LEDC, timers, DMA, …)
`esp-hal` feature `log-04`	esp-hal itself logs through the `log` crate
`esp-bootloader-esp-idf`	Provides `esp_app_desc!()`, the application descriptor required by the second-stage bootloader
`log`	The logging facade used by `info!`, `warn!`, `error!`
`esp-println` features `esp32s3`, `log-04`	Console output and the `log` backend. Exactly one output feature is active: `auto` (default), `jtag-serial`, `uart`, `no-op`
`esp-backtrace` features `panic-handler`, `println`	Defines the `#[panic_handler]` and prints through esp-println
`critical-section`	The standard critical-section interface used for data shared with interrupts
`[profile.dev] opt-level = "s"`	Even debug builds are size-optimised: unoptimised code is too large and too slow
`[profile.release] lto = 'fat'`, `codegen-units = 1`, `opt-level = 's'`, `debug = 2`	Smallest code and best optimisation; debug symbols are kept (they do not go to flash)

Dependencies **added** by the laboratory project (see Appendix A for the full file): `esp-alloc` (heap), `esp-rtos`, `esp-radio`, `embassy-executor`, `embassy-time`, `embassy-sync`, `embassy-net`, `embassy-futures`, `embassy-usb`, `trouble-host`, `bt-hci`, `embedded-hal`, `embedded-hal-async`, `embedded-hal-bus`, `embedded-io-async`, `mfrc522`, `heapless`, `nb`, `static_cell`, `libm`.

5.2 .cargo/config.toml

```
[target.xtensa-esp32s3-none-elf]
runner = "espflash flash --monitor --chip esp32s3"

[env]
ESP_LOG="info"

[build]
rustflags = [
  "-C", "link-arg=-nostartfiles",
]

target = "xtensa-esp32s3-none-elf"

[unstable]
build-std = ["core"]
```

Line	Meaning
`[target.xtensa-esp32s3-none-elf] runner = "espflash flash --monitor --chip esp32s3"`	`cargo run` builds the program and then invokes this command with the ELF path appended: flash the board and open the serial monitor
`[env] ESP_LOG = "info"`	Compile-time log level for esp-println's logger (`error`, `warn`, `info`, `debug`, `trace`)
`[build] target = "xtensa-esp32s3-none-elf"`	Compile for the ESP32-S3 by default (no `--target` needed)
`rustflags = ["-C", "link-arg=-nostartfiles"]`	The linker is the Xtensa GCC; this stops it from adding the C runtime start-up files, because esp-hal supplies its own
`[unstable] build-std = ["core"]`	Build the `core` library from source for this target (the lab project adds `"alloc"`)

5.3 build.rs

The build script runs before compilation. It does two things (the full generated file follows):

1. It prints `cargo:rustc-link-arg=-Tlinkall.x`, adding the esp-hal linker script **last**.
2. `linker_be_nice()` registers the build script itself as the linker's `--error-handling-script`. When linking fails with an undefined symbol, the linker calls it with the symbol name, and it prints a hint instead of leaving you with a cryptic message. The hints cover `_stack_start` (missing linker script), `esp_rtos_*` (esp-radio used without initialising esp-rtos), `malloc`/`free` (missing `esp-alloc`), and `defmt` symbols.

```
fn main() {
    linker_be_nice();
    // make sure linkall.x is the last linker script (otherwise might cause problems with
flip-link)
    println!("cargo:rustc-link-arg=-Tlinkall.x");
}

fn linker_be_nice() {
    let args: Vec<String> = std::env::args().collect();
    if args.len() > 1 {
        let kind = &args[1];
        let what = &args[2];

        match kind.as_str() {
            "undefined-symbol" => match what.as_str() {
                what if what.starts_with("_defmt_") => {
                    eprintln!();
                    eprintln!(
```

```
                        "□ `defmt` not found - make sure `defmt.x` is added as a linker
script and you have included `use defmt_rtt as _;`"
                    );
                    eprintln!();
                }
                "_stack_start" => {
                    eprintln!();
                    eprintln!("□ Is the linker script `linkall.x` missing?");
                    eprintln!();
                }
                what if what.starts_with("esp_rtos_") => {
                    eprintln!();
                    eprintln!(
                        "□ `esp-radio` has no scheduler enabled. Make sure you have
initialized `esp-rtos` or provided an external scheduler."
                    );
                    eprintln!();
                }
                "embedded test linker_file_not_added_to_rustflags" => {
                    eprintln!();
                    eprintln!(
                        "□ `embedded-test` not found - make sure `embedded-test.x` is
added as a linker script for tests"
                    );
                    eprintln!();
                }
                "free"
                | "malloc"
                | "calloc"
                | "get_free_internal_heap_size"
                | "malloc_internal"
                | "realloc_internal"
                | "calloc_internal"
                | "free_internal" => {
                    eprintln!();
                    eprintln!(
                        "□ Did you forget the `esp-alloc` dependency or didn't enable the
`compat` feature on it?"
                    );
                    eprintln!();
                }
                _ => (),
            },
            // we don't have anything helpful for "missing-lib" yet
            _ => {
                std::process::exit(1);
            }
        }

        std::process::exit(0);
    }

    println!(
        "cargo:rustc-link-arg=-Wl,--error-handling-script={}",
        std::env::current_exe().unwrap().display()
    );
}
```

5.4 `rust-toolchain.toml` and `.clippy.toml`

```
[toolchain]
channel = "esp"
```

`channel = "esp"` makes rustup use the Espressif toolchain installed by `espup` in this directory, so `cargo build` finds the Xtensa compiler automatically. `stack-size-threshold = 1024` (in `.clippy.toml`) configures the `clippy::large_stack_frames` lint that `main.rs` denies, to catch accidentally large stack buffers on a chip with little RAM.

5.5 .vscode/settings.json

```
{
  "rust-analyzer.cargo.allTargets": false,
  "rust-analyzer.cargo.target": "xtensa-esp32s3-none-elf",
  "rust-analyzer.server.extraEnv": {
    "RUSTUP_TOOLCHAIN": "stable"
  },
  "rust-analyzer.cargo.extraEnv": {
    "RUSTUP_TOOLCHAIN": "esp"
  },
}
```

rust-analyzer analyses the project for `xtensa-esp32s3-none-elf` (not the host), only the binaries (`allTargets: false`), with cargo running under the `esp` toolchain while the rust-analyzer server itself uses `stable`.

5.6 src/bin/main.rs (generated)

```
#![no_std]
#![no_main]
#![deny(
    clippy::mem_forget,
    reason = "mem::forget is generally not safe to do with esp_hal types, especially
those \
    holding buffers for the duration of a data transfer."
)]
#![deny(clippy::large_stack_frames)]

use esp_hal::{
    clock::CpuClock,
    main,
    time::{Duration, Instant},
};

use log::info;

use esp_backtrace as _;


// This creates a default app-descriptor required by the esp-idf bootloader.
// For more information see: <https://docs.espressif.com/projects/esp-
idf/en/stable/esp32/api-reference/system/app_image_format.html#application-description>
esp_bootloader_esp_idf::esp_app_desc!();

#[allow(
    clippy::large_stack_frames,
    reason = "it's not unusual to allocate larger buffers etc. in main"
)]
#[main]
fn main() -> ! {
    // generator version: 1.3.0
    // generator parameters: --chip esp32s3 -o unstable-hal -o log -o esp-backtrace -o
vscode

    esp_println::logger::init_logger_from_env();

    let config = esp_hal::Config::default().with_cpu_clock(CpuClock::max());
    let _peripherals = esp_hal::init(config);




    loop {
        info!("Hello world!");
        let delay_start = Instant::now();
        while delay_start.elapsed() < Duration::from_millis(500) {}
    }
```

```
    // for inspiration have a look at the examples at https://github.com/esp-rs/esp-hal/tree/esp-hal-v1.1.0/examples
}
```

Line / item	Meaning
`#![no_std]`, `#![no_main]`	No standard library; no default entry shim
`#![deny(clippy::mem_forget ...)]`	Lint: forbids `core::mem::forget`, unsafe for hardware drivers
`#![deny(clippy::large_stack_frames)]`	Lint: warns about big stack frames (limit from `.clippy.toml`)
`use esp_backtrace as _;`	Linking the panic handler (unused name, `as _`)
`esp_bootloader_esp_idf::esp_app_desc!();`	Embeds the app descriptor
`#[main] fn main() -> !`	Entry point; never returns
`esp_println::logger::init_logger_from_env()`	Starts the log backend (level from `ESP_LOG`)
`Config::default().with_cpu_clock(CpuClock::max())`	System configuration: 240 MHz CPU
`esp_hal::init(config)`	Initialises the chip and returns the peripherals
`Instant`/`Duration` busy-wait	Blocks for 500 ms without a timer driver

5.7 `src/lib.rs`

The generator creates an empty library (`#![no_std]`). The lab project puts shared code in it (`board.rs`, `lcd1602.rs`) so that every binary can use it with `use esp32s3_labs::...`.

Chapter 6 - Firmware entry point, panic handler and peripheral initialisation

6.1 What happens at power-up

1. **ROM bootloader** (inside the chip) reads the strapping pins (chapter 7), decides between *download mode* and *normal boot*, and loads the second-stage bootloader from flash.
2. **Second-stage bootloader** (flashed by `espflash`) checks the partition table, finds the application image and verifies its descriptor. The `esp_app_desc!()` macro in your program exists for this step.
3. **Start-up code** from the runtime crate behind esp-hal sets up the stack, initialises `.data` and `.bss`, installs exception vectors and calls the function marked `#[main]`.
4. **`esp_hal::init(config)`**, called first in `main`, applies the configuration (CPU clock), **disables the RWDT, super watchdog and TIMG0/TIMG1 watchdogs**, and returns the `Peripherals` singleton struct.
5. Your code runs. With `esp-rtos` the sequence adds `esp_rtos::start(..)` which starts the scheduler and the Embassy time driver; `#[esp_rtos::main]` then runs `async fn main` on the Embassy executor.

6.2 The panic handler

Rust requires a `#[panic_handler]` in every `no_std` binary. `esp-backtrace` provides it (feature `panic-handler`) and, with the `println` feature, prints the panic message and a backtrace through the console. The esp-hal documentation shows the minimal alternative of a custom handler that calls `esp_hal::system::software_reset()`. Choose by policy: a laboratory wants the message; a product usually resets.

6.3 Peripheral initialisation pattern

```
let peripherals = esp_hal::init(esp_hal::Config::default());   // once, first
let mut led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());
let uart = Uart::new(peripherals.UART1, Config::default())?.with_tx(peripherals.GPIO47);
```

Each driver takes the tokens it needs by value. The same pattern is used throughout Part II.

Chapter 7 - The board pin map and GPIO conflicts

This chapter is the foundation of every wiring table in the manual. Its main rule: **a GPIO must never be driven by two active peripherals at the same time**. Rust's ownership model (chapter 2) guarantees this *inside your program*; it cannot know about the board, so the board-level conflicts below must be checked by you.

7.1 Fixed connections given for this project

OV2640 camera (uses 14 GPIOs, always reserved):

Camera signal	GPIO	Camera signal	GPIO
XCLK	15	D4	12
SIOD (SCCB data)	4	D5	18
SIOC (SCCB clock)	5	D6	17
D0	11	D7	16
D1	9	VSYNC	6
D2	8	HREF	7
D3	10	PCLK	13
PWDN / RESET	unused		

LCD1602 over I2C: SDA = GPIO41, SCL = GPIO40, address 0x27. These two pins form the board's I2C bus (I2C0 in the labs).

7.2 Complete GPIO classification

The pin attributes come from `esp-metadata/devices/esp32s3.toml` of the esp-hal tag (strapping, flash/PSRAM and analog channel information). Items marked *board-dependent* must be confirmed in your board's schematic.

GPIO	Function / restriction	Status in this manual
0	Strapping (boot mode). Usually the BOOT button	Avoid
1	ADC1_CH0, touch 1	**Free**: ADC input (Labs 04, Projects 1, 3, LED in Project 2)
2	ADC1_CH1, touch 2	**Free**: ADC input (Lab 04, Project 3)
3	Strapping (JTAG source), ADC1_CH2	Avoid
4, 5	Camera SIOD, SIOC (ADC1_CH3, CH4)	**Camera**
6, 7	Camera VSYNC, HREF (ADC1_CH5, CH6)	**Camera**
8, 9, 10	Camera D2, D1, D3 (ADC1_CH7-CH9)	**Camera**
11, 12, 13	Camera D0, D4, PCLK (ADC2)	**Camera**
14	ADC2_CH3, touch 14	**Free**: button input
15, 16, 17, 18	Camera XCLK, D7, D6, D5 (ADC2)	**Camera**
19, 20	USB D-, D+ (USB Serial/JTAG and USB OTG)	Reserved for USB
21	General purpose	**Free**: LED / PWM / SPI CS / TWAI
26-32	SPI flash and PSRAM lines (per the esp-hal pin table: 26 PSRAM; 27-31 flash and PSRAM; 32 flash)	Not available

GPIO	Function / restriction	Status in this manual
33-37	Octal flash / octal PSRAM lines	Not available when the module has octal PSRAM (R8 variants)
38	General purpose	**Free**: SPI MOSI, encoder, MCPWM
39	General purpose (JTAG MTCK)	**Free**: SPI SCK, encoder, MCPWM
40	LCD SCL (JTAG MTDO)	**LCD**
41	LCD SDA (JTAG MTDI)	**LCD**
42	General purpose (JTAG MTMS)	**Free**: SPI MISO, UART RX
43, 44	UART0 TX, RX (boot console on boards with a UART bridge)	Leave for the console
45, 46	Strapping (VDD_SPI voltage, boot message / ROM log)	Avoid
47	General purpose	**Free**: servo, UART TX (see 1.8 V note)
48	General purpose; WS2812 RGB LED on many boards	**Free**: RMT / WS2812 (see 1.8 V note)

Free pool for peripherals other than the camera and LCD: GPIO1, 2, 14, 21, 38, 39, 42, 47, 48 (nine pins). The JTAG pins 39-42 are ordinary GPIOs unless the pad-JTAG interface is selected (by default JTAG uses the USB Serial/JTAG block).

7.3 Conflicts to check (and why)

1. **Camera versus ADC.** The camera consumes GPIO4-13 and 15-18. That removes seven of the ten ADC1 pins (GPIO4-10) and seven of the ten ADC2 pins (GPIO11-13 and 15-18). Only **GPIO1 and GPIO2** remain as usable ADC1 inputs (GPIO3 is a strapping pin), which is why the analog labs use them. ADC2 pins are unusable anyway while Wi-Fi runs.
2. **Camera versus PSRAM.** The camera needs frame buffers in PSRAM, so a PSRAM-equipped module is recommended. Octal PSRAM (R8) takes GPIO33-37 and flash/PSRAM take GPIO26-32; these are not available as GPIO regardless of the camera.
3. **USB.** GPIO19/20 are the USB pads. USB Serial/JTAG (console, flashing) and USB OTG (Lab 22b) use the *same* pads and cannot run together.
4. **Strapping pins** (0, 3, 45, 46) are sampled at reset; an external circuit that pulls them to the wrong level changes the boot mode or flash voltage.
5. **Console pins.** On boards with a USB-UART bridge, GPIO43/44 carry the console: do not reuse them unless you move logging.
6. **Board-level devices** that the chip cannot know about. Many S3 camera boards place a microSD socket on GPIO38/39/40, an RGB LED on GPIO48 (or GPIO38), a user LED, or the BOOT button on GPIO0. The LCD on GPIO40 given for this project would collide with such an SD socket; check your schematic.
7. **1.8 V pads (board-dependent).** On modules with 1.8 V flash/PSRAM supply (the "V" variants, e.g. R8V), the Espressif datasheet states that GPIO47 and GPIO48 are powered from VDD_SPI at 1.8 V. If your module is such a variant, move the servo/WS2812 signals to GPIO38/39/42. *This statement comes from the author's reading of the datasheet and is not verified in the source tree; confirm it for your module.*
8. **Peripheral-level conflicts.** `Mem2Mem` (Lab 14) consumes SPI2; ADC2 versus Wi-Fi; `esp-println auto` output versus USB Serial/JTAG use (Lab 22a).
9. **The LCD supply.** The pull-up and 5 V issue of the LCD backpack is explained in Lab 11: the bus must not rise above 3.3 V.

7.4 Pin usage audit generated from the source files

The next table is produced by a script that scans every file in `src/bin` for `peripherals.GPIOnn` and checks the pins against the rules above. `OK` means that the program uses no camera pin, no flash/PSRAM pin, no strapping pin and uses USB pins only in the USB labs.

Binary	GPIOs used by the program	Camera guard	Result
`lab00_hello`	none	-	OK
`lab01_gpio_output`	21	reserved	OK
`lab02_gpio_input`	14, 21	-	OK
`lab03_cpu_clock`	21	-	OK
`lab04_adc`	1, 2	-	OK
`lab05_ledc_pwm`	21	-	OK
`lab06_servo`	47	-	OK
`lab07_timer`	21	-	OK
`lab08_gpio_interrupt`	14, 21	-	OK
`lab09_uart`	42, 47	-	OK
`lab10_i2c_scan`	40, 41	-	OK
`lab11_lcd1602`	40, 41	reserved	OK
`lab12_spi`	21, 38, 39, 42	-	OK
`lab13_rc522`	14, 21, 38, 39, 42	-	OK
`lab14_dma_mem2mem`	none	-	OK
`lab14b_spi_dma`	21, 38, 39, 42	-	OK
`lab15_watchdog`	none	-	OK
`lab16_rmt_ws2812`	48	-	OK
`lab17_pcnt_encoder`	38, 39	-	OK
`lab18_mcpwm`	38, 39	-	OK
`lab19_twai`	21	-	OK
`lab20_wifi`	none	-	OK
`lab21_ble`	none	-	OK
`lab22a_usb_serial_jtag`	none	-	OK
`lab22b_usb_otg_cdc`	19, 20 (USB D+/D-, expected)	-	OK
`lab23_embassy`	14, 21	-	OK
`lab24_psram`	none	-	OK
`proj1_basic_monitor`	1, 21, 40, 41	reserved	OK
`proj2_rfid_access`	1, 14, 21, 38, 39, 40, 41, 42, 47	reserved	OK
`proj3_iot_monitor`	1, 2, 14, 21, 40, 41	reserved	OK

7.5 Enforcing the camera reservation in code

`reserve_camera_pins!(peripherals)` (chapter 2) turns the camera pin list into a type-level lock. The labs that touch the I2C bus or are likely to be extended (Labs 01 and 11 and all projects) call it first, so adding code that uses a camera pin later fails to compile.

Chapter 8 - Build, flash and monitor

8.1 Project layout and commands

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                                  # once per shell
```

```
cargo build --release --bin lab01_gpio_output          # compile only
cargo run   --release --bin lab01_gpio_output          # compile + flash + monitor
```

`cargo run` uses the `runner` from `.cargo/config.toml`, i.e. `espflash flash --monitor --chip esp32s3 <elf>`. The same operation by hand:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab01 gpio output
espflash board-info          # identify the chip, flash size, MAC (verifies the
connection)
espflash list-ports          # show serial ports
espflash monitor             # monitor without flashing
espflash erase-flash         # wipe the chip (recovery)
```

The subcommands `flash`, `monitor`, `board-info`, `list-ports`, `erase-flash`, `reset`, `save-image` are listed in the `espflash` README. **Always use `--release`** for these labs: unoptimised builds are large and some drivers (PSRAM) require it.

8.2 Choosing the console: `esp-println` output features

`esp-println` has exactly one active output feature. In this project the default `auto` is used.

Feature	Output goes to	Use when
`auto` (default)	Chooses UART0 or USB Serial/JTAG automatically	Most boards
`jtag-serial`	USB Serial/JTAG (GPIO19/20)	Board with only the native USB port
`uart`	UART0 (GPIO43/44)	USB-UART bridge boards, or when USB OTG is used (Lab 22b)
`no-op`	Nowhere	Silent firmware

To change it, edit the `esp-println` line in `Cargo.toml`, for example `default-features = false, features = ["esp32s3", "log-04", "uart"]`.

8.3 Download mode

If a board does not respond to `espflash`, hold the **BOOT** button, tap **RESET**, release BOOT (this makes the ROM bootloader read GPIO0 low and enter download mode), then run the flash command again. This is necessary after Lab 22b, because the application then owns the USB pins.

8.4 Reading the log

`ESP_LOG = "info"` in `.cargo/config.toml` sets the level at compile time. The monitor prints the ROM and bootloader banner, then your `info!`/`warn!`/`error!` lines. Exit the monitor with `Ctrl+C`.

Part II - ESP32-S3 peripheral laboratories

How every laboratory is organised

Each laboratory has the same ten sections: (1) objective, (2) required components, (3) wiring table, (4) hardware principle, (5) complete Rust source code, (6) explanation of every important function, method, type and parameter, (7) expected output or behaviour, (8) compilation and flashing commands, (9) common errors and troubleshooting, and (10) links to the esp-hal 1.1.1 documentation. A status block at the top gives the source file, the verification status (see the legend at the beginning of this manual) and the stability level of the APIs used.

Conventions

- All labs live in one Cargo package; lab *N* is the binary `src/bin/labNN_name.rs`.
- Run commands from the project root with the Xtensa environment sourced: `. "$HOME/export-esp.sh"`.
- Pin numbers are **GPIO numbers**, not board header positions. Board silkscreens differ: always confirm with your schematic (chapter 7).
- “Representative output” blocks show what the log lines look like; the exact prefix, colours and timestamps depend on `esp-println` features and are not a guarantee. Numeric values (timings, UIDs, IP addresses) are examples.
- Because behaviour was never observed on hardware, treat “Expected behaviour” as the *specification* of what the code is written to do, and report any difference as a bug in the lab or in your wiring.
- All logging uses the `log` macros through `esp-println`. Open the monitor with `cargo run` (chapter 8).

Lab 00 - Hello world, logging and the build pipeline

Item	Value
Source file	`src/bin/lab00_hello.rs`
Verification status	BUILD-VERIFIED (generator output, unchanged)
API stability	esp-hal root, `clock`, `time` (STABLE) + `log` output via esp-println
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Confirm that the toolchain, the generated project, the flashing tool and the serial monitor all work before touching any peripheral. You will build the file produced by `esp-generate`, flash it and read the log output.

2. Required components

- ESP32-S3 development board
- USB cable that carries data (not charge-only)
- Linux Mint PC with the toolchain from Part I

3. Wiring

No external wiring. Connect the board's USB port that your board routes to the programming interface (a UART bridge chip, or the native USB Serial/JTAG port on GPIO19/20).

4. Hardware principle

A bare-metal program has no operating system: after reset the ROM bootloader loads the second-stage bootloader from flash, which maps your image and jumps to the entry point generated by the `#[main]` macro. `esp_hal::init` configures clocks and returns the peripheral tokens. The loop prints a message every 500 ms through the `log` facade; `esp-println` implements the backend and writes bytes to the console device (UART0 or USB Serial/JTAG, selected automatically by its default `auto` feature).

5. Complete Rust source code

File `src/bin/lab00_hello.rs` (unmodified copy of the compiled file):

```
#![no_std]
#![no_main]
#![deny(
    clippy::mem_forget,
    reason = "mem::forget is generally not safe to do with esp_hal types, especially
those \
    holding buffers for the duration of a data transfer."
)]
#![deny(clippy::large_stack_frames)]

use esp_hal::{
    clock::CpuClock,
    main,
    time::{Duration, Instant},
};

use log::info;

use esp_backtrace as _;


// This creates a default app-descriptor required by the esp-idf bootloader.
// For more information see: <https://docs.espressif.com/projects/esp-
idf/en/stable/esp32/api-reference/system/app_image_format.html#application-description>
esp_bootloader_esp_idf::esp_app_desc!();
```

```rust
#[allow(
    clippy::large_stack_frames,
    reason = "it's not unusual to allocate larger buffers etc. in main"
)]
#[main]
fn main() -> ! {
    // generator version: 1.3.0
    // generator parameters: --chip esp32s3 -o unstable-hal -o log -o esp-backtrace -o vscode

    esp_println::logger::init_logger_from_env();

    let config = esp_hal::Config::default().with_cpu_clock(CpuClock::max());
    let _peripherals = esp_hal::init(config);




    loop {
        info!("Hello world!");
        let delay_start = Instant::now();
        while delay_start.elapsed() < Duration::from_millis(500) {}
    }

    // for inspiration have a look at the examples at https://github.com/esp-rs/esp-hal/tree/esp-hal-v1.1.0/examples
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`#![no_std]` / `#![no_main]`	No standard library and no compiler-generated `main`: the HAL provides the runtime and entry point.
`use esp_backtrace as _;`	Links the panic handler (feature `panic-handler`) which prints the panic message and a backtrace through `esp-println`.
`esp_bootloader_esp_idf::esp_app_desc!()`	Embeds the ESP-IDF application descriptor the second-stage bootloader expects.
`#[main] fn main() -> !`	Entry point. Must never return, hence `-> !`.
`esp_println::logger::init_logger_from_env()`	Installs the `log` backend; verbosity comes from the `ESP_LOG` variable fixed at build time in `.cargo/config.toml`.
`esp_hal::Config::default().with_cpu_clock(CpuClock::max())`	Builds the system configuration: run the CPU at the maximum preset (240 MHz on ESP32-S3).
`esp_hal::init(config)`	Applies the configuration and returns `Peripherals` (see Part III).
`Instant` / `Duration`	Busy-wait for 500 ms using the free-running system timer.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

After flashing, the monitor shows the ROM and bootloader banner followed by one `Hello world!` line about every half second (INFO level). Exact prefixes depend on the esp-println features.

Representative log lines:

```
INFO - Hello world!
INFO - Hello world!
INFO - Hello world!
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                          # once per shell
cargo build --release --bin lab00_hello      # compile only
cargo run   --release --bin lab00_hello      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab00_hello
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`error: could not find 'xtensa-esp32s3-none-elf'` / unknown target	The Xtensa toolchain is not active. Run `source ~/export-esp.sh` in this shell and check that `rust-toolchain.toml` says `channel = "esp"`.
`linker 'xtensa-esp32s3-elf-gcc' not found`	`~/export-esp.sh` was not sourced in the shell that runs cargo (the GCC bin directory is not on PATH).
`Error: Permission denied` / port not found when flashing	Add your user to the `dialout` group (`sudo usermod -aG dialout $USER`) and log in again; try another cable.
Flash succeeds but monitor stays empty	The board's console is wired to the other USB port or UART. Try the other connector, or build with a different esp-println output feature (Part I, chapter 8).
Boot loop printing `invalid header` / `No bootable app`	Image/bootloader mismatch or wrong chip. Use `espflash board-info`, then `espflash erase-flash` and flash again.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::clock
- esp_hal::time
- esp_hal::system
- Official esp-hal v1.1.1 examples

Lab 01 - GPIO digital output: blinking an LED

Item	Value
Source file	`src/bin/lab01_gpio_output.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `gpio::{Output, OutputConfig, Level}`,`time::{Instant, Duration}`, `esp_hal::main`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Drive a GPIO as a push-pull digital output, understand ownership of a pin token, and see how the camera pins are protected at compile time.

2. Required components

- 1 x LED (any colour)
- 1 x resistor 330 ohm (for 3.3 V and a standard LED, about 4 mA)
- Breadboard and jumper wires

3. Wiring

Signal / part	ESP32-S3 pin	Note
LED anode (long leg) via 330 ohm	GPIO21	Output, active high
LED cathode	GND	Common ground

4. Hardware principle

A GPIO output pad contains a push-pull driver: writing 1 connects the pin to 3.3 V through a P-channel transistor, writing 0 connects it to ground through an N-channel transistor. The LED must be current-limited: with a typical red LED (Vf about 2 V) and 330 ohm the current is (3.3-2.0)/330 = about 4 mA, well within the ESP32-S3 pad limits. In Rust the *token* `peripherals.GPIO21` represents exclusive access to the pad; `Output::new` consumes it, so nothing else can configure the same pad.

5. Complete Rust source code

File `src/bin/lab01_gpio_output.rs` (unmodified copy of the compiled file):

```
//! Lab 01 - GPIO digital output (LED blink)
//!
//! Wiring: GPIO21 --[330 ohm]--> LED anode, LED cathode --> GND.
//! API level: STABLE only (gpio, time, main). Works without the `unstable` feature.
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    gpio::{Level, Output, OutputConfig},
    main,
    time::{Duration, Instant},
};
use esp32s3_labs::reserve_camera_pins;
use log::info;

// App descriptor required by the ESP-IDF 2nd-stage bootloader.
esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
```

```
    esp_println::logger::init_logger_from_env();

    // Takes ownership of every peripheral exactly once.
    let peripherals = esp_hal::init(esp_hal::Config::default());

    // Ownership demo: the OV2640 pins are moved into `_camera` and can never be
    // claimed again by accident. Remove this line and the build still works;
    // keep it and any later use of e.g. `peripherals.GPIO15` is a compile error.
    let _camera = reserve_camera_pins!(peripherals);

    // `Output::new(pin, initial_level, config)` consumes the GPIO21 token.
    let mut led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());

    info!("Lab 01: blinking LED on GPIO21");
    loop {
        led.toggle();
        info!("LED is {}", if led.is_set_high() { "ON" } else { "OFF" });

        // Busy-wait for 500 ms using the free-running system timer.
        let start = Instant::now();
        while start.elapsed() < Duration::from_millis(500) {}
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`reserve_camera_pins!(peripherals)`	Moves the 14 OV2640 pin tokens into a struct that is never used again. Any later use of e.g. `peripherals.GPIO15` fails to compile (see chapter 7).
`Output::new(pin, Level::Low, OutputConfig::default())`	Consumes the GPIO21 token, sets the initial level *before* enabling the driver (no glitch), and applies default drive strength and push-pull mode.
`led.toggle()`	Inverts the output level.
`led.is_set_high()`	Reads back the level that the driver is commanding (not the electrical pad voltage).
`Instant::now()` / `.elapsed()`	Non-blocking style busy-wait: loop until 500 ms have elapsed.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The LED blinks at 1 Hz (500 ms on, 500 ms off) and the log prints `LED is ON` / `LED is OFF` alternately.

Representative log lines:

```
INFO - Lab 01: blinking LED on GPIO21
INFO - LED is ON
INFO - LED is OFF
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab01_gpio_output     # compile only
cargo run   --release --bin lab01_gpio_output     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab01_gpio_output
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy

Symptom	Likely cause and remedy
LED never lights	Reversed polarity (long leg to the pin), no common ground, or a different physical pin than GPIO21 - check the board's silkscreen, which often shows names, not numbers.
`error[E0382]: use of moved value: peripherals.GPIO21`	You used the same token twice. Each GPIO can be given to exactly one driver; see Part I chapter 2.
LED always on at boot	The pin is a strapping pin or driven by the bootloader. GPIO21 is not; if you change pins avoid 0, 3, 45, 46 (chapter 7).

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::gpio
- esp_hal::time
- esp-hal ownership model ('Peripheral singletons')
- Official esp-hal v1.1.1 examples

Lab 02 - GPIO digital input, pull-up / pull-down and debouncing

Item	Value
Source file	`src/bin/lab02_gpio_input.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `gpio::{Input, InputConfig, Pull, Level}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Read a push button reliably: choose the correct internal pull resistor, interpret active-low vs active-high logic, and filter contact bounce in software.

2. Required components

- 1 x tactile push button
- 1 x LED + 330 ohm (from Lab 01), optional
- Jumper wires

3. Wiring

Signal / part	ESP32-S3 pin	Note
Button terminal 1	GPIO14	Input with internal pull-up
Button terminal 2	GND	Pull-up mode (default)
(alternative) Button terminal 2	3V3	Use with `Pull::Down`
LED anode via 330 ohm	GPIO21	Mirrors the button

4. Hardware principle

A floating input picks up noise and reads random values. A pull resistor (about 45 kohm inside the chip) defines the idle level: `Pull::Up` idles high, so the button must connect the pin to GND (active low); `Pull::Down` idles low and the button connects to 3.3 V (active high). Mechanical contacts bounce for 1-20 ms, producing several edges per press. The lab accepts a new level only after it has stayed unchanged for 20 ms.

5. Complete Rust source code

File `src/bin/lab02_gpio_input.rs` (unmodified copy of the compiled file):

```
//! Lab 02 - GPIO digital input, pull-up / pull-down, software debounce
//!
//! Default wiring (PULL = Pull::Up):   GPIO14 --[push button]--> GND
//! Alternative    (PULL = Pull::Down):  GPIO14 --[push button]--> 3V3
//! LED on GPIO21 (see Lab 01) mirrors the button.
//! API level: STABLE only.
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    gpio::{Input, InputConfig, Level, Output, OutputConfig, Pull},
    main,
    time::{Duration, Instant},
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

/// Change to `Pull::Down` if the button connects the pin to 3V3.
```

```rust
const PULL: Pull = Pull::Up;
const DEBOUNCE: Duration = Duration::from_millis(20);

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    let mut led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());
    let button = Input::new(peripherals.GPIO14, InputConfig::default().with_pull(PULL));

    // With a pull-up the idle level is High and "pressed" is Low; with a
    // pull-down it is the opposite.
    let pressed_level = match PULL {
        Pull::Up => Level::Low,
        _ => Level::High,
    };

    info!("Lab 02: press the button on GPIO14 (pull = {:?})", PULL);

    let mut stable = button.level();
    let mut last_raw = stable;
    let mut last_change = Instant::now();

    loop {
        let raw = button.level();
        if raw != last_raw {
            last_raw = raw;
            last_change = Instant::now(); // restart the debounce window
        }
        // Accept a new level only after it stayed unchanged for DEBOUNCE.
        if raw != stable && last_change.elapsed() >= DEBOUNCE {
            stable = raw;
            if stable == pressed_level {
                info!("button PRESSED");
                led.set_high();
            } else {
                info!("button RELEASED");
                led.set_low();
            }
        }
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`InputConfig::default().with_pull(PULL)`	`InputConfig` is `#[non_exhaustive]`, so it is built with the generated builder method `with_pull`.
`Input::new(peripherals.GPIO14, config)`	Consumes the pin token and configures the pad as an input.
`button.level()`	Samples the pin and returns `Level::Low` or `Level::High`.
`pressed_level`	Computed from `PULL`: Low for pull-up, High for pull-down.
Debounce state machine	`last_raw`/`last_change` restart a 20 ms window on every raw edge; `stable` changes only when the window has elapsed.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Pressing the button prints `button PRESSED` and lights the LED; releasing prints `button RELEASED`. One message per press, not a burst.

Representative log lines:

```
INFO - Lab 02: press the button on GPIO14 (pull = Up)
INFO - button PRESSED
INFO - button RELEASED
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab02_gpio_input     # compile only
cargo run   --release --bin lab02_gpio_input     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab02_gpio_input
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Press and release both print `PRESSED`/random output	The pin is floating: pull resistor not enabled or button wired to the wrong terminals.
Always reads pressed	With `Pull::Up` the button leg must go to GND; check you did not wire it to 3V3.
Multiple events per press	Debounce too short for your switch: raise `DEBOUNCE` to 40-50 ms.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::gpio
- esp_hal::time
- Official esp-hal v1.1.1 examples

Lab 03 - CPU clock configuration

Item	Value
Source file	`src/bin/lab03_cpu_clock.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `esp_hal::Config::with_cpu_clock`, `clock::{CpuClock, cpu_clock, xtal_clock}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Select the CPU frequency (80, 160 or 240 MHz), read it back, and measure the effect on execution time.

2. Required components

- ESP32-S3 board only
- Optional: oscilloscope or logic analyser on GPIO21

3. Wiring

Signal / part	ESP32-S3 pin	Note
Probe (optional)	GPIO21	Toggled continuously for 1 s per round

4. Hardware principle

The ESP32-S3 derives the CPU clock from the 40 MHz crystal through a PLL (320 MHz or 480 MHz) and dividers. esp-hal exposes three presets for the S3: 80 MHz (reset default), 160 MHz and 240 MHz. A fixed loop of integer operations takes time inversely proportional to the clock: 240 MHz is about three times faster than 80 MHz. The higher the clock, the more current the chip draws, and Wi-Fi/BLE require at least 80 MHz.

5. Complete Rust source code

File `src/bin/lab03_cpu_clock.rs` (unmodified copy of the compiled file):

```
//! Lab 03 - CPU clock configuration
//!
//! The CPU clock is chosen once, at `esp_hal::init()`. Select it at build time:
//!
//!   cargo run --release --bin lab03_cpu_clock --features clk-80
//!   cargo run --release --bin lab03_cpu_clock --features clk-160
//!   cargo run --release --bin lab03_cpu_clock --features clk-240
//!
//! Without a feature the example uses `CpuClock::max()` (240 MHz on ESP32-S3).
//! Optional: probe GPIO21 with a scope to see the toggle rate change.
//! API level: `esp_hal::clock` is STABLE.
#![no_std]
#![no_main]

use core::hint::black_box;

use esp_backtrace as _;
use esp_hal::{
    clock::{CpuClock, cpu_clock, xtal_clock},
    gpio::{Level, Output, OutputConfig},
    main,
    time::{Duration, Instant},
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();
```

```rust
/// Pick the clock preset from Cargo features.
const fn selected_clock() -> CpuClock {
    if cfg!(feature = "clk-80") {
        CpuClock::_80MHz
    } else if cfg!(feature = "clk-160") {
        CpuClock::_160MHz
    } else if cfg!(feature = "clk-240") {
        CpuClock::_240MHz
    } else {
        CpuClock::max()
    }
}

/// A fixed amount of integer work. `black_box` stops the optimiser from
/// folding the loop into a constant.
fn workload() -> u32 {
    let mut acc: u32 = 0;
    for i in 0..1_000_000u32 {
        acc = black_box(acc.wrapping_mul(31).wrapping_add(i));
    }
    acc
}

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();

    let config = esp_hal::Config::default().with_cpu_clock(selected_clock());
    let peripherals = esp_hal::init(config);

    let mut probe = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());

    info!("Requested preset : {:?}", selected_clock());
    info!("CPU clock        : {} MHz", cpu_clock().as_mhz());
    info!("XTAL clock       : {} MHz", xtal_clock().as_mhz());

    loop {
        let t0 = Instant::now();
        let result = workload();
        let us = t0.elapsed().as_micros();
        info!("1,000,000 iterations took {} us (checksum {})", us, result);

        // Toggle the probe pin as fast as the loop allows for ~1 s.
        let t1 = Instant::now();
        let mut toggles: u32 = 0;
        while t1.elapsed() < Duration::from_secs(1) {
            probe.toggle();
            toggles += 1;
        }
        info!("probe pin toggled {} times in 1 s", toggles);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
Cargo features `clk-80`, `clk-160`, `clk-240`	Defined in `Cargo.toml`; select the preset at build time with `--features`.
`CpuClock::_80MHz` / `_160MHz` / `_240MHz` / `max()`	The only S3 presets in 1.1.1. `max()` returns the highest.
`with_cpu_clock(...)`	Builder on `esp_hal::Config`; the argument is a `ClockConfig` into which a `CpuClock` converts.
`cpu_clock()` / `xtal_clock()`	Return a `Rate`; `as_mhz()` converts to MHz.
`black_box`	Prevents the optimiser from deleting the benchmark loop.
`probe.toggle()` loop	Counts toggles in one second; on a scope the pin frequency scales with the CPU clock.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Three log lines at start (requested preset, CPU MHz, XTAL MHz), then the time for 1,000,000 iterations (roughly three times shorter at 240 MHz than at 80 MHz) and the toggle count per second.

Representative log lines:

```
INFO - Requested preset : _240MHz
INFO - CPU clock        : 240 MHz
INFO - XTAL clock       : 40 MHz
INFO - 1,000,000 iterations took ... us
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                        # once per shell
cargo build --release --bin lab03_cpu_clock     # compile only
cargo run   --release --bin lab03_cpu_clock     # compile, flash, open monitor
```

Variants for this lab:

```
cargo run --release --bin lab03_cpu_clock --features clk-80
cargo run --release --bin lab03_cpu_clock --features clk-160
cargo run --release --bin lab03_cpu_clock --features clk-240
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab03_cpu_clock
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`unexpected cfg`/feature not found	Features must be passed to the package that defines them: run from the project root with `--features clk-160`.
Numbers do not change between presets	cargo reused an old binary: add `--release` consistently and check the printed `Requested preset` line.
Serial garbage after changing the clock	UART baud rates are re-derived by the HAL; if you use an external console tool, reset the board and reopen the monitor.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::clock
- TRM chapter ‘Reset and Clock’ (Espressif documentation portal)
- Official esp-hal v1.1.1 examples

Lab 04 - ADC analog-to-digital conversion

Item	Value
Source file	`src/bin/lab04_adc.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature (confirmed: fails without it, see chapter 2)
API stability	UNSTABLE: `analog::adc::{Adc, AdcConfig, Attenuation, AdcCalCurve}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Measure an analog voltage with the 12-bit SAR ADC, convert it to millivolts using the factory calibration, and reduce noise by oversampling.

2. Required components

- 1 x 10 kohm potentiometer
- Jumper wires

3. Wiring

Signal / part	ESP32-S3 pin	Note
Potentiometer end A	3V3	
Potentiometer end B	GND	
Wiper (middle)	GPIO1	ADC1_CH0 (calibrated, mV)
Optional second input	GPIO2	ADC1_CH1 (raw counts)

4. Hardware principle

A successive-approximation (SAR) ADC compares the input against a binary-weighted reference: 12 bits give 4096 levels. The input stage can attenuate the signal; `Attenuation::_11dB` lets the ADC read roughly 0 to 3.1 V instead of 0 to 0.9 V. Each chip is characterised at the factory and the calibration constants are stored in eFuse; `AdcCalCurve` applies them so the result is in millivolts. ADC2 shares hardware with Wi-Fi and cannot be used while Wi-Fi runs, so analog inputs in this manual use ADC1 pins: GPIO1 to GPIO10. GPIO4-10 are camera pins, so GPIO1 and GPIO2 are the free ADC1 pins (GPIO3 is a strapping pin).

5. Complete Rust source code

File `src/bin/lab04_adc.rs` (unmodified copy of the compiled file):

```
//! Lab 04 - ADC (analog-to-digital conversion)
//!
//! Wiring: potentiometer 10 k: one end 3V3, other end GND, wiper -> GPIO1 (ADC1_CH0).
//!         Optional second input on GPIO2 (ADC1_CH1), read as raw counts.
//! Use ADC1 pins (GPIO1..GPIO10): ADC2 is unusable while Wi-Fi is running.
//! GPIO4..GPIO10 are taken by the camera, so GPIO1/GPIO2 are the free ADC1 pins.
//! API level: UNSTABLE (`esp_hal::analog::adc` requires the `unstable` feature).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
    main,
    peripherals::ADC1,
    time::{Duration, Instant},
};
use log::info;
```

```rust
esp_bootloader_esp_idf::esp_app_desc!();

const SAMPLES: u32 = 16;

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    let mut adc_config = AdcConfig::new();

    // GPIO1: 11 dB attenuation (~0 .. 3.1 V) with the factory curve calibration
    // stored in eFuse -> readings are in millivolts.
    let mut pin_mv =
        adc_config.enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(peripherals.GPIO1,
Attenuation:: 11dB);
    // GPIO2: no calibration -> raw 12-bit counts (0..=4095).
    let mut pin_raw = adc_config.enable_pin(peripherals.GPIO2, Attenuation::_11dB);

    let mut adc = Adc::new(peripherals.ADC1, adc_config);

    loop {
        // Oversample to reduce noise.
        let mut sum_mv: u32 = 0;
        let mut sum_raw: u32 = 0;
        for _ in 0..SAMPLES {
            // read_oneshot() returns nb::Result<u16, ()>; WouldBlock means
            // "conversion not finished yet", block! simply retries.
            sum_mv += nb::block!(adc.read_oneshot(&mut pin_mv)).unwrap() as u32;
            sum_raw += adc.read_blocking(&mut pin_raw) as u32;
        }
        let mv = sum_mv / SAMPLES;
        let raw = sum_raw / SAMPLES;

        info!("GPIO1 = {} mV   |   GPIO2 raw = {} / 4095", mv, raw);

        let t = Instant::now();
        while t.elapsed() < Duration::from_millis(500) {}
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`AdcConfig::new()`	Empty configuration; pins are registered one by one.
`enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(pin, Attenuation::_11dB)`	Registers GPIO1 with a calibration scheme; the returned `AdcPin` yields millivolts.
`enable_pin(pin, attenuation)`	Same without calibration: raw 0-4095 counts.
`Adc::new(peripherals.ADC1, adc_config)`	Creates the blocking ADC driver and consumes the `ADC1` token.
`nb::block!(adc.read_oneshot(&mut pin))`	`read_oneshot` returns `nb::Result<u16, ()>`; `WouldBlock` means 'conversion not finished', `block!` retries until done.
`adc.read_blocking(&mut pin)`	Convenience variant that returns `u16` directly.
Oversampling (16 samples)	Averaging N samples reduces random noise by roughly sqrt(N).

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Twice per second one line with the GPIO1 voltage in millivolts and the GPIO2 raw count out of 4095. Turning the knob sweeps GPIO1 from about 0 to about 3100 mV.

Representative log lines:

```
INFO - GPIO1 = 1648 mV   |   GPIO2 raw = 2045 / 4095
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab04_adc      # compile only
cargo run   --release --bin lab04_adc      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab04_adc
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`E0603: module 'analog' is private`	The `unstable` feature of esp-hal is missing. The real compiler output is shown in chapter 2; enable `features = ["unstable"]`.
Reading stuck at 4095 or ~3100 mV	Input above the attenuation range: reduce the source voltage or use a higher attenuation.
Noisy values	Add a 100 nF capacitor from the pin to ground and keep wires short; increase oversampling.
`pin does not implement AdcChannel`	The pin is not an ADC1/ADC2 channel of the unit you chose; see the table in chapter 7.
Readings wrong while Wi-Fi is on	You used an ADC2 pin (GPIO11-20). Move the signal to ADC1.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::analog (unstable)
- Official esp-hal v1.1.1 examples

Lab 05 - LEDC PWM: LED brightness control

Item	Value
Source file	`src/bin/lab05_ledc_pwm.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `ledc::{Ledc, LowSpeed, timer, channel}`, `delay::Delay`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Generate a hardware PWM signal and use it to dim an LED with fades and a gamma-corrected ramp.

2. Required components

- 1 x LED
- 1 x resistor 330 ohm

3. Wiring

Signal / part	ESP32-S3 pin	Note
LED anode via 330 ohm	GPIO21	PWM output
LED cathode	GND	

4. Hardware principle

PWM (pulse-width modulation) switches the output on and off faster than the eye can follow; the average power is proportional to the duty cycle (the fraction of the period spent high). The LEDC peripheral contains timers that count from 0 to 2^N-1 (N is the duty resolution) at the PWM frequency times 2^N, and channels that compare the counter with a duty register to set or clear the output. The ESP32-S3 provides 4 timers and 8 channels in low-speed mode. A hardware fade engine steps the duty register automatically. Human brightness perception is roughly logarithmic, so a squared (gamma about 2) duty curve looks more linear than a linear ramp.

5. Complete Rust source code

File `src/bin/lab05_ledc_pwm.rs` (unmodified copy of the compiled file):

```
//! Lab 05 - LEDC PWM: LED brightness control
//!
//! Wiring: GPIO21 --[330 ohm]--> LED --> GND.
//! The LEDC peripheral generates PWM in hardware; the CPU only writes duty values.
//! API level: UNSTABLE (`esp_hal::ledc`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    gpio::DriveMode,
    ledc::{
        LSGlobalClkSource, Ledc, LowSpeed,
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
    },
    main,
    time::Rate,
};
use log::info;
```

```rust
esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    let mut ledc = Ledc::new(peripherals.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);

    // 1) A timer defines frequency and resolution (shared by up to 4 channels).
    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            duty: timer::config::Duty::Duty10Bit, // 1024 duty steps
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_khz(5), // 5 kHz: flicker-free for the eye
        })
        .unwrap();

    // 2) A channel binds a timer to an output pin.
    let mut channel0 = ledc.channel(channel::Number::Channel0, peripherals.GPIO21);
    channel0
        .configure(channel::config::Config {
            timer: &timer0,
            duty_pct: 0,
            drive_mode: DriveMode::PushPull,
        })
        .unwrap();

    info!("Lab 05: breathing LED on GPIO21");
    loop {
        // Hardware fade 0 % -> 100 % in 1500 ms, then back.
        channel0.start_duty_fade(0, 100, 1500).unwrap();
        while channel0.is_duty_fade_running() {}
        channel0.start_duty_fade(100, 0, 1500).unwrap();
        while channel0.is_duty_fade_running() {}

        // Manual stepping with a square-law "gamma" curve: perceived brightness
        // is roughly logarithmic, so duty = x^2 looks more linear than duty = x.
        for step in 0..=100u32 {
            channel0.set_duty((step * step / 100) as u8).unwrap();
            delay.delay_millis(15);
        }
        info!("cycle finished");
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Ledc::new(peripherals.LEDC)`	Creates the controller and enables its clock.
`set_global_slow_clock(LSGlobalClkSource::APBClk)`	Selects the 80 MHz APB clock for the low-speed timers.
`ledc.timer::<LowSpeed>(timer::Number::Timer0)`	Obtains timer 0.
`timer.configure(Config { duty: Duty10Bit, clock_source: APBClk, frequency: 5 kHz })`	Computes the clock divider so one period is 1024 steps at 5 kHz; returns an error if the divider is out of range.
`ledc.channel(Number::Channel0, GPIO21)`	Binds channel 0 to the pin.
`channel.configure(Config { timer, duty_pct, drive_mode })`	Connects the channel to the timer with an initial duty and push-pull output.
`start_duty_fade(0, 100, 1500)`	Fades from 0 % to 100 % in 1500 ms in hardware.
`is_duty_fade_running()`	Polled until the fade finishes.

Item	Explanation
`set_duty(pct)`	Immediate duty in whole percent.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The LED breathes up and down over 3 s per cycle, then ramps up smoothly with a gamma curve. The log prints `cycle finished` after each round.

Representative log lines:

```
INFO - Lab 05: breathing LED on GPIO21
INFO - cycle finished
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab05_ledc_pwm      # compile only
cargo run   --release --bin lab05_ledc_pwm      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab05_ledc_pwm
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`E0603 ... ledc is private`	`unstable` feature missing.
`Err(Divisor)` from `timer.configure(..)`	Frequency and resolution are incompatible (the clock divider would be out of range): lower the frequency or the number of duty bits. `Err(FrequencyUnset)` means the `frequency` field was never set.
`Err(Duty)` from `channel.configure(..)` or `set_duty(..)`	The percentage was above 100.
LED fully on or off, no dimming	`duty_pct` was 0 or 100, or the pin is also used by another driver.
Visible flicker	Frequency too low: use at least 1 kHz for LEDs.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::ledc
- Official esp-hal v1.1.1 examples

Lab 06 - Servo motor control with PWM

Item	Value
Source file	`src/bin/lab06_servo.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `ledc`; embedded-hal `pwm::SetDutyCycle` trait
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Position a hobby servo by generating a 50 Hz PWM signal whose pulse width encodes the angle.

2. Required components

- 1 x SG90 or MG90S hobby servo
- External 5 V supply (500 mA or more) or a USB 5 V rail that is not the ESP32's 3V3 pin
- Jumper wires
- Optional: 470 uF capacitor across the servo supply

3. Wiring

Signal / part	ESP32-S3 pin	Note
Servo signal (orange / white)	GPIO47	PWM, 3.3 V logic is accepted by common micro servos
Servo +5 V (red)	External 5 V	NOT from 3V3; servos draw current spikes
Servo GND (brown / black)	External GND AND ESP32 GND	Common ground is mandatory

4. Hardware principle

Hobby servos expect a pulse every 20 ms (50 Hz). The pulse width selects the angle: about 0.5 ms for 0 degrees, 1.5 ms for 90 degrees and 2.5 ms for 180 degrees (check your servo's datasheet; many are specified for 1-2 ms). With a 14-bit LEDC timer at 50 Hz the period is 16384 counts, so one count is about 1.22 microseconds, giving a fine resolution of well under one degree. The code uses the embedded-hal `SetDutyCycle` interface: `max_duty_cycle()` returns 16384 and `set_duty_cycle(n)` sets the high time to n counts.

Power: a servo can draw several hundred milliamps when it moves. Powering it from the board's 3V3 regulator causes brown-outs and resets.

5. Complete Rust source code

File `src/bin/lab06_servo.rs` (unmodified copy of the compiled file):

```
//! Lab 06 - Servo motor control with PWM (LEDC, 50 Hz)
//!
//! Wiring (SG90 / MG90S class hobby servo):
//!   servo signal (orange/white) -> GPIO47
//!   servo +5 V   (red)          -> EXTERNAL 5 V supply (NOT the 3V3 pin)
//!   servo GND    (brown/black)  -> supply GND AND ESP32-S3 GND (common ground!)
//!
//! A hobby servo wants one pulse every 20 ms; the pulse width sets the angle:
//!   0.5 ms ~ 0 deg, 1.5 ms ~ 90 deg, 2.5 ms ~ 180 deg (typical; check your datasheet).
//! API level: UNSTABLE (`esp_hal::ledc`). Uses the embedded-hal `SetDutyCycle` trait.
#![no_std]
#![no_main]

use embedded_hal::pwm::SetDutyCycle;
```

```rust
use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    gpio::DriveMode,
    ledc::{
        LSGlobalClkSource, Ledc, LowSpeed,
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
    },
    main,
    time::Rate,
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

const PERIOD_US: u32 = 20_000; // 50 Hz
const MIN_US: u32 = 500; // pulse for 0 degrees
const MAX_US: u32 = 2_500; // pulse for 180 degrees

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    let mut ledc = Ledc::new(peripherals.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);

    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            // 14 bits = 16384 steps per 20 ms period -> ~1.2 us resolution.
            duty: timer::config::Duty::Duty14Bit,
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_hz(50),
        })
        .unwrap();

    let mut servo = ledc.channel(channel::Number::Channel0, peripherals.GPIO47);
    servo
        .configure(channel::config::Config {
            timer: &timer0,
            duty_pct: 0,
            drive_mode: DriveMode::PushPull,
        })
        .unwrap();

    // embedded-hal: duty is expressed against max_duty_cycle() (= 2^14 here).
    let max = servo.max_duty_cycle() as u32;
    info!("max duty = {} counts per {} us period", max, PERIOD_US);

    let angle_to_ticks = |deg: u32| -> u16 {
        let pulse_us = MIN_US + (MAX_US - MIN_US) * deg.min(180) / 180;
        (pulse_us * max / PERIOD_US) as u16
    };

    loop {
        for deg in (0..=180).step_by(30) {
            servo.set_duty_cycle(angle_to_ticks(deg)).unwrap();
            info!("angle = {} deg", deg);
            delay.delay_millis(600);
        }
        for deg in (0..=180).rev().step_by(30) {
            servo.set_duty_cycle(angle_to_ticks(deg)).unwrap();
            info!("angle = {} deg", deg);
            delay.delay_millis(600);
        }
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Duty14Bit` + `Rate::from_hz(50)`	Highest practical resolution at 50 Hz: the divider (80 MHz / (50 x 16384)) is about 97.7, within the divider range.
`servo.max_duty_cycle()`	embedded-hal method; returns 2^14 = 16384.
`angle_to_ticks`	`pulse_us = 500 + 2000 * deg / 180` then `ticks = pulse_us * max / 20000`.
`servo.set_duty_cycle(ticks)`	Writes the hardware duty value; values above max are clamped.
`(0..=180).step_by(30)` / `.rev()`	Sweeps 0 to 180 and back in 30 degree steps.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The servo horn moves in 30 degree steps from 0 to 180 degrees and back, pausing 600 ms at each position, while the log prints the angle.

Representative log lines:

```
INFO - max duty = 16384 counts per 20000 us period
INFO - angle = 0 deg
INFO - angle = 30 deg
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                      # once per shell
cargo build --release --bin lab06_servo     # compile only
cargo run   --release --bin lab06_servo     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab06_servo
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Servo jitters or the board resets	Insufficient supply current or missing common ground; add a bulk capacitor.
Servo hits its end stops and buzzes	Your servo's real range is narrower: reduce `MIN_US`/`MAX_US` to 1000/2000.
`Err(Divisor)` from `timer.configure(..)`	14 bits at 50 Hz needs a divider of about 97.7 from the 80 MHz APB clock; keep `LSClockSource::APBClk`.
No movement	Signal wire on the wrong pin, or GPIO47 is unavailable on your module (1.8 V SPI supply variants, see chapter 7).

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::ledc
- embedded-hal pwm::SetDutyCycle
- Official esp-hal v1.1.1 examples

Lab 07 - Hardware timers and periodic events

Item	Value
Source file	`src/bin/lab07_timer.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `timer::{PeriodicTimer, timg::TimerGroup}`, `#[handler]`; STABLE `#[ram]`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Use a hardware timer to raise an interrupt every 250 ms, toggling an LED from an interrupt service routine (ISR) while the main loop does other work.

2. Required components

- 1 x LED + 330 ohm

3. Wiring

Signal / part	ESP32-S3 pin	Note
LED anode via 330 ohm	GPIO21	Toggled by the ISR
LED cathode	GND	

4. Hardware principle

A general-purpose timer counts clock ticks; when the counter reaches an alarm value it raises an interrupt and (in periodic mode) reloads. The CPU stops what it is doing, jumps to the ISR, then resumes. Data shared between the ISR and `main` must be protected: the lab stores the timer and the LED in `static` items wrapped in a `critical_section::Mutex<RefCell<Option<..>>>` (interrupts are disabled while the lock is held) and the counter in an atomic. The ISR is placed in IRAM with `#[ram]` so it can run while the flash cache is busy.

5. Complete Rust source code

File `src/bin/lab07_timer.rs` (unmodified copy of the compiled file):

```
//! Lab 07 - Hardware timers and periodic events (TIMG0 timer + interrupt)
//!
//! Wiring: GPIO21 --[330 ohm]--> LED --> GND (toggled from the timer interrupt).
//! The CPU is free to do other work (here: print a counter) while the timer
//! peripheral raises an interrupt every 250 ms.
//! API level: UNSTABLE (`esp_hal::timer`, `esp_hal::handler`).
#![no_std]
#![no_main]

use core::{
    cell::RefCell,
    sync::atomic::{AtomicU32, Ordering},
};

use critical_section::Mutex;
use esp_backtrace as _;
use esp_hal::{
    Blocking,
    gpio::{Level, Output, OutputConfig},
    handler,
    main, ram,
    time::{Duration, Instant},
    timer::{PeriodicTimer, timg::TimerGroup},
};
use log::info;
```

```rust
esp_bootloader_esp_idf::esp_app_desc!();

// Resources shared between `main` and the interrupt handler live in statics
// behind a critical-section Mutex<RefCell<Option<..>>>.
static TIMER: Mutex<RefCell<Option<PeriodicTimer<'static, Blocking>>>> =
    Mutex::new(RefCell::new(None));
static LED: Mutex<RefCell<Option<Output<'static>>>> = Mutex::new(RefCell::new(None));
static TICKS: AtomicU32 = AtomicU32::new(0);

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    let led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());

    // A timer group contains two general-purpose timers and a watchdog.
    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let mut periodic = PeriodicTimer::new(timg0.timer0);

    critical_section::with(|cs| {
        periodic.set_interrupt_handler(on_timer); // register the ISR
        periodic.listen(); // unmask the timer interrupt
        periodic.start(Duration::from_millis(250)).unwrap(); // auto-reload
        TIMER.borrow_ref_mut(cs).replace(periodic);
        LED.borrow_ref_mut(cs).replace(led);
    });

    info!("Lab 07: timer interrupt every 250 ms");
    let mut last = 0;
    loop {
        let t = Instant::now();
        while t.elapsed() < Duration::from_millis(1000) {}
        let now = TICKS.load(Ordering::Relaxed);
        info!("ticks so far: {} (+{} in the last second)", now, now - last);
        last = now;
    }
}

/// Interrupt service routine. `#[ram]` keeps it in IRAM so it can run even
/// while flash cache is busy.
#[handler]
#[ram]
fn on_timer() {
    critical_section::with(|cs| {
        // 1) Acknowledge the interrupt, otherwise it fires again immediately.
        TIMER.borrow_ref_mut(cs).as_mut().unwrap().clear_interrupt();
        // 2) Do the (short!) work.
        LED.borrow_ref_mut(cs).as_mut().unwrap().toggle();
    });
    TICKS.fetch_add(1, Ordering::Relaxed);
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`TimerGroup::new(peripherals.TIMG0)`	TIMG0 contains `timer0`, `timer1` and a watchdog.
`PeriodicTimer::new(timg0.timer0)`	Wraps the timer as an auto-reloading periodic timer (`Blocking` mode).
`set_interrupt_handler(on_timer)`	Registers the ISR; `on_timer` was produced by `#[handler]`.
`listen()`	Unmasks the timer interrupt.
`start(Duration::from_millis(250))`	Programs the alarm and starts counting.
`static TIMER: Mutex<RefCell<Option<...>>>`	Pattern to move a driver into a static so the ISR can reach it.
`clear_interrupt()` in the ISR	Acknowledges the interrupt; without it the ISR re-enters forever.

Item	Explanation
`AtomicU32::fetch_add`	Lock-free counter shared with `main`.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The LED toggles every 250 ms (2 Hz blink) with no blocking in `main`; once per second the log prints the number of ticks (about 4 per second).

Representative log lines:

```
INFO - Lab 07: timer interrupt every 250 ms
INFO - ticks so far: 4 (+4 in the last second)
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab07_timer      # compile only
cargo run   --release --bin lab07_timer      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab07_timer
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Board freezes right after `listen()`	The ISR did not clear the interrupt, or it panicked (e.g. `unwrap()` on a `None` static because the timer was moved in after `start`).
`warning: unused import: InterruptConfigurable`	Seen while developing this lab: `PeriodicTimer` has an inherent `set_interrupt_handler`, so the trait import is unnecessary here (it is only needed for generic code).
Ticks drift	Busy waiting in `main` is only used for printing; the timer itself is hardware-accurate.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::timer
- esp_hal::interrupt
- Official esp-hal v1.1.1 examples

Lab 08 - GPIO interrupts and interrupt handlers

Item	Value
Source file	`src/bin/lab08_gpio_interrupt.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `gpio::Io`, `Input::{listen, clear_interrupt, is_interrupt_set}`, `Event`, `#[handler]`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

React to a button press with an interrupt instead of polling, using a minimal ISR that debounces and signals the main loop.

2. Required components

- 1 x push button
- 1 x LED + 330 ohm

3. Wiring

Signal / part	ESP32-S3 pin	Note
Button	GPIO14 to GND	Internal pull-up enabled, falling edge = press
LED anode via 330 ohm	GPIO21	Toggled by `main` on each valid press

4. Hardware principle

A GPIO can raise an interrupt on a rising edge, a falling edge, either edge, or a low/high level. On ESP32-S3 all GPIO interrupts share one CPU interrupt line, so there is one handler for all pins and it must check which pin fired. An ISR should do the minimum: here it timestamps, debounces (50 ms), increments a counter and sets a flag; `main` reads the flag and does the slow work (logging, toggling the LED). Variables written in the ISR are `Atomic*` so `main` can read them safely without locking.

5. Complete Rust source code

File `src/bin/lab08_gpio_interrupt.rs` (unmodified copy of the compiled file):

```
//! Lab 08 - GPIO interrupts and interrupt handlers
//!
//! Wiring: push button between GPIO14 and GND (internal pull-up is enabled),
//!         LED on GPIO21 (330 ohm to GND) toggles on each valid press.
//! The ISR does only the minimum (debounce, counter, flag); logging is done in `main`.
//! API level: UNSTABLE (`Io::set_interrupt_handler`, `Input::listen`, ...).
#![no_std]
#![no_main]

use core::{
    cell::RefCell,
    sync::atomic::{AtomicBool, AtomicU32, Ordering},
};

use critical_section::Mutex;
use esp_backtrace as _;
use esp_hal::{
    gpio::{Event, Input, InputConfig, Io, Level, Output, OutputConfig, Pull},
    handler, main, ram,
    time::Instant,
};
use log::info;
```

```rust
esp_bootloader_esp_idf::esp_app_desc!();

static BUTTON: Mutex<RefCell<Option<Input<'static>>>> = Mutex::new(RefCell::new(None));
static PRESSES: AtomicU32 = AtomicU32::new(0);
static LED_REQUEST: AtomicBool = AtomicBool::new(false);
static LAST_MS: AtomicU32 = AtomicU32::new(0);

const DEBOUNCE_MS: u32 = 50;

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    // All GPIO interrupts share ONE handler; register it through `Io`.
    let mut io = Io::new(peripherals.IO_MUX);
    io.set_interrupt_handler(gpio_isr);

    let mut led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());
    let mut button = Input::new(
        peripherals.GPIO14,
        InputConfig::default().with_pull(Pull::Up),
    );

    critical_section::with(|cs| {
        button.listen(Event::FallingEdge); // button press pulls the pin low
        BUTTON.borrow_ref_mut(cs).replace(button);
    });

    info!("Lab 08: press the button on GPIO14");
    let mut shown = 0;
    loop {
        // `swap(false)` reads the flag and clears it atomically.
        if LED_REQUEST.swap(false, Ordering::AcqRel) {
            led.toggle();
        }
        let n = PRESSES.load(Ordering::Relaxed);
        if n != shown {
            shown = n;
            info!("valid presses: {}", n);
        }
    }
}

#[handler]
#[ram]
fn gpio_isr() {
    let now_ms = Instant::now().duration_since_epoch().as_millis() as u32;

    critical_section::with(|cs| {
        let mut slot = BUTTON.borrow_ref_mut(cs);
        let button = slot.as_mut().unwrap();
        // The handler is shared by every GPIO: check that *our* pin fired.
        if button.is_interrupt_set() {
            if now_ms.wrapping_sub(LAST_MS.load(Ordering::Relaxed)) > DEBOUNCE_MS {
                LAST_MS.store(now_ms, Ordering::Relaxed);
                PRESSES.fetch_add(1, Ordering::Relaxed);
                LED_REQUEST.store(true, Ordering::Release);
            }
            button.clear_interrupt(); // always acknowledge
        }
    });
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Io::new(peripherals.IO_MUX)`	Creates the I/O manager. UNSTABLE in 1.1.1 (`#[instability::unstable]`).

Item	Explanation
`io.set_interrupt_handler(gpio_isr)`	Registers the single GPIO handler.
`button.listen(Event::FallingEdge)`	Arms the interrupt for a falling edge.
`static BUTTON: Mutex<RefCell<Option<Input<'static>>>>`	The `Input` is moved into a static so that the ISR can call `is_interrupt_set()` and `clear_interrupt()`.
`is_interrupt_set()`	True if this pin caused the interrupt.
`clear_interrupt()`	Acknowledge; mandatory.
`LED_REQUEST.swap(false, ...)`	Atomically read-and-clear the flag in `main`.
`#[handler]` and `#[ram]`	Marks the function as an ISR and places it in IRAM.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Each press increments `valid presses` once and toggles the LED, with no repeats caused by bounce.

Representative log lines:

```
INFO - Lab 08: press the button on GPIO14
INFO - valid presses: 1
INFO - valid presses: 2
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab08_gpio_interrupt     # compile only
cargo run   --release --bin lab08_gpio_interrupt     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab08_gpio_interrupt
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Counter increases by 3-10 per press	Bounce is faster than your debounce window: lengthen `DEBOUNCE_MS` or add a 10 nF capacitor across the button.
Interrupt never fires	`listen` was not called on the pin, or the handler was not registered with `Io::set_interrupt_handler`.
Endless interrupts, program appears frozen	`clear_interrupt()` was not called on every path.
Level events retrigger constantly	Use edge events for buttons; level events fire as long as the level persists.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::gpio
- esp_hal::interrupt
- Official example: examples/interrupt/gpio

Lab 09 - UART serial communication

Item	Value
Source file	`src/bin/lab09_uart.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `uart::{Uart, Config}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Transmit and receive data over a hardware UART using a loopback wire for a self-test that needs no other device.

2. Required components

- 1 jumper wire
- Optional: 3.3 V USB-UART adapter to talk to a PC

3. Wiring

Signal / part	ESP32-S3 pin	Note
UART1 TX	GPIO47	Connect to RX with one jumper
UART1 RX	GPIO42	Loopback: GPIO47 <-> GPIO42
(adapter option) adapter RX	GPIO47	
(adapter option) adapter TX	GPIO42	
(adapter option) adapter GND	GND	

4. Hardware principle

UART is asynchronous: both sides agree on a baud rate (here 115200 bit/s) and a frame format (start bit, 8 data bits, no parity, 1 stop bit = 8N1). There is no clock line, so the receiver re-synchronises on every start bit. The ESP32-S3 has three UART controllers (UART0 to UART2); the GPIO matrix lets any of them use almost any pins. UART0 usually carries the boot console on GPIO43/44, so the lab uses UART1. Each controller has a 128-byte hardware FIFO. In loopback, whatever is sent arrives at the receiver, which verifies the data path and the baud-rate settings.

5. Complete Rust source code

File `src/bin/lab09_uart.rs` (unmodified copy of the compiled file):

```
//! Lab 09 - UART serial communication (loopback self-test)
//!
//! Wiring: UART1  TX = GPIO47, RX = GPIO42. Connect GPIO47 to GPIO42 with ONE
//! jumper wire: every byte sent is received again.
//! (UART0 on GPIO43/GPIO44 is left alone: it usually carries the console.)
//! To talk to a PC instead, use a 3.3 V USB-UART adapter: TX->RX, RX->TX, GND->GND.
//! API level: STABLE (`esp_hal::uart`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    main,
    time::{Duration, Instant},
    uart::{Config, Uart},
};
```

```rust
use log::{error, info};

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    // 115200 baud, 8 data bits, no parity, 1 stop bit are the defaults except
    // for the baud rate, which we set explicitly for clarity.
    let mut uart = Uart::new(peripherals.UART1, Config::default().with_baudrate(115_200))
        .unwrap()
        .with_tx(peripherals.GPIO47)
        .with_rx(peripherals.GPIO42);

    let mut counter: u32 = 0;
    loop {
        // --- transmit ------------------------------------------------------
        let mut msg = heapless::String::<32>::new();
        core::fmt::write(&mut msg, format_args!("Hello UART #{}\r\n", counter)).unwrap();
        let sent = uart.write(msg.as_bytes()).unwrap(); // bytes accepted by the FIFO
        uart.flush().unwrap(); // wait until the last bit left the wire
        counter += 1;

        // --- receive (poll with a 200 ms timeout) --------------------------
        let start = Instant::now();
        let mut buf = [0u8; 32];
        let mut got = 0;
        while got < sent && start.elapsed() < Duration::from_millis(200) {
            if uart.read_ready() {
                got += uart.read(&mut buf[got..]).unwrap();
            }
        }

        if got == sent && buf[..got] == *msg.as_bytes() {
            info!("loopback OK ({} bytes): {:?}", got,
core::str::from_utf8(&buf[..got]).unwrap_or("?").trim_end());
        } else {
            error!("loopback FAILED: sent {} bytes, received {} - is the jumper fitted?",
sent, got);
        }

        let t = Instant::now();
        while t.elapsed() < Duration::from_secs(1) {}
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Uart::new(peripherals.UART1, Config::default().with_baudrate(115_200))`	Returns `Result<Uart, ConfigError>`; `unwrap()` is acceptable here because the configuration is constant.
`.with_tx(GPIO47).with_rx(GPIO42)`	Attach pins; each consumes a token.
`uart.write(bytes)`	Pushes into the TX FIFO; returns the number of bytes accepted.
`uart.flush()`	Waits until the last bit has left the wire.
`uart.read_ready()`	True if at least one byte is waiting.
`uart.read(&mut buf)`	Copies waiting bytes; returns the count.
`heapless::String<32>`	Stack-allocated string used with `core::fmt::write` (no heap in `no_std`).
200 ms timeout loop	Avoids blocking forever when the jumper is missing.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Every second: `loopback OK (n bytes): "Hello UART #k"`. Without the jumper the lab prints `loopback FAILED ... is the jumper fitted?`.

Representative log lines:

```
INFO - loopback OK (15 bytes): "Hello UART #0"
ERROR - loopback FAILED: sent 15 bytes, received 0 - is the jumper fitted?
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab09_uart      # compile only
cargo run   --release --bin lab09_uart      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab09_uart
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`loopback FAILED`	Jumper missing or on the wrong pins; verify GPIO47 and GPIO42 are the pins you wired.
Garbled text with a PC adapter	Baud-rate mismatch, or adapter set to 5 V logic (damages the pin): use a 3.3 V adapter.
Console output disappears	If you also changed UART0, you may have broken the console; UART0 is on GPIO43/44 on most boards.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::uart
- Official esp-hal v1.1.1 examples

Lab 10 - I2C master: bus scan

Item	Value
Source file	`src/bin/lab10_i2c_scan.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `i2c::master::{I2c, Config}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Initialise the I2C master and discover which devices respond on the bus.

2. Required components

- The LCD1602 I2C backpack already on the board (or any I2C device)
- 3.3 V-safe bus (see the warning in Lab 11)

3. Wiring

Signal / part	ESP32-S3 pin	Note
SDA	GPIO41	Open-drain, needs a pull-up to 3.3 V
SCL	GPIO40	Open-drain, needs a pull-up to 3.3 V
GND	GND	Common

4. Hardware principle

I2C is a two-wire, open-drain bus: devices only pull SDA/SCL low, and pull-up resistors (typically 2.2-10 kohm) return them high. Each transfer starts with a START condition and a 7-bit address plus a R/W bit; a device that recognises its address pulls SDA low during the 9th clock (ACK). A bus scan sends a write-address byte with no data to each address and records which ones ACK. Common addresses: 0x27 or 0x3F for a PCF8574 LCD backpack, 0x38 for an AHT20, 0x68 for an MPU6050/DS3231, 0x76/0x77 for a BME280.

5. Complete Rust source code

File `src/bin/lab10_i2c_scan.rs` (unmodified copy of the compiled file):

```
//! Lab 10 - I2C master: bus scan
//!
//! Wiring (the LCD1602 backpack that already sits on the board):
//!   SDA -> GPIO41, SCL -> GPIO40, GND common. Use a 3.3 V-safe bus (see manual).
//! The scan sends an address byte with an empty write; a device that exists
//! pulls SDA low (ACK) and the driver returns Ok(()).
//! API level: STABLE (`esp_hal::i2c::master`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    i2c::master::{Config, I2c},
    main,
    time::{Duration, Instant, Rate},
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
```

```
    let peripherals = esp_hal::init(esp_hal::Config::default());

    let mut i2c = I2c::new(
        peripherals.I2C0,
        Config::default().with_frequency(Rate::from_khz(100)),
    )
    .unwrap() // Err only if the configuration is invalid
    .with_sda(peripherals.GPIO41)
    .with_scl(peripherals.GPIO40);

    loop {
        info!("Scanning 7-bit addresses 0x08..=0x77 ...");
        let mut found = 0;
        for addr in 0x08u8..=0x77 {
            if i2c.write(addr, &[]).is_ok() {
                info!("  device found at 0x{:02X}", addr);
                found += 1;
            }
        }
        info!("scan complete: {} device(s)", found);

        let t = Instant::now();
        while t.elapsed() < Duration::from_secs(3) {}
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`I2c::new(peripherals.I2C0, Config::default().with_frequency(Rate::from_khz(100)))`	Creates a blocking master at 100 kHz (standard mode); returns `Result`.
`.with_sda(GPIO41).with_scl(GPIO40)`	Attach the open-drain pins (the chip's weak internal pull-ups are enabled but external ones are recommended).
`i2c.write(addr, &[])`	A zero-length write is a pure address probe; `Ok(())` means the device ACKed, `Err(..)` means no ACK.
Range `0x08..=0x77`	The 7-bit addresses outside this range are reserved by the I2C specification.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Every 3 seconds the scan lists the devices found, e.g. `device found at 0x27` for the LCD backpack, then `scan complete: 1 device(s)`.

Representative log lines:

```
INFO - Scanning 7-bit addresses 0x08..=0x77 ...
INFO -   device found at 0x27
INFO - scan complete: 1 device(s)
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab10_i2c_scan    # compile only
cargo run   --release --bin lab10_i2c_scan    # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab10_i2c_scan
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
0 devices found	SDA/SCL swapped, no common ground, no pull-ups, or the backpack's 5 V pull-ups prevent the ESP32 from seeing valid levels (see Lab 11 warning).
Phantom devices at every address	SDA stuck low (short or a device holding the bus): disconnect devices one by one.
`Err(Timeout)` /hang	A device is stretching the clock; check wiring and supply.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::i2c
- esp-hal hil-test i2c scan usage (empty write)
- Official esp-hal v1.1.1 examples

Lab 11 - LCD1602 on I2C (PCF8574 backpack)

Item	Value
Source file	`src/bin/lab11_lcd1602.rs`
Verification status	BUILD-VERIFIED; I2C is stable, `Delay` unstable. Custom driver `src/lcd1602.rs` is NOT hardware-tested
API stability	STABLE I2C; UNSTABLE `delay::Delay`; embedded-hal `I2c` + `DelayNs` traits
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Display text on a 16x2 character LCD through an I2C I/O expander, using a small driver written in this manual (HD44780 4-bit protocol) and formatted output with `write!`.

2. Required components

- LCD1602 with PCF8574 I2C backpack (address 0x27; some use 0x3F)
- Level shifter (e.g. BSS138-based bidirectional) OR a backpack whose pull-ups are tied to 3.3 V
- Jumper wires

3. Wiring

Signal / part	ESP32-S3 pin	Note
Backpack SDA	GPIO41	Through a level shifter if the backpack is powered from 5 V
Backpack SCL	GPIO40	Same
Backpack GND	GND	
Backpack VCC	5 V only with level-safe bus; otherwise 3.3 V	See the warning

4. Hardware principle

The HD44780 controller is driven in 4-bit mode: each byte is split into two nibbles, put on D4-D7, and latched on the falling edge of EN. RS selects instruction (0) or data (1). The backpack uses a PCF8574 8-bit I2C expander wired P0=RS, P1=RW, P2=EN, P3=backlight, P4-P7=D4-D7, so one I2C byte carries the whole parallel bus state. A nibble therefore needs two writes (EN high, then EN low). The initialisation sequence forces the controller into a known state by sending the 8-bit function-set command three times before switching to 4-bit mode.

5 V warning (important). The ESP32-S3 I/O pins are 3.3 V devices and are not 5 V tolerant: the datasheet's absolute maximum for a pin is about VDD+0.3 V. A typical backpack has on-board pull-up resistors from SDA/SCL to its VCC; when VCC is 5 V these pull the bus to 5 V and can damage the ESP32-S3. Removing the pull-ups and pulling up to 3.3 V does not fix the problem either, because a PCF8574 powered from 5 V specifies a high-level input threshold of 0.7 x VDD = 3.5 V, which 3.3 V logic does not reach reliably. Safe options: (1) a bidirectional level shifter between the ESP32 bus (3.3 V pull-ups) and the backpack (5 V); (2) power the backpack from 3.3 V (many LCDs are readable but dim at 3.3 V; check yours); (3) use a 3.3 V-native LCD module. These statements follow the PCF8574 and ESP32-S3 datasheets and should be checked against your parts.

5. Complete Rust source code

File `src/bin/lab11_lcd1602.rs` (unmodified copy of the compiled file):

```
//! Lab 11 - LCD1602 over I2C (PCF8574 backpack, address 0x27)
//!
//! Wiring: SDA -> GPIO41, SCL -> GPIO40, GND, VCC (see the 5 V warning in the manual:
```

```rust
//! the ESP32-S3 pins are NOT 5 V tolerant -> use a level shifter or a 3.3 V-safe
backpack).
//! Uses the shared driver in `src/lcd1602.rs` (build-verified, not hardware-tested).
//! API level: STABLE I2C + UNSTABLE `esp_hal::delay::Delay`.
#![no_std]
#![no_main]

use core::fmt::Write;

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    i2c::master::{Config, I2c},
    main,
    time::Rate,
};
use esp32s3_labs::{lcd1602::Lcd1602, reserve_camera_pins};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

const LCD_ADDR: u8 = 0x27; // some backpacks use 0x3F

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let _camera = reserve_camera_pins!(peripherals);

    let i2c = I2c::new(
        peripherals.I2C0,
        Config::default().with_frequency(Rate::from_khz(100)),
    )
    .unwrap()
    .with_sda(peripherals.GPIO41)
    .with_scl(peripherals.GPIO40);

    let mut lcd = Lcd1602::new(i2c, Delay::new(), LCD_ADDR);
    lcd.init().expect("LCD not answering on I2C - check wiring/address");

    lcd.set_cursor(0, 0).unwrap();
    lcd.print("ESP32-S3 + Rust").unwrap();

    let delay = Delay::new();
    let mut seconds: u32 = 0;
    loop {
        lcd.set_cursor(0, 1).unwrap();
        // `write!` works because Lcd1602 implements core::fmt::Write.
        write!(lcd, "uptime {:>6} s", seconds).unwrap();
        info!("uptime {} s", seconds);
        seconds += 1;
        delay.delay_millis(1000);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Lcd1602::new(i2c, Delay::new(), 0x27)`	Generic over any `embedded_hal::i2c::I2c` and `DelayNs`; the esp-hal `I2c` and `Delay` implement them.
`lcd.init()`	Runs the HD44780 power-on sequence: wait 50 ms, three 0x30 nibbles, 0x20 (4-bit), function set 0x28 (2 lines, 5x8), display on 0x0C, entry mode 0x06, clear.
`set_cursor(col, row)`	DDRAM address: row 0 starts at 0x00, row 1 at 0x40.
`print(&str)`	Sends each byte as data (RS = 1).
`write!(lcd, "uptime {:>6} s", n)`	The driver implements `core::fmt::Write`, so standard formatting works without allocation.

Item	Explanation
`expect(...)` on `init()`	`init` returns `Result<(), I2C::Error>`; the lab stops with a message if the LCD does not ACK.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Line 1 shows `ESP32-S3 + Rust`; line 2 shows `uptime  0 s` increasing every second. Contrast may need adjusting with the blue potentiometer on the backpack.

Representative log lines:

```
INFO - uptime 0 s
INFO - uptime 1 s
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab11_lcd1602     # compile only
cargo run   --release --bin lab11_lcd1602     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab11_lcd1602
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Backlight on but no characters	Contrast potentiometer on the backpack not adjusted.
`LCD not answering on I2C`	Wrong address (try 0x3F), wiring, or the level problem described above; run Lab 10 first.
Garbage characters / only first line	Initialisation timing: the first power-up needs the full 50 ms wait; do not remove the delays.
ESP32 resets or pins get hot	Do NOT connect a 5 V-pulled bus directly: use a level shifter.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::i2c
- esp_hal::delay
- HD44780 datasheet (Hitachi) / PCF8574 datasheet (Texas Instruments / NXP)
- Official esp-hal v1.1.1 examples

Lab 12 - SPI master: full-duplex loopback

Item	Value
Source file	`src/bin/lab12_spi.rs`
Verification status	BUILD-VERIFIED; also built with the `unstable` feature DISABLED
API stability	STABLE: `spi::master::{Spi, Config}`, `spi::Mode`, `gpio::Output` (software chip select)
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Configure SPI as a master, perform full-duplex transfers, and verify the data path with a MOSI-to-MISO jumper.

2. Required components

- 1 jumper wire

3. Wiring

Signal / part	ESP32-S3 pin	Note
SCK	GPIO39	SPI2 clock
MOSI	GPIO38	Master out
MISO	GPIO42	Master in; jumper from GPIO38
CS	GPIO21	Software-driven chip select (idle high)
Loopback jumper	GPIO38 <-> GPIO42	

4. Hardware principle

SPI is a synchronous, full-duplex bus: on every SCK pulse the master shifts one bit out on MOSI while shifting one bit in from MISO. Clock polarity (CPOL) and phase (CPHA) define four modes; mode 0 idles low and samples on the rising edge, and is the most common. A chip-select line frames each transaction. On ESP32-S3 the general-purpose SPI controllers are SPI2 and SPI3 (SPI0/SPI1 are reserved for flash and PSRAM); the GPIO matrix routes signals to almost any pin. With MOSI wired to MISO the received bytes equal the transmitted bytes.

SPI hardware chip select (`Spi::with_cs`) is marked unstable in 1.1.1, so this stable lab drives CS manually; this also scales to several devices on one bus.

5. Complete Rust source code

File `src/bin/lab12_spi.rs` (unmodified copy of the compiled file):

```
//! Lab 12 - SPI master (full-duplex loopback)
//!
//! Wiring (SPI2 / "FSPI" on the GPIO matrix):
//!   SCK  = GPIO39      MOSI = GPIO38      MISO = GPIO42      CS = GPIO21 (software-
driven)
//!   Jumper: GPIO38 (MOSI)  <-->  GPIO42 (MISO)   => whatever is sent comes back.
//! API level: STABLE only (`esp_hal::spi::master` + `gpio`). The hardware chip-select
helper
//! `Spi::with_cs` is marked #[instability::unstable] in 1.1.1, so this lab drives CS
with a
//! plain `Output` pin, which also works with any number of devices on one bus.
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    gpio::{Level, Output, OutputConfig},
```

```
    main,
    spi::{
        Mode,
        master::{Config, Spi},
    },
    time::{Duration, Instant, Rate},
};
use log::{error, info};

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    let mut spi = Spi::new(
        peripherals.SPI2,
        Config::default()
            .with_frequency(Rate::from_mhz(1))
            .with_mode(Mode::_0), // CPOL=0, CPHA=0
    )
    .unwrap()
    .with_sck(peripherals.GPIO39)
    .with_mosi(peripherals.GPIO38)
    .with_miso(peripherals.GPIO42);

    // Chip select is an ordinary output: idle high, low while a transfer is in progress.
    let mut cs = Output::new(peripherals.GPIO21, Level::High, OutputConfig::default());

    let mut round: u8 = 0;
    loop {
        // `transfer` is full duplex and works in place: the buffer holds the
        // bytes to send on entry and the bytes received on exit.
        let mut buf = [0xA5, round, 0x3C, 0xFF, 0x00];
        let sent = buf;
        cs.set_low();
        spi.transfer(&mut buf).unwrap();
        cs.set_high();

        if buf == sent {
            info!("SPI loopback OK: {:02X?}", buf);
        } else {
            error!("SPI loopback FAILED: sent {:02X?} got {:02X?}", sent, buf);
        }
        round = round.wrapping_add(1);

        let t = Instant::now();
        while t.elapsed() < Duration::from_millis(500) {}
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Spi::new(peripherals.SPI2, Config::default().with_frequency(Rate::from_mhz(1)).with_mode(Mode::_0))`	Blocking master at 1 MHz, mode 0. Returns `Result`.
`.with_sck(..).with_mosi(..).with_miso(..)`	Attach pins (consume tokens).
`Output::new(GPIO21, Level::High, ...)`	CS idle high.
`spi.transfer(&mut buf)`	Full-duplex in place: the buffer is sent and overwritten with the received bytes.
`cs.set_low()` / `cs.set_high()`	Frame the transfer.
Comparison `buf == sent`	Detects a broken loopback.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Every 500 ms `SPI loopback OK: [A5, nn, 3C, FF, 00]` where `nn` increments. Remove the jumper and the lab logs `SPI loopback FAILED` with the bytes read back (typically all 00 or FF).

Representative log lines:

```
INFO - SPI loopback OK: [A5, 00, 3C, FF, 00]
ERROR - SPI loopback FAILED: sent [A5, 01, 3C, FF, 00] got [00, 00, 00, 00, 00]
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                      # once per shell
cargo build --release --bin lab12_spi     # compile only
cargo run   --release --bin lab12_spi     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab12_spi
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Always all zeros	MISO not connected to MOSI, or the jumper is on the wrong pin.
Always `FF`	MISO floating high: connect the jumper; check there is no other device driving it.
Intermittent errors at high frequency	Long wires: lower the frequency or shorten the jumper.
`E0603 ... with_cs`/unstable method	`Spi::with_cs` needs the `unstable` feature; this lab avoids it on purpose.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::spi
- Official esp-hal v1.1.1 examples
- Official example: examples/peripheral/spi/loopback

Lab 13 - RC522 RFID reader over SPI

Item	Value
Source file	`src/bin/lab13_rc522.rs`
Verification status	BUILD-VERIFIED; SPI/GPIO stable, `Delay` unstable. Driver crate mfrc522 0.8.0. NOT hardware-tested
API stability	STABLE SPI/GPIO; UNSTABLE `delay`; `embedded-hal-bus::ExclusiveDevice`; `mfrc522` 0.8.0
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Read the unique identifier (UID) of an ISO 14443A card or tag with an MFRC522-based RC522 module over SPI.

2. Required components

- RC522 RFID module (MFRC522, 13.56 MHz)
- MIFARE Classic 1K card or key-fob tag
- Jumper wires (8)

3. Wiring

Signal / part	ESP32-S3 pin	Note
RC522 SDA (this is chip select, NSS)	GPIO21	Software CS
RC522 SCK	GPIO39	SPI2 clock
RC522 MOSI	GPIO38	
RC522 MISO	GPIO42	
RC522 RST	GPIO14	Reset, active low
RC522 3.3V	3V3	3.3 V ONLY - do not use 5 V
RC522 GND	GND	
RC522 IRQ	not connected	Polling is used

4. Hardware principle

The RC522 module contains an MFRC522 reader IC that generates a 13.56 MHz field, powers passive cards and exchanges data using ISO/IEC 14443 type A modulation. The microcontroller talks to the IC over SPI (up to 10 Mbit/s): each transaction sends an address byte (bit 7 = read, bits 6..1 = register address) followed by data. Reading a card follows a fixed protocol: REQA (request) -> the card answers with ATQA; anticollision cascade levels return the UID in 4-byte parts (UID size 4, 7 or 10 bytes); SELECT addresses one card; HLTA puts it to sleep. The lab uses the `mfrc522` crate for these steps. The `Spi` driver implements embedded-hal's `SpiBus`; the crate needs a `SpiDevice` (bus plus chip select), which `embedded_hal_bus::spi::ExclusiveDevice` builds from the bus and a GPIO.

The module is a 3.3 V device; its logic pins are not 5 V tolerant.

5. Complete Rust source code

File `src/bin/lab13_rc522.rs` (unmodified copy of the compiled file):

```
//! Lab 13 - RC522 (MFRC522) RFID reader over SPI
//!
//! Wiring (RC522 module is a 3.3 V device - do NOT power it from 5 V):
//!   RC522 SDA (= chip select) -> GPIO21     RC522 SCK  -> GPIO39
//!   RC522 MOSI                -> GPIO38     RC522 MISO -> GPIO42
//!   RC522 RST                 -> GPIO14     RC522 3.3V -> 3V3,  GND -> GND
```

```
//!   RC522 IRQ                  -> not connected
//!
//! The SPI *bus* (`Spi`) implements embedded-hal's `SpiBus`. The `mfrc522` crate
//! needs a `SpiDevice` (bus + chip-select), which `embedded-hal-bus::ExclusiveDevice`
//! builds for us.
//! API level: STABLE SPI + UNSTABLE Delay. Driver crate: mfrc522 0.8.0 (build-verified
only).
#![no_std]
#![no_main]

use embedded_hal_bus::spi::ExclusiveDevice;
use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    gpio::{Level, Output, OutputConfig},
    main,
    spi::{
        Mode,
        master::{Config, Spi},
    },
    time::Rate,
};
use log::{info, warn};
use mfrc522::{Mfrc522, comm::blocking::spi::SpiInterface};

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    // Hardware reset of the RC522: pull RST low, then release.
    let mut rst = Output::new(peripherals.GPIO14, Level::Low, OutputConfig::default());
    delay.delay_millis(50);
    rst.set_high();
    delay.delay_millis(50);

    let spi_bus = Spi::new(
        peripherals.SPI2,
        Config::default()
            .with_frequency(Rate::from_mhz(4))
            .with_mode(Mode::_0),
    )
    .unwrap()
    .with_sck(peripherals.GPIO39)
    .with_mosi(peripherals.GPIO38)
    .with_miso(peripherals.GPIO42);

    // Chip select is driven in software by ExclusiveDevice.
    let cs = Output::new(peripherals.GPIO21, Level::High, OutputConfig::default());
    let spi_dev = ExclusiveDevice::new(spi_bus, cs, Delay::new()).unwrap();

    // The RC522 needs NSS high for >= 50 ns between transfers; in release
    // builds the CPU is fast enough to violate that without a small delay.
    let nss_delay = Delay::new();
    let itf = SpiInterface::new(spi_dev).with_delay(move || nss_delay.delay_nanos(200));

    let mut rfid = Mfrc522::new(itf).init().expect("RC522 init failed (wiring?)");
    match rfid.version() {
        Ok(v) => info!("RC522 VersionReg = 0x{:02X} (0x91 / 0x92 are genuine)", v),
        Err(_) => warn!("could not read VersionReg"),
    }

    info!("Present a MIFARE card or tag to the reader ...");
    loop {
        // REQA: ask any card in the field to answer; Ok(..) means one did.
        if let Ok(atqa) = rfid.new_card_present() {
            // Anticollision + SELECT -> unique ID (4, 7 or 10 bytes).
            match rfid.select(&atqa) {
                Ok(uid) => {
```

```
                    info!("card UID = {:02X?}", uid.as_bytes());
                    let _ = rfid.hlta(); // put the card to sleep so it is read once
                }
                Err(_) => warn!("card detected but SELECT failed"),
            }
        }
        delay.delay_millis(200);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
RST pulse (`Output` low 50 ms, then high)	Hardware reset of the reader before configuration.
`Spi::new(SPI2, Config { 4 MHz, Mode::_0 }) + .with_sck/.with_mosi/.with_miso`	SPI bus; no hardware CS.
`ExclusiveDevice::new(spi_bus, cs, Delay::new())`	Turns bus + CS pin + delay into an `embedded_hal::spi::SpiDevice`; returns `Result` (the pin's error type).
`SpiInterface::new(spi_dev).with_delay(...)`	Communication layer of the `mfrc522` crate; the closure inserts a short pause so NSS stays high at least 50 ns between transfers.
`Mfrc522::new(itf).init()`	Configures timers, antenna gain and CRC; returns an initialised reader or an error.
`rfid.version()`	Reads VersionReg: 0x91 or 0x92 on genuine chips; clones report other non-zero values.
`rfid.new_card_present()`	Sends REQA; `Ok(AtqA)` if a card answered.
`rfid.select(&atqa)`	Anticollision + SELECT; returns a `Uid`.
`uid.as_bytes()`	The UID as a byte slice (4, 7 or 10 bytes).
`rfid.hlta()`	Halts the card so it is read once until it leaves and re-enters the field.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

At start the version register value is printed, then `Present a MIFARE card or tag to the reader ...`. Each presented card prints `card UID = [DE, AD, BE, EF]` (your own bytes).

Representative log lines:

```
INFO - RC522 VersionReg = 0x92 (0x91 / 0x92 are genuine)
INFO - Present a MIFARE card or tag to the reader ...
INFO - card UID = [04, A1, 2B, 7C]
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab13_rc522      # compile only
cargo run   --release --bin lab13_rc522      # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab13_rc522
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy

Symptom	Likely cause and remedy
`RC522 init failed` / `VersionReg = 0x00 or 0xFF`	Wiring or power: check 3.3 V, SDA/CS pin, MISO, and that RST is released (high).
Version register is neither 0x91 nor 0x92	Clone chips (e.g. FM17522) report different values and often still work.
Card detected but `SELECT failed`	Card too far/unstable in the field; hold it 1-3 cm above the antenna.
Works at 1 MHz, fails at 4 MHz	Long or noisy wires: lower `with_frequency`.
Reads the same card twice	`hlta()` was skipped; the card re-answers REQA immediately.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::spi
- mfrc522 crate 0.8.0
- embedded-hal-bus 0.3.0
- Official esp-hal v1.1.1 examples

Lab 14 - DMA: memory-to-memory transfer

Item	Value
Source file	`src/bin/lab14_dma_mem2mem.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `dma::{Mem2Mem, BurstConfig}`; macro `dma_buffers!`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Copy a buffer with the DMA controller (no CPU involvement) and verify the result.

2. Required components

- ESP32-S3 board only

3. Wiring

No external wiring.

4. Hardware principle

Direct Memory Access lets a peripheral move data between memory and peripherals (or memory and memory) without the CPU executing a copy loop. ESP32-S3 uses GDMA with five channels. A transfer is described by a linked list of *descriptors*; each descriptor holds a buffer pointer, a length (at most 4092 bytes) and flags, and points to the next one, so a 10 KiB buffer needs three descriptors. The `dma_buffers!` macro allocates static buffers and the right number of descriptors. Rust's ownership model keeps the buffers borrowed for the duration of the transfer so safe code cannot touch memory the DMA engine is still using.

ESP32-S3 detail: in memory-to-memory mode the data path goes through the SPI2 DMA peripheral, so `Mem2Mem::new` takes the `SPI2` token as well as the channel: SPI2 is not available to the same program afterwards.

5. Complete Rust source code

File `src/bin/lab14_dma_mem2mem.rs` (unmodified copy of the compiled file):

```
//! Lab 14a - DMA memory-to-memory transfer
//!
//! No wiring needed. The DMA controller copies a 10 KiB buffer without CPU help.
//! NOTE (ESP32-S3): `Mem2Mem::new` consumes DMA_CH0 *and* SPI2, because the
//! copy path is routed through the SPI2 DMA peripheral. SPI2 is therefore not
//! available to the same program afterwards.
//! API level: UNSTABLE (`esp_hal::dma`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    dma::{BurstConfig, Mem2Mem},
    dma_buffers, main,
    time::{Duration, Instant},
};
use log::{error, info};

esp_bootloader_esp_idf::esp_app_desc!();

const DATA_SIZE: usize = 10 * 1024;

#[main]
```

```rust
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    // The macro creates static buffers + DMA descriptors (linked list entries,
    // each describing <= 4092 bytes) and returns `&'static mut` references.
    let (rx_buffer, rx_descriptors, tx_buffer, tx_descriptors) = dma_buffers!(DATA_SIZE);

    let mem2mem = Mem2Mem::new(peripherals.DMA_CH0, peripherals.SPI2);
    let mut mem2mem = mem2mem
        .with_descriptors(rx_descriptors, tx_descriptors, BurstConfig::default())
        .unwrap();

    for (i, b) in tx_buffer.iter_mut().enumerate() {
        *b = (i % 251) as u8; // 251 is prime -> pattern does not repeat at 256
    }

    info!("starting DMA copy of {} bytes", DATA_SIZE);
    let t0 = Instant::now();
    match mem2mem.start_transfer(rx_buffer, tx_buffer) {
        Ok(transfer) => {
            transfer.wait().unwrap(); // blocks until the DMA "done" interrupt flag
            let us = t0.elapsed().as_micros();
            let ok = rx_buffer.iter().zip(tx_buffer.iter()).all(|(a, b)| a == b);
            if ok {
                info!("DMA copy verified OK in {} us", us);
            } else {
                error!("DMA copy MISMATCH");
            }
        }
        Err(e) => error!("start_transfer failed: {:?}", e),
    }

    loop {
        delay.delay(Duration::from_secs(2));
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`dma_buffers!(DATA_SIZE)`	Returns `(rx_buffer, rx_descriptors, tx_buffer, tx_descriptors)` as `&'static mut` slices. Each macro call site may run only once.
`Mem2Mem::new(peripherals.DMA_CH0, peripherals.SPI2)`	Creates the engine on channel 0.
`.with_descriptors(rx_d, tx_d, BurstConfig::default())`	Supplies descriptor lists; returns `Result<SimpleMem2Mem, DmaError>`.
`start_transfer(rx_buffer, tx_buffer)`	Starts both directions; returns a transfer handle.
`transfer.wait()`	Blocks until done.
Pattern `(i % 251) as u8`	251 is prime, so the pattern does not repeat at 256 and misalignment would be detected.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

`starting DMA copy of 10240 bytes` followed by `DMA copy verified OK in <n> us`.

Representative log lines:

```
INFO - starting DMA copy of 10240 bytes
INFO - DMA copy verified OK in ... us
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                        # once per shell
cargo build --release --bin lab14_dma_mem2mem     # compile only
cargo run   --release --bin lab14_dma_mem2mem     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab14_dma_mem2mem
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`start_transfer failed: ...`	Buffer/descriptor mismatch: use `dma_buffers!` with the same size for both directions.
`SPI2` already moved	Mem2Mem owns SPI2: do not use SPI2 in the same program.
`E0603 dma is private`	The `unstable` feature is missing.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::dma
- Official example: examples/peripheral/dma/mem2mem

Lab 14B - DMA with SPI: full-duplex loopback

Item	Value
Source file	`src/bin/lab14b_spi_dma.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature (`with_dma`, `with_cs`)
API stability	UNSTABLE: `dma::{DmaRxBuf, DmaTxBuf}`, `Spi::with_dma`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Use DMA to run an SPI transfer in the background, and understand how ownership moves buffers and the driver into the transfer and back.

2. Required components

- 1 jumper wire

3. Wiring

Signal / part	ESP32-S3 pin	Note
SCK	GPIO39	
MOSI	GPIO38	
MISO	GPIO42	Jumper to GPIO38
CS	GPIO21	Hardware chip select via `with_cs` (unstable)

4. Hardware principle

With DMA the CPU builds a buffer and starts the transfer; the DMA engine feeds the SPI FIFO and stores the received bytes without CPU copies. This frees the CPU for other work and sustains high clock rates. `spi.transfer(rx_len, rx_buf, tx_len, tx_buf)` consumes the driver and both buffers and returns a handle; `wait()` blocks until the transfer ends and returns them. Because the buffers are moved, the borrow checker prevents reading or writing them mid-transfer.

5. Complete Rust source code

File `src/bin/lab14b_spi_dma.rs` (unmodified copy of the compiled file):

```
//! Lab 14b - SPI + DMA full-duplex loopback
//!
//! Wiring: SCK=GPIO39, MOSI=GPIO38, MISO=GPIO42, CS=GPIO21, jumper GPIO38 <-> GPIO42.
//! With DMA the CPU only prepares buffers; the transfer runs in the background
//! and `wait()` hands the buffers (and the SPI driver) back when finished.
//! API level: UNSTABLE (`esp_hal::dma`, `Spi::with_dma`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    dma::{DmaRxBuf, DmaTxBuf},
    dma_buffers, main,
    spi::{
        Mode,
        master::{Config, Spi},
    },
    time::Rate,
};
use log::{error, info};
```

```rust
esp_bootloader_esp_idf::esp_app_desc!();

const BUF: usize = 256;

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    let (rx_buffer, rx_descriptors, tx_buffer, tx_descriptors) = dma_buffers!(BUF);
    let mut dma_rx = DmaRxBuf::new(rx_descriptors, rx_buffer).unwrap();
    let mut dma_tx = DmaTxBuf::new(tx_descriptors, tx_buffer).unwrap();

    let mut spi = Spi::new(
        peripherals.SPI2,
        Config::default()
            .with_frequency(Rate::from_mhz(2))
            .with_mode(Mode::_0),
    )
    .unwrap()
    .with_sck(peripherals.GPIO39)
    .with_mosi(peripherals.GPIO38)
    .with_miso(peripherals.GPIO42)
    .with_cs(peripherals.GPIO21)
    .with_dma(peripherals.DMA_CH0); // SpiDma: DMA-capable driver

    let mut round: u8 = 0;
    loop {
        for (i, b) in dma_tx.as_mut_slice().iter_mut().enumerate() {
            *b = (i as u8).wrapping_add(round);
        }

        // Ownership: the driver and both buffers are MOVED into the transfer and
        // given back by wait(); the compiler therefore forbids touching them
        // while the DMA engine is still reading/writing them.
        let transfer = spi
            .transfer(dma_rx.len(), dma_rx, dma_tx.len(), dma_tx)
            .map_err(|e| e.0)
            .unwrap();
        (spi, (dma_rx, dma_tx)) = transfer.wait();

        let ok = dma_rx.as_slice() == dma_tx.as_slice();
        if ok {
            info!("SPI-DMA loopback OK, first bytes {:02X?}", &dma_rx.as_slice()[..8]);
        } else {
            error!("SPI-DMA loopback MISMATCH - jumper GPIO38<->GPIO42 fitted?");
        }
        round = round.wrapping_add(1);
        delay.delay_millis(1000);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`DmaRxBuf::new(rx_descriptors, rx_buffer), DmaTxBuf::new(..)`	Wrap buffer + descriptors; return `Result<_, DmaBufError>`.
`.with_dma(peripherals.DMA_CH0)`	Turns `Spi<Blocking>` into `SpiDma<Blocking>`.
`dma_tx.as_mut_slice()`	Fill the transmit data.
`spi.transfer(...)` followed by `map_err(..)`	Starts the transfer; on error it returns a tuple of the error and the moved values, and the closure keeps only the first field (`e.0`).
`(spi, (dma_rx, dma_tx)) = transfer.wait();`	Takes ownership of the driver and buffers back.
`dma_rx.as_slice() == dma_tx.as_slice()`	Loopback verification.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Every second `SPI-DMA loopback OK, first bytes [..]`; without the jumper, `MISMATCH`.

Representative log lines:

```
INFO - SPI-DMA loopback OK, first bytes [00, 01, 02, 03, 04, 05, 06, 07]
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab14b_spi_dma     # compile only
cargo run   --release --bin lab14b_spi_dma     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab14b_spi_dma
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`MISMATCH`	Jumper missing between GPIO38 and GPIO42.
`use of moved value: spi`	You touched the driver while it was inside `transfer`: wait first.
Hang inside `wait()`	DMA channel/peripheral mismatch or the buffers were not created with `dma_buffers!`.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::dma
- esp_hal::spi
- Official example: spi/loopback_dma_psram

Lab 15 - Watchdog timer

Item	Value
Source file	`src/bin/lab15_watchdog.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature
API stability	UNSTABLE: `timer::timg::{TimerGroup, Wdt, MwdtStage}`; `system::reset_reason` (unstable item)
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Arm the main-system watchdog, keep it alive by feeding it, then provoke a reset by deliberately hanging, and read the reset reason after reboot.

2. Required components

- ESP32-S3 board only

3. Wiring

No external wiring.

4. Hardware principle

A watchdog is a hardware countdown timer that resets the system if software fails to restart ('feed') it in time, protecting against crashes and infinite loops. Each timer group has a main-system watchdog (MWDT) with up to four stages, each with its own timeout and action. Stage 0 is configured by `enable()` to reset the system. **esp-hal's `init` disables the RWDT, the super watchdog and TIMG0/TIMG1 watchdogs**, so the application decides whether to use them. After a watchdog reset the ROM records the cause; `reset_reason()` returns it (for the TIMG0 watchdog the S3 enum value is `CoreMwdt0`).

5. Complete Rust source code

File `src/bin/lab15_watchdog.rs` (unmodified copy of the compiled file):

```
//! Lab 15 - Watchdog timer (TIMG0 main-system watchdog)
//!
//! No wiring needed. `esp_hal::init()` DISABLES all watchdogs, so we enable the
//! TIMG0 watchdog ourselves, feed it five times, then "hang" on purpose. After
//! 3 s without feeding the watchdog resets the chip and the reset reason is
//! printed on the next boot.
//! API level: UNSTABLE (`esp_hal::timer::timg::Wdt`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    main,
    system::reset_reason,
    time::{Duration, Instant},
    timer::timg::{MwdtStage, TimerGroup},
};
use log::{info, warn};

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
```

```
    info!("reset reason: {:?}", reset_reason());

    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let mut wdt = timg0.wdt;
    wdt.set_timeout(MwdtStage::Stage0, Duration::from_millis(3000));
    wdt.enable(); // stage 0 action = reset the whole system
    info!("watchdog armed: 3 s timeout");

    for i in 1..=5 {
        let t = Instant::now();
        while t.elapsed() < Duration::from_millis(1000) {}
        wdt.feed(); // restart the 3 s countdown
        info!("fed the watchdog ({}/5)", i);
    }

    warn!("simulating a software hang - no more feeding; reset expected in ~3 s");
    loop {} // never feeds again
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`reset_reason()`	Prints the previous reset cause on every boot (`ChipPowerOn` at power-up).
`TimerGroup::new(peripherals.TIMG0).wdt`	Moves the watchdog field out of the timer group.
`wdt.set_timeout(MwdtStage::Stage0, Duration::from_millis(3000))`	3 s without feeding triggers stage 0.
`wdt.enable()`	Arms the watchdog; stage 0 action = system reset.
`wdt.feed()`	Restarts the countdown (done 5 times, once per second).
`loop {}`	Simulated hang: no more feeding.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Boot 1: `reset reason: Some(ChipPowerOn)`, `watchdog armed: 3 s timeout`, five `fed the watchdog (n/5)` lines, the warning, then after about 3 s the chip reboots. Boot 2 prints `reset reason: Some(CoreMwdt0)` and the cycle repeats.

Representative log lines:

```
INFO - reset reason: Some(ChipPowerOn)
INFO - watchdog armed: 3 s timeout
INFO - fed the watchdog (1/5)
WARN - simulating a software hang ...
INFO - reset reason: Some(CoreMwdt0)
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab15_watchdog     # compile only
cargo run   --release --bin lab15_watchdog     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab15_watchdog
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
No reset occurs	The watchdog is not armed (`enable()` missing) or was fed in the hang loop.
Reset immediately after arming	Timeout shorter than the interval between `feed()` calls.
Different reason value than expected	Verify against the `SocResetReason` enum in the ESP32-S3 source; the exact variant depends on the watchdog stage action.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::timer
- esp_hal::system
- Official esp-hal v1.1.1 examples

Lab 16 - RMT peripheral: driving a WS2812 RGB LED

Item	Value
Source file	`src/bin/lab16_rmt_ws2812.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Timings NOT verified on hardware
API stability	UNSTABLE: `rmt::{Rmt, TxChannelConfig, TxChannelCreator, PulseCode}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Generate a precisely timed one-wire waveform with the RMT peripheral to set the colour of a WS2812 (NeoPixel) LED.

2. Required components

- WS2812 / WS2812B LED (single LED, ring, or the on-board LED if your board has one)
- Optional level shifter 74AHCT125 if the LED is powered from 5 V

3. Wiring

Signal / part	ESP32-S3 pin	Note
WS2812 DIN	GPIO48	Many ESP32-S3 boards already connect an RGB LED here (some use GPIO38): check your schematic
WS2812 VDD	5 V (or 3.3 V for one LED, may work)	Datasheet: input high needs 0.7 x VDD
WS2812 GND	GND	Common

4. Hardware principle

A WS2812 encodes each bit in the *duration of the high pulse* within an 800 kHz frame: a '0' is about 0.4 us high then 0.85 us low, a '1' is about 0.8 us high then 0.45 us low (tolerance about +-150 ns). 24 bits (G, R, B, most significant bit first) set one LED, and a low level lasting more than 50 us latches the data. Software bit-banging at this speed is fragile; the RMT peripheral instead stores a list of 32-bit *pulse codes* (level1, duration1, level2, duration2) in dedicated RAM and plays them back with clock-accurate timing. With the 80 MHz source and divider 1 one tick is 12.5 ns, so T0H = 32 ticks (400 ns), T0L = 68 (850 ns), T1H = 64 (800 ns), T1L = 36 (450 ns).

5. Complete Rust source code

File `src/bin/lab16_rmt_ws2812.rs` (unmodified copy of the compiled file):

```
//! Lab 16 - RMT peripheral: driving a WS2812 ("NeoPixel") RGB LED
//!
//! Wiring: WS2812 DIN -> GPIO48 (on many ESP32-S3 boards an RGB LED is already wired to
//! GPIO48 - check your schematic; some use GPIO38). WS2812 VDD 5 V (3.3 V often works
for a
//! single LED), GND common. The LED's data input expects ~0.7*VDD high level, so with a
//! 5 V LED use a level shifter (e.g. 74AHCT125) for reliable operation.
//!
//! The RMT peripheral turns a list of `PulseCode`s (level/duration pairs) into an
//! exact waveform without CPU involvement - ideal for the 800 kHz one-wire WS2812
//! protocol (each bit = one high pulse + one low pulse of different lengths).
//! API level: UNSTABLE (`esp_hal::rmt`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
```

```
    delay::Delay,
    gpio::Level,
    main,
    rmt::{PulseCode, Rmt, TxChannelConfig, TxChannelCreator},
    time::Rate,
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

// RMT tick = 1 / 80 MHz = 12.5 ns (clk_divider = 1).
const T0H: u16 = 32; // 0.40 us
const T0L: u16 = 68; // 0.85 us
const T1H: u16 = 64; // 0.80 us
const T1L: u16 = 36; // 0.45 us

/// Convert one RGB colour to 24 pulse codes (WS2812 expects the order G, R, B,
/// most significant bit first) plus the end marker that terminates the frame.
fn encode(r: u8, g: u8, b: u8) -> [PulseCode; 25] {
    let bit0 = PulseCode::new(Level::High, T0H, Level::Low, T0L);
    let bit1 = PulseCode::new(Level::High, T1H, Level::Low, T1L);
    let mut out = [PulseCode::end_marker(); 25];
    let word = ((g as u32) << 16) | ((r as u32) << 8) | b as u32;
    for i in 0..24 {
        out[i] = if word & (1 << (23 - i)) != 0 { bit1 } else { bit0 };
    }
    out
}

/// Integer HSV (h in 0..=255 around the colour wheel, full saturation) -> RGB.
fn wheel(h: u8, brightness: u8) -> (u8, u8, u8) {
    let region = h / 43;
    let rem = (h as u16 - region as u16 * 43) * 6; // 0..=252
    let (r, g, b) = match region {
        0 => (255, rem as u8, 0),
        1 => (255 - rem as u8, 255, 0),
        2 => (0, 255, rem as u8),
        3 => (0, 255 - rem as u8, 255),
        4 => (rem as u8, 0, 255),
        _ => (255, 0, 255 - rem as u8),
    };
    let s = |c: u8| ((c as u16 * brightness as u16) / 255) as u8;
    (s(r), s(g), s(b))
}

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    // RMT source clock on ESP32-S3 is 80 MHz.
    let rmt = Rmt::new(peripherals.RMT, Rate::from_mhz(80)).unwrap();
    let mut channel = rmt
        .channel0
        .configure_tx(&TxChannelConfig::default().with_clk_divider(1))
        .unwrap()
        .with_pin(peripherals.GPIO48);

    info!("Lab 16: rainbow on the WS2812 (GPIO48)");
    let mut hue: u8 = 0;
    loop {
        let (r, g, b) = wheel(hue, 32); // keep it dim: 32/255
        let data = encode(r, g, b);

        // transmit() consumes the channel and returns a transaction; wait()
        // returns the channel when the frame (and its end marker) has been sent.
        let transaction = channel.transmit(&data).unwrap();
        channel = transaction.wait().unwrap();

        hue = hue.wrapping_add(2);
        delay.delay_millis(20); // also provides the >50 us reset/latch gap
```

```
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Rmt::new(peripherals.RMT, Rate::from_mhz(80))`	Creates the RMT driver; returns `Result`. 80 MHz is the S3 source clock.
`rmt.channel0.configure_tx(&TxChannelConfig::default().with_clk_divider(1))`	Configures TX channel 0 (S3 has four TX channels); returns `Result`.
`.with_pin(peripherals.GPIO48)`	Binds the output pin.
`PulseCode::new(Level::High, 32, Level::Low, 68)`	One symbol: high for 32 ticks then low for 68 ticks.
`PulseCode::end_marker()`	Terminates the sequence.
`channel.transmit(&data)`	Consumes the channel and returns a transaction.
`transaction.wait()`	Returns the channel when sending is complete.
`wheel(hue, 32)`	Integer colour-wheel with brightness limited to 32/255 for eye comfort.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The LED slowly cycles through the colour wheel (rainbow) at low brightness.

Representative log lines:

```
INFO - Lab 16: rainbow on the WS2812 (GPIO48)
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab16_rmt_ws2812     # compile only
cargo run   --release --bin lab16_rmt_ws2812     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab16_rmt_ws2812
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
LED stays dark	Wrong pin for your board's LED, no 5 V power, or data level too low: use a level shifter.
Wrong colours	Some LEDs use RGB instead of GRB order: swap in `encode`.
First LED flickers randomly	Missing reset gap or ground not common; the 20 ms delay provides the latch gap.
`Err(...)` from `Rmt::new`	Requested frequency not reachable from the clock source on this chip.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::rmt

- WS2812B datasheet (Worldsemi)
- Official esp-hal v1.1.1 examples

Lab 17 - PCNT pulse counter and rotary encoder

Item	Value
Source file	`src/bin/lab17_pcnt_encoder.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Not hardware-tested
API stability	UNSTABLE: `pcnt::{Pcnt, unit, channel}`, `Input::peripheral_input`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Decode a quadrature rotary encoder entirely in hardware with the pulse counter and extend the 16-bit count to 32 bits using limit interrupts.

2. Required components

- KY-040 or similar rotary encoder (A/CLK, B/DT, optional switch)
- Optional 10 kohm pull-ups if the module has none

3. Wiring

Signal / part	ESP32-S3 pin	Note
Encoder A (CLK)	GPIO38	Internal pull-up
Encoder B (DT)	GPIO39	Internal pull-up
Encoder +	3V3	
Encoder GND	GND	

4. Hardware principle

A quadrature encoder outputs two square waves A and B that are 90 degrees out of phase. Which signal leads tells the rotation direction. The PCNT peripheral has units with two channels each; a channel has a *pulse* (edge) input and a *control* input and, for each edge, can increment, decrement or hold the counter depending on the control level. Wiring channel 0 as (edge=A, control=B) and channel 1 as (edge=B, control=A) with opposite actions counts all four edges per detent (x4 decoding) with a built-in glitch filter. The hardware counter is a signed 16-bit value; when it hits the configured limits (here +-100) an interrupt fires and the handler accumulates the overflow so the application sees a 32-bit position.

5. Complete Rust source code

File `src/bin/lab17_pcnt_encoder.rs` (unmodified copy of the compiled file):

```
//! Lab 17 - PCNT pulse counter: rotary (quadrature) encoder
//!
//! Wiring (KY-040 style encoder, 3.3 V logic):
//!   CLK (A) -> GPIO38      DT (B) -> GPIO39      + -> 3V3      GND -> GND
//!   (internal pull-ups are enabled; add 10 k external ones if the module has none)
//! The PCNT hardware decodes both edges of A and B in hardware (x4 resolution), so no
//! CPU time is spent on edges. Two channels per unit implement the quadrature logic.
//! The 16-bit counter is extended to i32 by counting limit events in an interrupt.
//! API level: UNSTABLE (`esp_hal::pcnt`).
#![no_std]
#![no_main]

use core::{
    cell::RefCell,
    sync::atomic::{AtomicI32, Ordering},
};
```

```rust
use critical_section::Mutex;
use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    gpio::{Input, InputConfig, Pull},
    handler, main,
    pcnt::{Pcnt, channel, unit},
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

static UNIT0: Mutex<RefCell<Option<unit::Unit<'static, 1>>>> =
Mutex::new(RefCell::new(None));
static OVERFLOW: AtomicI32 = AtomicI32::new(0);

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    let mut pcnt = Pcnt::new(peripherals.PCNT);
    pcnt.set_interrupt_handler(on_pcnt);

    let u0 = pcnt.unit1;
    u0.set_low_limit(Some(-100)).unwrap();
    u0.set_high_limit(Some(100)).unwrap();
    // Glitch filter: ignore pulses shorter than 1023 APB cycles (~12.8 us at 80 MHz).
    u0.set_filter(Some(1023)).unwrap();
    u0.clear();

    let cfg = InputConfig::default().with_pull(Pull::Up);
    let pin_a = Input::new(peripherals.GPIO38, cfg);
    let pin_b = Input::new(peripherals.GPIO39, cfg);
    let a = pin_a.peripheral_input();
    let b = pin_b.peripheral_input();

    // Channel 0: edges of A are counted, direction is controlled by level of B.
    let ch0 = &u0.channel0;
    ch0.set_ctrl_signal(b.clone());
    ch0.set_edge_signal(a.clone());
    ch0.set_ctrl_mode(channel::CtrlMode::Reverse, channel::CtrlMode::Keep);
    ch0.set_input_mode(channel::EdgeMode::Increment, channel::EdgeMode::Decrement);

    // Channel 1: the mirror image, counts the edges of B controlled by A.
    let ch1 = &u0.channel1;
    ch1.set_ctrl_signal(a);
    ch1.set_edge_signal(b);
    ch1.set_ctrl_mode(channel::CtrlMode::Reverse, channel::CtrlMode::Keep);
    ch1.set_input_mode(channel::EdgeMode::Decrement, channel::EdgeMode::Increment);

    u0.listen(); // enable the limit interrupts
    u0.resume(); // start counting
    let counter = u0.counter.clone();
    critical_section::with(|cs| UNIT0.borrow_ref_mut(cs).replace(u0));

    info!("Lab 17: turn the encoder");
    let mut last: i32 = 0;
    loop {
        let value = counter.get() as i32 + OVERFLOW.load(Ordering::SeqCst);
        if value != last {
            last = value;
            info!("encoder position: {} (about {} detents)", value, value / 4);
        }
        delay.delay_millis(20);
    }
}

#[handler]
fn on_pcnt() {
    critical_section::with(|cs| {
```

```
        let mut slot = UNIT0.borrow_ref_mut(cs);
        if let Some(u0) = slot.as_mut() {
            if u0.interrupt_is_set() {
                let events = u0.events();
                if events.high_limit {
                    OVERFLOW.fetch_add(100, Ordering::SeqCst);
                } else if events.low_limit {
                    OVERFLOW.fetch_add(-100, Ordering::SeqCst);
                }
                u0.reset_interrupt();
            }
        }
    });
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Pcnt::new(peripherals.PCNT)` and `pcnt.unit1`	The peripheral and one of its four units.
`set_low_limit(Some(-100))`, `set_high_limit(Some(100))`	Limit events (also resets the counter at the limit).
`set_filter(Some(1023))`	Glitch filter: ignore pulses shorter than 1023 APB cycles (about 12.8 us).
`Input::new(.., Pull::Up)` + `.peripheral_input()`	Gets a signal handle without consuming the pin.
`channel0.set_ctrl_signal` / `set_edge_signal`	Assign control (B) and edge (A) inputs.
`set_input_mode(Increment, Decrement)` / `set_ctrl_mode(Reverse, Keep)`	Edge actions and how the control level reverses them.
`u0.listen()`, `u0.resume()`	Enable limit interrupts and start counting.
`counter.get()`	Reads the 16-bit counter (clonable handle `Counter`).
ISR: `events()`, `reset_interrupt()`	Detects high/low limit and adds +-100 to `OVERFLOW`.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Turning the knob prints `encoder position: n (about n/4 detents)`: positive in one direction, negative in the other, continuing past +-100.

Representative log lines:

```
INFO - encoder position: 4 (about 1 detents)
INFO - encoder position: -12 (about -3 detents)
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab17_pcnt_encoder     # compile only
cargo run   --release --bin lab17_pcnt_encoder     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab17_pcnt_encoder
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Counts the wrong direction	Swap A and B, or swap the Increment/Decrement actions.
Position jumps/noisy	Contact bounce: increase the glitch filter or add 10 nF capacitors to ground.
Four counts per click is too many	Divide by 4, or use only channel 0 for x2 decoding.
Counter wraps incorrectly	The ISR must run: check `listen()` and that the handler was registered.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::pcnt
- Official esp-hal v1.1.1 examples

Lab 18 - MCPWM: motor-control PWM signals

Item	Value
Source file	`src/bin/lab18_mcpwm.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Dead-time helper NOT used (see text)
API stability	UNSTABLE: `mcpwm::{McPwm, PeripheralClockConfig, operator::PwmPinConfig, timer::PwmWorkingMode}`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Produce two independently controlled 20 kHz PWM outputs suitable for driving the IN1/IN2 inputs of an H-bridge motor driver.

2. Required components

- H-bridge driver (TB6612FNG, DRV8833, L298N …) and a small DC motor with its own supply
- Or: two LEDs with 330 ohm resistors / an oscilloscope

3. Wiring

Signal / part	ESP32-S3 pin	Note
PWM A -> driver IN1	GPIO38	20 kHz
PWM B -> driver IN2	GPIO39	20 kHz
Driver GND	ESP32 GND	Common ground
Motor supply	External	Never from the ESP32 3V3 pin

4. Hardware principle

A DC motor's speed follows the average voltage, controlled by PWM duty. An H-bridge switches the motor between supply and ground; with two inputs: IN1 = PWM and IN2 = 0 drives forward, IN1 = 0 and IN2 = PWM drives reverse. The ESP32-S3 has two MCPWM units, each with three timers and three operators; an operator has two outputs (A and B) whose compare values are set independently. A 20 kHz carrier is above the audible range. The MCPWM source clock is 160 MHz: a prescaler produces a 32 MHz peripheral clock, and the timer period register is set to 99 so that a compare value of 0..100 equals percent duty.

MCPWM also offers a dead-time module for complementary half-bridge drive (`Operator::with_linked_pins`) which inserts a delay between turning one transistor off and the other on to prevent shoot-through. In 1.1.1 the module documentation still lists dead time as not yet implemented, so this lab does not use it; if you do, check the waveform on an oscilloscope first. This is an *unverified pattern*.

5. Complete Rust source code

File `src/bin/lab18_mcpwm.rs` (unmodified copy of the compiled file):

```
//! Lab 18 - MCPWM: motor-control PWM signals
//!
//! Wiring (H-bridge such as TB6612FNG / L298N / DRV8833, logic side only):
//!   PWM A -> GPIO38  (bridge IN1)      PWM B -> GPIO39  (bridge IN2)
//!   Motor supply and GND are on the bridge; share GND with the ESP32-S3.
//! With no motor you can watch both pins with a scope or LEDs (330 ohm).
//! One timer + one operator give two independently-controlled outputs:
//!   forward : A = duty ramp, B = 0     reverse : A = 0, B = duty ramp
//! API level: UNSTABLE (`esp_hal::mcpwm`). The complementary dead-time helper
//! `Operator::with_linked_pins` exists but is NOT used here: the module docs of
//! 1.1.1 still list dead time as "not yet implemented" - verify on a scope first.
```

```rust
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    main,
    mcpwm::{
        McPwm, PeripheralClockConfig,
        operator::PwmPinConfig,
        timer::PwmWorkingMode,
    },
    time::Rate,
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    // 32 MHz peripheral clock derived from the 160 MHz MCPWM source clock.
    let clock_cfg = PeripheralClockConfig::with_frequency(Rate::from_mhz(32)).unwrap();
    let mut mcpwm = McPwm::new(peripherals.MCPWM0, clock_cfg);

    // Operator 0 follows timer 0.
    mcpwm.operator0.set_timer(&mcpwm.timer0);
    let (mut pwm_a, mut pwm_b) = mcpwm.operator0.with_pins(
        peripherals.GPIO38,
        PwmPinConfig::UP_ACTIVE_HIGH,
        peripherals.GPIO39,
        PwmPinConfig::UP_ACTIVE_HIGH,
    );

    // Timer period register = 99 -> timestamps 0..=100 are percent duty.
    // 20 kHz is above the audible range, typical for small DC motors.
    let timer_cfg = clock_cfg
        .timer_clock_with_frequency(99, PwmWorkingMode::Increase, Rate::from_khz(20))
        .unwrap();
    mcpwm.timer0.start(timer_cfg);

    info!("Lab 18: MCPWM 20 kHz, ramping A then B");
    loop {
        pwm_b.set_timestamp(0);
        for duty in (0..=100).step_by(5) {
            pwm_a.set_timestamp(duty);
            info!("forward duty {} %", duty);
            delay.delay_millis(200);
        }
        pwm_a.set_timestamp(0);
        for duty in (0..=100).step_by(5) {
            pwm_b.set_timestamp(duty);
            info!("reverse duty {} %", duty);
            delay.delay_millis(200);
        }
        pwm_b.set_timestamp(0);
        delay.delay_millis(1000);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`PeripheralClockConfig::with_frequency(Rate::from_mhz(32))`	Derives the peripheral clock; returns `Result`.
`McPwm::new(peripherals.MCPWM0, clock_cfg)`	Creates the block with timers `timer0..2` and operators `operator0..2`.

Item	Explanation
`mcpwm.operator0.set_timer(&mcpwm.timer0)`	Operator 0 follows timer 0.
`operator0.with_pins(pin_a, PwmPinConfig::UP_ACTIVE_HIGH, pin_b, PwmPinConfig::UP_ACTIVE_HIGH)`	Two outputs, each high from the start of the period until its compare value.
`clock_cfg.timer_clock_with_frequency(99, PwmWorkingMode::Increase, Rate::from_khz(20))`	Finds the prescaler so 100 counts take 50 us.
`mcpwm.timer0.start(timer_cfg)`	Starts the timer.
`pwm_a.set_timestamp(duty)`	Compare value = percent duty (0..=100).

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

On a scope: 20 kHz on GPIO38 with duty ramping 0..100 % in 5 % steps every 200 ms while GPIO39 stays low; then the roles swap. The log prints `forward duty n %` / `reverse duty n %`.

Representative log lines:

```
INFO - Lab 18: MCPWM 20 kHz, ramping A then B
INFO - forward duty 5 %
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab18_mcpwm     # compile only
cargo run   --release --bin lab18_mcpwm     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab18_mcpwm
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`Err(FrequencyError)` from `with_frequency`/`timer_clock_with_frequency`	Requested frequency cannot be derived from the source clock with the chosen period.
Both outputs active at once on the H-bridge	Never drive IN1 and IN2 PWM simultaneously unless the driver defines that as braking.
Motor hot/stalled at low duty	Motors need a minimum duty to overcome friction; start at 20-30 %.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::mcpwm
- Official esp-hal v1.1.1 examples

Lab 19 - TWAI (CAN 2.0) communication

Item	Value
Source file	`src/bin/lab19_twai.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Self-test mode only; bus operation NOT hardware-tested
API stability	UNSTABLE: `twai::{TwaiConfiguration, EspTwaiFrame, StandardId, filter::SingleStandardFilter}`; `nb`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Configure the CAN controller, send a frame and receive it again in self-test mode (no second node needed), and learn the wiring for a real two-node bus.

2. Required components

- Self-test: none
- Real bus: 3.3 V CAN transceiver (SN65HVD230, TJA1051T/3), two nodes, 120 ohm termination resistors at both bus ends, twisted pair

3. Wiring

Signal / part	ESP32-S3 pin	Note
Self-test: TX+RX	GPIO21	One pin, `new_no_transceiver` (open-drain with pull-up)
Bus: transceiver TXD	GPIO21	`TwaiConfiguration::new(.., rx, tx, ..)`
Bus: transceiver RXD	GPIO14	

4. Hardware principle

CAN (Controller Area Network) is a differential multi-master bus with message-based addressing: each frame has an identifier (11-bit standard or 29-bit extended) that doubles as priority (lower wins arbitration), up to 8 data bytes, a CRC and an acknowledge slot that every receiver drives. Espressif calls the controller TWAI and it implements CAN 2.0 (not CAN FD). The bit rate must be identical on all nodes. A transceiver translates the controller's logic levels to the differential CANH/CANL signals. In self-test mode the controller transmits without requiring an acknowledgement and a frame flagged for self-reception is also placed in its own receive FIFO, enabling a test with no other hardware. The acceptance filter decides which identifiers reach the FIFO; the lab accepts all standard IDs.

5. Complete Rust source code

File `src/bin/lab19_twai.rs` (unmodified copy of the compiled file):

```
//! Lab 19 - TWAI (CAN 2.0) communication
//!
//! TWAI is Espressif's name for the CAN 2.0 controller. This lab runs in SELF-TEST mode:
//! the controller sends a frame and receives its own copy, so it needs NO second node
and
//! NO transceiver. Wiring: nothing but GPIO21 (it acts as both TX and RX, open-drain
with
//! pull-up, via `new_no_transceiver`).
//!
//! For a real bus use `TwaiConfiguration::new(TWAI0, rx, tx, ...)` with a 3.3 V
transceiver
//! (e.g. SN65HVD230): GPIO21 -> TXD, GPIO14 <- RXD, 120 ohm termination at both bus
```

```
ends,
//! and switch `TwaiMode::SelfTest` to `TwaiMode::Normal`.
//! API level: UNSTABLE (`esp_hal::twai`). Needs `nb` for the non-blocking API.
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{
    delay::Delay,
    main,
    twai::{self, EspTwaiFrame, StandardId, TwaiMode, filter::SingleStandardFilter},
};
use log::info;
use nb::block;

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let delay = Delay::new();

    // One physical pin used as both RX and TX (no transceiver).
    // `split` is unsafe because it hands out two handles to one GPIO.
    let (rx_pin, tx_pin) = unsafe { peripherals.GPIO21.split() };

    let mut config = twai::TwaiConfiguration::new_no_transceiver(
        peripherals.TWAI0,
        rx_pin,
        tx_pin,
        twai::BaudRate::B125K,
        TwaiMode::SelfTest,
    );

    // Accept every standard (11-bit) identifier: 'x' = don't care.
    config.set_filter(
        const { SingleStandardFilter::new(b"xxxxxxxxxxx", b"x", [b"xxxxxxxx",
b"xxxxxxxx"]) },
    );

    // start() leaves configuration mode: baud rate and filter are now locked.
    let mut can = config.start();

    let mut n: u8 = 0;
    loop {
        // In self-test mode the frame must be created with `new self reception`
        // so that the controller also loops it back into its RX FIFO.
        let tx = EspTwaiFrame::new_self_reception(StandardId::new(0x123).unwrap(), &[n,
0xDE, 0xAD]).unwrap();
        block!(can.transmit(&tx)).unwrap();

        let rx = block!(can.receive()).unwrap();
        info!("sent #{} and received back: {:?}", n, rx);
        info!(
            "error counters  TX={}  RX={}",
            can.transmit_error_count(),
            can.receive_error_count()
        );

        n = n.wrapping_add(1);
        delay.delay_millis(1000);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`unsafe { peripherals.GPIO21.split() }`	Creates an input and output handle for one physical pin; `unsafe` because two handles alias the pin.

Item	Explanation
`TwaiConfiguration::new_no_transceiver(TWAI0, rx, tx, BaudRate::B125K, TwaiMode::SelfTest)`	Configuration mode; the pin is open-drain.
`config.set_filter(const { SingleStandardFilter::new(..) })`	Accept-all mask: 'x' means don't care; the filter must be a compile-time constant.
`config.start()`	Leaves configuration mode; bit rate and filter are now locked.
`EspTwaiFrame::new_self_reception(StandardId::new(0x123).unwrap(), &data)`	Frame with the self-reception request bit set.
`block!(can.transmit(&tx))`	`transmit` returns `nb::Result`; `block!` retries on `WouldBlock`.
`block!(can.receive())`	Waits for the looped-back frame.
`transmit_error_count()` / `receive_error_count()`	TEC/REC error counters; a healthy node stays near 0.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Every second: `sent #n and received back: ...` showing the frame (ID 0x123, data `[n, DE, AD]`), and error counters `TX=0 RX=0`.

Representative log lines:

```
INFO - sent #0 and received back: EspTwaiFrame { ... }
INFO - error counters  TX=0  RX=0
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab19_twai     # compile only
cargo run   --release --bin lab19_twai     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab19_twai
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`transmit` never completes in normal mode	No other node ACKs: use self-test mode or connect a second node at the same bit rate.
Error counters climb	Bit-rate mismatch or missing termination.
Frames with other IDs are never received	The acceptance filter is blocking them; use an all-'x' mask or match your IDs.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::twai
- Official example: examples/peripheral/twai

Lab 20 - Wi-Fi connectivity: station, DHCP and a TCP echo server

Item	Value
Source file	`src/bin/lab20_wifi.rs`
Verification status	BUILD-VERIFIED (companion crates esp-radio 0.18.0, esp-rtos 0.3.0, embassy-net 0.9). NOT run against a real access point
API stability	esp-radio `wifi` API (not part of esp-hal); esp-rtos scheduler; embassy-net
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Join a 2.4 GHz Wi-Fi network as a station, obtain an IP address by DHCP, and run a TCP echo server on port 7777, all with Embassy tasks.

2. Required components

- ESP32-S3 board
- 2.4 GHz Wi-Fi access point (WPA2) and its SSID and password
- PC on the same network with `nc` (netcat) or telnet

3. Wiring

No external wiring (antenna is on the module). Keep the antenna area clear of metal.

4. Hardware principle

Wi-Fi on the ESP32-S3 (IEEE 802.11 b/g/n, 2.4 GHz) is implemented by Espressif's closed-source radio firmware, accessed from Rust through the `esp-radio` crate. The radio stack needs a heap (allocated with `esp-alloc`) and a small RTOS scheduler (`esp-rtos`) which also provides the Embassy time driver and executor integration. `esp_radio::wifi::new` returns a `WifiController` (connect, scan, configure) and network interfaces; the *station* interface is given to `embassy-net`, a TCP/IP stack built on smoltcp, which performs DHCP (the access point leases the board an address) and exposes `TcpSocket`. Four cooperating tasks run on one executor: `main` (initialisation), `connection` (reconnect loop), `net_task` (drives the stack) and `echo_task` (the TCP server). Note that ADC2 pins cannot be sampled while Wi-Fi is active.

5. Complete Rust source code

File `src/bin/lab20_wifi.rs` (unmodified copy of the compiled file):

```
//! Lab 20 - Wi-Fi connectivity (station mode, DHCP, TCP echo server)
//!
//! Build with your network credentials in environment variables:
//!   SSID="MyWifi" PASSWORD="secret" cargo run --release --bin lab20_wifi
//! (2.4 GHz networks only: the ESP32-S3 radio has no 5 GHz.)
//!
//! After the board prints "Got IP: a.b.c.d/24" test the server from a PC:
//!   nc <ip> 7777        (anything you type is echoed back)
//!
//! Architecture (all cooperative Embassy tasks on one executor):
//!   main        - initialises hardware, radio and network stack, spawns tasks
//!   connection  - keeps the Wi-Fi link up (connect / wait for disconnect / retry)
//!   net_task    - runs the embassy-net stack (smoltcp) forever
//!   echo_task   - listens on TCP port 7777 and echoes data
//! Wi-Fi uses ADC2-free pins only: never read ADC2 pins (GPIO11..20) while Wi-Fi is on.
//! API level: esp-radio 0.18.0 / esp-rtos 0.3.0 / embassy-net 0.9 (companion crates, not
esp-hal).
#![no_std]
#![no_main]
```

```rust
use embassy_executor::Spawner;
use embassy_net::{Runner, StackResources, tcp::TcpSocket};
use embassy_time::{Duration, Timer};
use embedded_io_async::Write;
use esp_alloc as _;
use esp_backtrace as _;
use esp_hal::{
    clock::CpuClock, interrupt::software::SoftwareInterruptControl, ram, rng::Rng,
    timer::timg::TimerGroup,
};
use esp_radio::wifi::{
    Config, ControllerConfig, Interface, WifiController, scan::ScanConfig,
sta::StationConfig,
};
use log::{info, warn};

esp_bootloader_esp_idf::esp_app_desc!();

// Allocate a value in a `static` and hand out a `&'static mut` to it.
macro_rules! mk_static {
    ($t:ty, $val:expr) => {{
        static STATIC_CELL: static_cell::StaticCell<$t> = static_cell::StaticCell::new();
        #[deny(unused_attributes)]
        let x = STATIC_CELL.uninit().write($val);
        x
    }};
}

const SSID: &str = match option_env!("SSID") {
    Some(s) => s,
    None => "YOUR_SSID",
};
const PASSWORD: &str = match option_env!("PASSWORD") {
    Some(s) => s,
    None => "YOUR_PASSWORD",
};

#[esp_rtos::main]
async fn main(spawner: Spawner) -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals =
esp_hal::init(esp_hal::Config::default().with_cpu_clock(CpuClock::max()));

    // The radio firmware needs a heap: one region in reclaimed IRAM, one in DRAM.
    esp_alloc::heap_allocator!(#[ram(reclaimed)] size: 64 * 1024);
    esp_alloc::heap_allocator!(size: 36 * 1024);

    // esp-rtos provides the scheduler + Embassy time driver. It needs one hardware
    // timer and one software interrupt.
    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let sw_int = SoftwareInterruptControl::new(peripherals.SW_INTERRUPT);
    esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);

    if SSID == "YOUR_SSID" {
        warn!("SSID/PASSWORD not set at build time - set them as environment variables");
    }

    let station = Config::Station(
        StationConfig::default()
            .with_ssid(SSID)
            .with_password(PASSWORD.into()),
    );
    let (mut controller, interfaces) = esp_radio::wifi::new(
        peripherals.WIFI,
        ControllerConfig::default().with_initial_config(station),
    )
    .unwrap();

    // IP configuration from the router's DHCP server.
    let net config = embassy_net::Config::dhcpv4(Default::default());
    let rng = Rng::new();
    let seed = (rng.random() as u64) << 32 | rng.random() as u64;
```

```
    let (stack, runner) = embassy_net::new(
        interfaces.station,
        net_config,
        mk_static!(StackResources<4>, StackResources::<4>::new()),
        seed,
    );

    info!("scanning for access points ...");
    let aps = controller
        .scan_async(&ScanConfig::default().with_max(10))
        .await
        .unwrap();
    for ap in aps {
        info!("  AP: {:?}", ap);
    }

    spawner.spawn(connection(controller).unwrap());
    spawner.spawn(net_task(runner).unwrap());

    stack.wait_config_up().await; // resolves once DHCP has assigned an address
    if let Some(cfg) = stack.config_v4() {
        info!("Got IP: {}", cfg.address);
    }

    spawner.spawn(echo_task(stack).unwrap());

    loop {
        Timer::after(Duration::from_secs(10)).await;
        info!("link up: {}", stack.is_link_up());
    }
}

#[embassy_executor::task]
async fn connection(mut controller: WifiController<'static>) {
    loop {
        info!("connecting to Wi-Fi ...");
        match controller.connect_async().await {
            Ok(info) => {
                info!("connected: {:?}", info);
                // Park here until the access point drops us, then retry.
                let reason = controller.wait_for_disconnect_async().await.ok();
                warn!("disconnected: {:?}", reason);
            }
            Err(e) => warn!("connect failed: {:?}", e),
        }
        Timer::after(Duration::from_secs(5)).await;
    }
}

#[embassy_executor::task]
async fn net_task(mut runner: Runner<'static, Interface<'static>>) {
    runner.run().await
}

#[embassy_executor::task]
async fn echo_task(stack: embassy_net::Stack<'static>) {
    let mut rx_buf = [0u8; 1024];
    let mut tx_buf = [0u8; 1024];
    loop {
        let mut socket = TcpSocket::new(stack, &mut rx_buf, &mut tx_buf);
        socket.set_timeout(Some(Duration::from_secs(30)));
        info!("echo: waiting for a client on port 7777");
        if socket.accept(7777).await.is_err() {
            warn!("echo: accept failed");
            continue;
        }
        info!("echo: client connected");
        let mut buf = [0u8; 128];
        loop {
            match socket.read(&mut buf).await {
                Ok(0) => break, // peer closed the connection
                Ok(n) => {
```

```
                    if socket.write_all(&buf[..n]).await.is_err() {
                        break;
                    }
                }
                Err(_) => break,
            }
        }
        socket.close();
        info!("echo: client gone");
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`esp_alloc::heap_allocator!(#[ram(reclaimed)] size: 64 * 1024)`	Registers a heap in reclaimed IRAM (and a second region in DRAM).
`esp_rtos::start(timg0.timer0, sw_int.software_interrupt0)`	Starts the scheduler using one hardware timer and one software interrupt.
`#[esp_rtos::main] async fn main(spawner: Spawner) -> !`	Async entry point on the Embassy executor.
`SSID` / `PASSWORD` via `option_env!`	Credentials are read from environment variables at build time.
`StationConfig::default().with_ssid(..).with_password(..)`	Station configuration (builder).
`esp_radio::wifi::new(peripherals.WIFI, ControllerConfig::default().with_initial_config(station))`	Starts the radio; returns `(WifiController, Interfaces)`.
`embassy_net::new(interfaces.station, Config::dhcpv4(..), resources, seed)`	Creates the stack and its runner; `seed` comes from the hardware RNG.
`controller.scan_async(..)`	Lists nearby access points.
`spawner.spawn(task(..).unwrap())`	Spawns a task; in embassy-executor 0.10 the task function returns a `Result`.
`stack.wait_config_up().await`	Resolves when DHCP finished.
`controller.connect_async()` / `wait_for_disconnect_async()`	Connection supervisor loop.
`TcpSocket::new(stack, &mut rx, &mut tx)` + `accept(7777)`	Listening socket with caller-supplied buffers; `read`/`write_all` echo data.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Log shows a scan list, `connecting to Wi-Fi ...`, `connected: ...`, `Got IP: 192.168.x.y/24`, then `echo: waiting for a client on port 7777`. From the PC, `nc <ip> 7777` echoes everything you type.

Representative log lines:

```
INFO - scanning for access points ...
INFO - connected: ...
INFO - Got IP: 192.168.1.57/24
INFO - echo: waiting for a client on port 7777
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                      # once per shell
cargo build --release --bin lab20_wifi     # compile only
cargo run   --release --bin lab20_wifi     # compile, flash, open monitor
```

Variants for this lab:

```
SSID="MyWifi" PASSWORD="secret" cargo run --release --bin lab20_wifi
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab20_wifi
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`WARN - SSID/PASSWORD not set at build time`	Pass them as environment variables on the cargo command line. They are baked into the firmware: do not publish it.
`undefined symbol: esp_rtos_*` at link time	`esp_rtos::start` is not called: the generated `build.rs` prints a hint about the missing scheduler.
`connect failed` loops	Wrong password, 5 GHz-only network, or WPA3-only configuration.
Never gets an IP	DHCP disabled or MAC filtering on the router.
Out-of-memory panic	Increase the heap sizes in `heap_allocator!`.
Unreliable ADC readings	ADC2 pins are unusable with Wi-Fi: use ADC1.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp-radio 0.18.0 on docs.rs
- Official example: examples/wifi/embassy_dhcp
- Embassy
- embassy-net

Lab 21 - Bluetooth Low Energy: GATT Battery Service

Item	Value
Source file	`src/bin/lab21_ble.rs`
Verification status	BUILD-VERIFIED (adapted from the official bas_peripheral example; trouble-host 0.6.0). NOT tested with a phone
API stability	esp-radio `ble` + `trouble-host` 0.6.0 (not part of esp-hal)
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Advertise as a BLE peripheral, accept a connection from a phone, and expose a standard GATT Battery Service whose value notifies every 2 seconds.

2. Required components

- ESP32-S3 board
- Smartphone with a BLE scanner app (nRF Connect, LightBlue)

3. Wiring

No external wiring.

4. Hardware principle

Bluetooth Low Energy has two layers of interaction. GAP (Generic Access Profile) governs discovery and connection: the peripheral broadcasts *advertising packets* containing flags, a list of service UUIDs and a name, and a *central* (the phone) may connect. GATT (Generic Attribute Profile) then organises data as *services* containing *characteristics*; each characteristic has a UUID, a value and properties (read, write, notify). The standard Battery Service has the 16-bit UUID 0x180F and a Battery Level characteristic 0x2A19 (0-100 %) with read and notify. Notifications push new values to a subscribed central without polling. In Rust, the `esp-radio` crate provides the HCI controller (`BleConnector`) and the `trouble-host` crate implements the host stack (L2CAP, ATT, GATT, GAP) with Embassy; the `#[gatt_server]` and `#[gatt_service]` macros generate the attribute table. The ESP32-S3 supports BLE 5 only (no Bluetooth Classic).

5. Complete Rust source code

File `src/bin/lab21_ble.rs` (unmodified copy of the compiled file):

```
//! Lab 21 - Bluetooth Low Energy: GATT Battery Service peripheral (trouBLE host stack)
//!
//! Adapted from the official esp-hal-v1.1.1 example `examples/ble/bas_peripheral`.
//! Test with a phone: install "nRF Connect" (or LightBlue), scan, connect to
//! "Trouble Example", open the Battery Service and subscribe to Battery Level:
//! the value counts up every 2 s through GATT notifications. No wiring needed.
//! Radio: ESP32-S3 has BLE 5 only (no Bluetooth Classic).
//! API level: esp-radio 0.18.0 (`ble` feature) + trouble-host 0.6.0.

#![no_std]
#![no_main]

use embassy_executor::Spawner;
use embassy_futures::{join::join, select::select};
use embassy_time::Timer;
use esp_alloc as _;
use esp_backtrace as _;
use esp_hal::{
    clock::CpuClock,
    interrupt::software::SoftwareInterruptControl,
    timer::timg::TimerGroup,
```

```
};
use esp_radio::ble::controller::BleConnector;
use log::{info, warn};
use trouble_host::prelude::*;

esp_bootloader_esp_idf::esp_app_desc!();

#[esp_rtos::main]
async fn main( s: Spawner) {
    esp_println::logger::init_logger_from_env();
    let peripherals =
esp_hal::init(esp_hal::Config::default().with_cpu_clock(CpuClock::max()));
    esp_alloc::heap_allocator!(size: 72 * 1024);
    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let sw_int = SoftwareInterruptControl::new(peripherals.SW_INTERRUPT);
    esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);

    let bluetooth = peripherals.BT;
    let connector = BleConnector::new(bluetooth, Default::default()).unwrap();
    let controller: ExternalController<_, 1> = ExternalController::new(connector);

    ble_bas_peripheral_run(controller).await;
}

/// Max number of connections
const CONNECTIONS_MAX: usize = 1;
/// Max number of L2CAP channels.
const L2CAP_CHANNELS_MAX: usize = 2; // Signal + att

// GATT Server definition
#[gatt_server]
struct Server {
    battery_service: BatteryService,
}

/// Battery service
#[gatt_service(uuid = service::BATTERY)]
struct BatteryService {
    /// Battery Level
    #[descriptor(uuid = descriptors::VALID_RANGE, read, value = [0, 100])]
    #[descriptor(uuid = descriptors::MEASUREMENT_DESCRIPTION, name = "hello", read, value
= "Battery Level", type = &'static str)]
    #[characteristic(uuid = characteristic::BATTERY_LEVEL, read, notify, value = 10)]
    level: u8,
    #[characteristic(uuid = "408813df-5dd4-1f87-ec11-cdb001100000", write, read, notify)]
    status: bool,
}

/// Run the BLE stack.
pub async fn ble_bas_peripheral_run<C>(controller: C)
where
    C: Controller,
{
    // Using a fixed "random" address can be useful for testing. In real scenarios, one
would
    // use e.g. the MAC 6 byte array as the address (how to get that varies by the
platform).
    let address: Address = Address::random([0xff, 0x8f, 0x1a, 0x05, 0xe4, 0xff]);
    info!("Our address = {:?}", address);

    let mut resources: HostResources<DefaultPacketPool, CONNECTIONS_MAX,
L2CAP_CHANNELS_MAX> =
        HostResources::new();
    let stack = trouble_host::new(controller, &mut
resources).set_random_address(address);
    let Host {
        mut peripheral,
        runner,
        ..
    } = stack.build();

    info!("Starting advertising and GATT service");
```

```
    let server = Server::new_with_config(GapConfig::Peripheral(PeripheralConfig {
        name: "TrouBLE",
        appearance: &appearance::power_device::GENERIC_POWER_DEVICE,
    }))
    .unwrap();

    let _ = join(ble_task(runner), async {
        loop {
            match advertise("Trouble Example", &mut peripheral, &server).await {
                Ok(conn) => {
                    // set up tasks when the connection is established to a central, so
they don't
                    // run when no one is connected.
                    let a = gatt_events_task(&server, &conn);
                    let b = custom_task(&server, &conn, &stack);
                    // run until any task ends (usually because the connection has been
closed),
                    // then return to advertising state.
                    select(a, b).await;
                }
                Err(e) => {
                    panic!("[adv] error: {:?}", e);
                }
            }
        }
    })
    .await;
}

/// This is a background task that is required to run forever alongside any other BLE
tasks.
///
/// ## Alternative
///
/// If you didn't require this to be generic for your application, you could statically
spawn this
/// with i.e.
///
/// ```rust,ignore
///
/// #[embassy_executor::task]
/// async fn ble_task(mut runner: Runner<'static, SoftdeviceController<'static>>) {
///     runner.run().await;
/// }
///
/// spawner.spawn(ble_task(runner).unwrap());
/// ```
async fn ble_task<C: Controller, P: PacketPool>(mut runner: Runner<'_, C, P>) {
    loop {
        if let Err(e) = runner.run().await {
            panic!("[ble_task] error: {:?}", e);
        }
    }
}

/// Stream Events until the connection closes.
///
/// This function will handle the GATT events and process them.
/// This is how we interact with read and write requests.
async fn gatt_events_task<P: PacketPool>(
    server: &Server<'_>,
    conn: &GattConnection<'_, '_, P>,
) -> Result<(), Error> {
    let level = server.battery_service.level;
    let reason = loop {
        match conn.next().await {
            GattConnectionEvent::Disconnected { reason } => break reason,
            GattConnectionEvent::Gatt { event } => {
                match &event {
                    GattEvent::Read(event) => {
                        if event.handle() == level.handle {
```

```
                            let value = server.get(&level);
                            info!("[gatt] Read Event to Level Characteristic: {:?}",
value);
                        }
                    }
                    GattEvent::Write(event) => {
                        if event.handle() == level.handle {
                            info!(
                                "[gatt] Write Event to Level Characteristic: {:?}",
                                event.data()
                            );
                        }
                    }
                    _ => {}
                };
                // This step is also performed at drop(), but writing it explicitly is
necessary
                // in order to ensure reply is sent.
                match event.accept() {
                    Ok(reply) => reply.send().await,
                    Err(e) => warn!("[gatt] error sending response: {:?}", e),
                };
            }
            _ => {} // ignore other Gatt Connection Events
        }
    };
    info!("[gatt] disconnected: {:?}", reason);
    Ok(())
}

/// Create an advertiser to use to connect to a BLE Central, and wait for it to connect.
async fn advertise<'values, 'server, C: Controller>(
    name: &'values str,
    peripheral: &mut Peripheral<'values, C, DefaultPacketPool>,
    server: &'server Server<'values>,
) -> Result<GattConnection<'values, 'server, DefaultPacketPool>, BleHostError<C::Error>>
{
    let mut advertiser_data = [0; 31];
    let len = AdStructure::encode_slice(
        &[
            AdStructure::Flags(LE_GENERAL_DISCOVERABLE | BR_EDR_NOT_SUPPORTED),
            AdStructure::ServiceUuids16(&[[0x0f, 0x18]]),
            AdStructure::CompleteLocalName(name.as_bytes()),
        ],
        &mut advertiser_data[..],
    )?;
    let advertiser = peripheral
        .advertise(
            &Default::default(),
            Advertisement::ConnectableScannableUndirected {
                adv_data: &advertiser_data[..len],
                scan_data: &[],
            },
        )
        .await?;
    info!("[adv] advertising");
    let conn = advertiser.accept().await?.with_attribute_server(server)?;
    info!("[adv] connection established");
    Ok(conn)
}

/// Example task to use the BLE notifier interface.
/// This task will notify the connected central of a counter value every 2 seconds.
/// It will also read the RSSI value every 2 seconds.
/// and will stop when the connection is closed by the central or an error occurs.
async fn custom_task<C: Controller, P: PacketPool>(
    server: &Server<'_>,
    conn: &GattConnection<'_, '_, P>,
    stack: &Stack<'_, C, P>,
) {
    let mut tick: u8 = 0;
```

```
    let level = server.battery_service.level;
    loop {
        tick = tick.wrapping_add(1);
        info!("[custom_task] notifying connection of tick {}", tick);
        if level.notify(conn, &tick).await.is_err() {
            info!("[custom_task] error notifying connection");
            break;
        };
        // read RSSI (Received Signal Strength Indicator) of the connection.
        if let Ok(rssi) = conn.raw().rssi(stack).await {
            info!("[custom_task] RSSI: {:?}", rssi);
        } else {
            info!("[custom_task] error getting RSSI");
            break;
        };
        Timer::after_secs(2).await;
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`BleConnector::new(.., peripherals.BT, Default::default())`	Creates the HCI transport over the radio.
`ExternalController` + `HostResources<DefaultPacketPool, 1, 2>`	Host-stack storage for 1 connection and 2 L2CAP channels (signalling + ATT).
`#[gatt_server] struct Server { battery_service: BatteryService }`	Generated attribute table.
`#[gatt_service(uuid = service::BATTERY)]` / `#[characteristic(uuid = BATTERY_LEVEL, read, notify, value = 10)]`	Declares the Battery Service and its characteristic with properties and initial value.
`AdStructure::Flags / ServiceUuids16 / CompleteLocalName`	Advertising payload: discoverable, Battery Service UUID, name `Trouble Example`.
`advertise(..)` + `gatt_events_task`	Advertise until connected, then serve GATT events.
`level.notify(conn, &tick).await`	Sends a notification every 2 s (`custom_task`).
`embassy-sync = 0.7.2` in `Cargo.toml`	trouble-host 0.6.0 depends on embassy-sync 0.7; the macro resolves the name through your crate.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Log: `Starting advertising and GATT service`. On the phone the device `Trouble Example` appears; after connecting, the Battery Service shows Battery Level and a counter increments every 2 s when notifications are enabled.

Representative log lines:

```
INFO - Starting advertising and GATT service
INFO - [adv] connection established
INFO - [custom_task] notifying connection of tick 1
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                    # once per shell
cargo build --release --bin lab21_ble     # compile only
cargo run   --release --bin lab21_ble     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab21_ble
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`error[E0277]: the trait bound NoopRawMutex: embassy_sync::blocking_mutex::raw::RawMutex is not satisfied`, with the note `there are multiple different versions of crate embassy_sync in the dependency graph`	Real error reproduced while preparing this manual: your crate depends on embassy-sync 0.8 while trouble-host 0.6.0 uses 0.7. Pin `embassy-sync = "0.7.2"` in `Cargo.toml`.
Device not visible	Phone Bluetooth/location permission off; board not advertising; try another scanner.
Connects then drops	Only 1 connection is configured; disconnect other centrals.
Out of memory	Increase `heap_allocator!` size.
Coexistence with Wi-Fi	Wi-Fi + BLE together needs the `coex` feature (enabled) and extra RAM.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp-radio 0.18.0 on docs.rs
- trouble-host
- Official example: examples/ble/bas_peripheral

Lab 22A - USB Serial/JTAG: blocking echo

Item	Value
Source file	`src/bin/lab22a_usb_serial_jtag.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Not hardware-tested
API stability	UNSTABLE: `usb_serial_jtag::UsbSerialJtag`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Use the chip's built-in USB device controller as a serial port with no external USB-UART chip, and convert typed characters to upper case.

2. Required components

- Board with the native USB connector (GPIO19 = D-, GPIO20 = D+)
- USB cable

3. Wiring

None: the PHY is built into the chip and uses the fixed pins GPIO19 and GPIO20.

4. Hardware principle

The ESP32-S3 has a fixed-function USB Serial/JTAG controller: a CDC-ACM serial port plus a JTAG debug interface on the same two USB pins, enumerated as VID 0x303A, PID 0x1001. It needs no host driver on Linux (`/dev/ttyACM0`). Because esp-println's default `auto` backend also writes to this controller, the lab avoids initialising the logger so the console and echo do not mix. The same controller can enter the ROM download mode, which is what `espflash` uses.

5. Complete Rust source code

File `src/bin/lab22a_usb_serial_jtag.rs` (unmodified copy of the compiled file):

```
//! Lab 22a - USB Serial/JTAG: blocking echo (upper-case converter)
//!
//! Wiring: none - use the board's native USB connector (GPIO19 = D-, GPIO20 = D+).
//! Open the CDC serial port with `espflash monitor`, `picocom` or any terminal and type.
//! This lab deliberately does NOT initialise the esp-println logger: on a board whose USB
//! port is wired to the USB Serial/JTAG controller, logging would share this very channel.
//! API level: UNSTABLE (`esp_hal::usb_serial_jtag`).
#![no_std]
#![no_main]

use esp_backtrace as _;
use esp_hal::{main, usb_serial_jtag::UsbSerialJtag};
use nb::block;

esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let mut usb = UsbSerialJtag::new(peripherals.USB_DEVICE);

    usb.write(b"\r\nLab 22a: type something, it comes back in UPPER CASE\r\n").ok();
    loop {
        // read_byte() is non-blocking (nb): block! spins until a byte arrives.
        let byte = block!(usb.read_byte()).unwrap();
```

```
        let out = match byte {
            b'\r' => b'\n', // terminal sends CR on Enter
            c => c.to_ascii_uppercase(),
        };
        if out == b'\n' {
            usb.write(b"\r\n").ok();
        } else {
            usb.write(&[out]).ok();
        }
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`UsbSerialJtag::new(peripherals.USB_DEVICE)`	Blocking driver for the controller.
`usb.write(bytes)`	Sends to the host; returns `Result`.
`block!(usb.read_byte())`	`read_byte` is `nb`: `WouldBlock` until a byte arrives.
`to_ascii_uppercase()`	Echo transform; CR is converted to CRLF for terminals.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Open `espflash monitor` or `picocom /dev/ttyACM0`: the banner appears and every typed lowercase letter returns in upper case.

Representative log lines:

```
Lab 22a: type something, it comes back in UPPER CASE
hello  ->  HELLO
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                        # once per shell
cargo build --release --bin lab22a_usb_serial_jtag     # compile only
cargo run   --release --bin lab22a_usb_serial_jtag     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab22a_usb_serial_jtag
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`/dev/ttyACM0` missing	Cable is charge-only or you used the UART port instead; check `dmesg`.
Permission denied	Join the `dialout` group.
Blocks with no host	Writes can stall if the host does not read: keep a terminal open.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::usb_serial_jtag
- Official example: examples/async/embassy_usb_serial_jtag

Lab 22B - USB OTG: CDC-ACM serial device with embassy-usb

Item	Value
Source file	`src/bin/lab22b_usb_otg_cdc.rs`
Verification status	BUILD-VERIFIED (adapted from the official embassy_usb_serial example). NOT tested on hardware
API stability	UNSTABLE: `otg_fs::{Usb, asynch::Driver}`; embassy-usb 0.6
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Turn the ESP32-S3 into a custom USB full-speed device using the OTG core and the `embassy-usb` stack.

2. Required components

- Native USB connector wired to GPIO19/GPIO20
- A way to enter download mode (BOOT + RESET) and, ideally, a second serial connection for logs

3. Wiring

Signal / part	ESP32-S3 pin	Note
USB D-	GPIO19	Fixed; shared with USB Serial/JTAG
USB D+	GPIO20	Fixed

4. Hardware principle

The OTG full-speed core is a configurable USB controller (12 Mbit/s) which can implement any USB class in software. `embassy-usb` provides the device stack (descriptors, enumeration, endpoints); the CDC-ACM class presents a virtual serial port. **GPIO19/GPIO20 are shared with the USB Serial/JTAG block**, so while this firmware runs the programming/logging interface on that connector is replaced by your device. To flash again, hold BOOT, tap RESET, then flash. For logs use UART0 (GPIO43/44) with a USB-UART bridge and build `esp-println` with its `uart` feature (see chapter 8).

5. Complete Rust source code

File `src/bin/lab22b_usb_otg_cdc.rs` (unmodified copy of the compiled file):

```
//! Lab 22b - USB OTG (full-speed device): CDC-ACM virtual serial port with embassy-usb
//!
//! Adapted from the official esp-hal-v1.1.1 example `examples/async/embassy_usb_serial`.
//! The USB OTG FS core uses the SAME two pins as USB Serial/JTAG: GPIO19 (D-) and GPIO20
(D+).
//! Consequences you must understand:
//!  * While this firmware runs, the USB Serial/JTAG logger/flash path on that connector
is gone.
//!    Console output (esp-println) goes to UART0 only if you build with the `uart`
feature of
//!    esp-println and use a USB-UART bridge on GPIO43/44. We use plain `println!` here,
which
//!    in the default `auto` mode is routed to the USB Serial/JTAG block and may
therefore be lost.
//!  * To re-flash, hold BOOT, tap RESET (enter ROM download mode), then flash again.
//! The enumerated device appears as "USB-serial example" (VID 0x303A, PID 0x3001);
anything
//! you type is echoed back in upper case.
//! API level: UNSTABLE (`esp_hal::otg_fs`) + embassy-usb 0.6.
#![no_std]
#![no_main]

use embassy_executor::Spawner;
```

```rust
use embassy_futures::join::join;
use embassy_usb::{
    Builder,
    class::cdc_acm::{CdcAcmClass, State},
    driver::EndpointError,
};
use esp_backtrace as _;
use esp_hal::{
    interrupt::software::SoftwareInterruptControl,
    otg_fs::{
        Usb,
        asynch::{Config, Driver},
    },
    timer::timg::TimerGroup,
};

esp_bootloader_esp_idf::esp_app_desc!();

#[esp_rtos::main]
async fn main(_spawner: Spawner) {
    let peripherals = esp_hal::init(esp_hal::Config::default());
    esp_alloc::heap_allocator!(size: 16 * 1024); // see Cargo.toml note: esp-rtos alloc
support is on

    let sw_int = SoftwareInterruptControl::new(peripherals.SW_INTERRUPT);
    let timg0 = TimerGroup::new(peripherals.TIMG0);
    esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);

    // Takes the USB0 peripheral plus its two data pins (note the order: D+ first).
    let usb = Usb::new(peripherals.USB0, peripherals.GPIO20, peripherals.GPIO19);

    let mut ep_out_buffer = [0u8; 1024];
    let driver = Driver::new(usb, &mut ep_out_buffer, Config::default());

    let mut config = embassy_usb::Config::new(0x303A, 0x3001);
    config.manufacturer = Some("Espressif");
    config.product = Some("USB-serial example");
    config.serial_number = Some("12345678");
    // Required for Windows compatibility (interface association descriptors).
    config.device_class = 0xEF;
    config.device_sub_class = 0x02;
    config.device_protocol = 0x01;
    config.composite_with_iads = true;

    let mut config_descriptor = [0; 256];
    let mut bos_descriptor = [0; 256];
    let mut control_buf = [0; 64];
    let mut state = State::new();

    let mut builder = Builder::new(
        driver,
        config,
        &mut config_descriptor,
        &mut bos_descriptor,
        &mut [], // no Microsoft OS descriptors
        &mut control_buf,
    );
    let mut class = CdcAcmClass::new(&mut builder, &mut state, 64);
    let mut usb = builder.build();

    let usb_fut = usb.run(); // enumeration, control transfers, ...
    let echo_fut = async {
        loop {
            class.wait_connection().await;
            let _ = echo(&mut class).await;
        }
    };
    join(usb_fut, echo_fut).await;
}

async fn echo<'d>(class: &mut CdcAcmClass<'d, Driver<'d>>) -> Result<(), Disconnected> {
    let mut buf = [0; 64];
```

```
    loop {
        let n = class.read_packet(&mut buf).await?;
        for c in buf[..n].iter_mut() {
            if c.is_ascii_lowercase() {
                *c = c.to_ascii_uppercase();
            }
        }
        class.write_packet(&buf[..n]).await?;
    }
}

struct Disconnected;

impl From<EndpointError> for Disconnected {
    fn from(val: EndpointError) -> Self {
        match val {
            EndpointError::BufferOverflow => panic!("USB buffer overflow"),
            EndpointError::Disabled => Disconnected,
        }
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`Usb::new(peripherals.USB0, peripherals.GPIO20, peripherals.GPIO19)`	Takes USB0 plus D+ then D- pins.
`Driver::new(usb, &mut ep_out_buffer, Config::default())`	The embassy-usb driver (endpoint buffers, interrupt).
`embassy_usb::Config::new(0x303A, 0x3001)`	VID/PID: Espressif's VID with PID 0x3001 as used by the official example; use your own for a product.
`config.composite_with_iads = true` etc.	Class codes required for Windows compatibility.
`Builder::new(..)` and `CdcAcmClass::new(&mut builder, &mut state, 64)`	Build descriptors and the CDC class with 64-byte packets.
`join(usb.run(), echo_fut)`	Runs the USB device and the echo loop concurrently.
`class.read_packet` / `write_packet`	Bulk endpoint transfers.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The host enumerates a serial port named `USB-serial example`; typed lowercase letters are echoed in upper case.

Representative log lines:

```
(host) lsusb: ID 303a:3001 Espressif USB-serial example
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab22b_usb_otg_cdc     # compile only
cargo run   --release --bin lab22b_usb_otg_cdc     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab22b_usb_otg_cdc
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Cannot reflash	Enter download mode with BOOT + RESET (USB Serial/JTAG is not running).
Device not enumerating	Wrong pin order in `Usb::new`, or 5 V on VBUS missing.
No logs	`println!` goes to the USB Serial/JTAG block in `auto` mode, which is not active here.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::otg_fs
- embassy-usb
- Official example: examples/async/embassy_usb_serial

Lab 23 - Asynchronous programming with Embassy

Item	Value
Source file	`src/bin/lab23_embassy.rs`
Verification status	BUILD-VERIFIED; `Input::wait_for_falling_edge` is a stable esp-hal method; esp-rtos 0.3.0 / embassy-executor 0.10 are companion crates
API stability	`#[esp_rtos::main]`, `embassy_executor`, `embassy_time`, `embassy_sync::Signal`; `gpio::Input` async methods
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Run several concurrent activities (blink, button, heartbeat) on one core with async tasks, a timer-driven `Ticker`, and a `Signal` for inter-task communication.

2. Required components

- 1 x LED + 330 ohm
- 1 x push button

3. Wiring

Signal / part	ESP32-S3 pin	Note
LED anode via 330 ohm	GPIO21	
Button	GPIO14 to GND	Internal pull-up

4. Hardware principle

Rust `async fn` compiles to a state machine (a *future*) that yields at each `.await`. The Embassy *executor* polls futures cooperatively on one thread: when a task awaits (for a timer, a pin edge, a message), the executor runs another ready task, and the CPU sleeps when nothing is ready. No stack per task and no preemption mean low overhead, but a task that blocks (busy-waits) stalls all others. Hardware events wake tasks through interrupts: `embassy_time` uses the timer set up by `esp_rtos::start`, and `Input::wait_for_falling_edge` completes inside the GPIO interrupt. A `Signal` is a one-slot mailbox holding the latest value; `select` waits for the first of two futures.

5. Complete Rust source code

File `src/bin/lab23_embassy.rs` (unmodified copy of the compiled file):

```
//! Lab 23 - Asynchronous programming with Embassy (esp-rtos executor)
//!
//! Wiring: LED GPIO21 --[330 ohm]--> GND;  button GPIO14 --> GND (internal pull-up).
//! Four concurrent activities share ONE core without threads or interrupts in user code:
//!   blinker      - blinks the LED; its period can change at any moment
//!   button task  - awaits a falling edge (no polling!), then cycles the blink period
//!   main         - prints a heartbeat with a drift-free `Ticker`
//! Tasks communicate through an Embassy `Signal` (latest-value mailbox).
//! `await` points are where the executor switches tasks: never block inside a task.
//! API level: `Input::wait_for_falling_edge` is a STABLE esp-hal method in 1.1.1 (the
generic
//! `wait_for` is unstable); esp-rtos 0.3.0 and embassy-executor 0.10 are companion
crates.
#![no_std]
#![no_main]

use embassy_executor::Spawner;
use embassy_futures::select::{Either, select};
```

```rust
use embassy_sync::{blocking_mutex::raw::CriticalSectionRawMutex, signal::Signal};
use embassy_time::{Duration, Ticker, Timer};
use esp_backtrace as _;
use esp_hal::{
    gpio::{Input, InputConfig, Level, Output, OutputConfig, Pull},
    interrupt::software::SoftwareInterruptControl,
    timer::timg::TimerGroup,
};
use log::info;

esp_bootloader_esp_idf::esp_app_desc!();

/// New blink half-period in milliseconds, written by `button task`, read by `blinker`.
static BLINK_MS: Signal<CriticalSectionRawMutex, u64> = Signal::new();

#[esp_rtos::main]
async fn main(spawner: Spawner) -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    esp_alloc::heap_allocator!(size: 16 * 1024); // see Cargo.toml note: esp-rtos alloc
support is on

    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let sw_int = SoftwareInterruptControl::new(peripherals.SW_INTERRUPT);
    esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);

    let led = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());
    let button = Input::new(peripherals.GPIO14,
InputConfig::default().with_pull(Pull::Up));

    // `spawn` needs `Result<SpawnToken, SpawnError>::unwrap()` in embassy-executor 0.10.
    spawner.spawn(blinker(led).unwrap());
    spawner.spawn(button_task(button).unwrap());

    let mut ticker = Ticker::every(Duration::from_secs(2));
    let mut beats = 0u32;
    loop {
        ticker.next().await;
        beats += 1;
        info!("heartbeat #{}", beats);
    }
}

#[embassy_executor::task]
async fn blinker(mut led: Output<'static>) {
    let mut half_period = 500u64;
    loop {
        led.toggle();
        // Wait for EITHER the timer OR a new period, whichever happens first.
        match select(Timer::after_millis(half_period), BLINK_MS.wait()).await {
            Either::First(()) => {}
            Either::Second(new_ms) => {
                info!("blink half-period -> {} ms", new_ms);
                half_period = new_ms;
            }
        }
    }
}

#[embassy_executor::task]
async fn button_task(mut button: Input<'static>) {
    const PERIODS: [u64; 4] = [500, 250, 100, 1000];
    let mut idx = 0;
    loop {
        button.wait_for_falling_edge().await; // sleeps until the pin goes low
        idx = (idx + 1) % PERIODS.len();
        BLINK_MS.signal(PERIODS[idx]);
        Timer::after_millis(200).await; // crude debounce
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`#[esp_rtos::main] async fn main(spawner: Spawner)`	Creates the executor and runs `main` as the first task.
`esp_rtos::start(timg0.timer0, sw_int.software_interrupt0)`	Provides the time driver and scheduler.
`spawner.spawn(blinker(led).unwrap())`	Spawns a task (the macro-generated function returns `Result<SpawnToken, SpawnError>`).
`static BLINK_MS: Signal<CriticalSectionRawMutex, u64>`	Mailbox between tasks.
`select(Timer::after_millis(p), BLINK_MS.wait()).await`	Waits for the timer or a new period.
`button.wait_for_falling_edge().await`	Sleeps until the pin goes low; no polling.
`Ticker::every(Duration::from_secs(2))`	Drift-free periodic wake-up for the heartbeat.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

The LED blinks at 1 Hz; each button press cycles the half-period 500 -> 250 -> 100 -> 1000 ms; the heartbeat line prints every 2 s even while the blink period changes.

Representative log lines:

```
INFO - heartbeat #1
INFO - blink half-period -> 250 ms
INFO - heartbeat #2
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                       # once per shell
cargo build --release --bin lab23_embassy     # compile only
cargo run   --release --bin lab23_embassy     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-elf/release/lab23_embassy
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
`error: ... Result ... must be used` when spawning	Use `spawner.spawn(task().unwrap())` with embassy-executor 0.10.
Executor stalls	A `loop {}` or blocking delay inside a task starves others: use `Timer::after(..).await`.
Missing symbol `esp_rtos_*`	`esp_rtos::start` was not called.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- Embassy
- Official example: examples/async/embassy_hello_world

- esp_hal::gpio

Lab 24 - Bonus: external PSRAM as heap

Item	Value
Source file	`src/bin/lab24_psram.rs`
Verification status	BUILD-VERIFIED; requires the `unstable` feature. Needs a PSRAM module; NOT hardware-tested
API stability	UNSTABLE: `esp_hal::psram`; `esp_alloc::psram_allocator!`
Not hardware-tested	Yes: behaviour below is the specification of the code, not an observed result

1. Objective

Add the external PSRAM to the heap and allocate a buffer too large for internal RAM.

2. Required components

- ESP32-S3 module with PSRAM (e.g. WROOM-1 N16R8 / N8R8 octal PSRAM, N16R2 quad)

3. Wiring

No wiring. PSRAM pins are internal to the module.

4. Hardware principle

The ESP32-S3 has 512 KB of internal SRAM; camera frames or large buffers need external RAM. PSRAM (pseudo-static RAM) is accessed through the SPI interface (quad or octal) and mapped into the address space through the cache, so it is slower than internal RAM. In `PsramMode::Auto` the driver tries octal then quad. Pins used by PSRAM or flash cannot be used as GPIO: in esp-hal's metadata GPIO26-32 are flash/PSRAM pins and GPIO33-37 are octal flash/PSRAM pins; Espressif's module datasheet states which are routed out on your module. Warning from the esp-alloc documentation: atomic instructions do not work correctly in PSRAM, so never put atomics (or `Arc`) there.

5. Complete Rust source code

File `src/bin/lab24_psram.rs` (unmodified copy of the compiled file):

```
//! Lab 24 (bonus) - External PSRAM as heap memory
//!
//! No wiring. Requires a board whose module has PSRAM (e.g. ESP32-S3-WROOM-1 N16R8 / N8R8:
//! octal PSRAM; N16R2 / N4R2: quad PSRAM). `PsramMode::Auto` (default) tries octal, then quad.
//! IMPORTANT consequences for this manual's pin plan:
//!   * Octal PSRAM (R8) occupies GPIO35, GPIO36, GPIO37 -> never use them as GPIO.
//!   * GPIO26..GPIO32 (and GPIO33/34 with octal flash) belong to flash/PSRAM too.
//! WARNING: atomic instructions do not work correctly in PSRAM (esp-alloc docs). Never allocate
//! `Atomic*` types (or anything containing them, e.g. `Arc`) from the PSRAM heap.
//! Build in release mode only.
//! API level: UNSTABLE (`esp_hal::psram`) + esp-alloc 0.10.0.
#![no_std]
#![no_main]

extern crate alloc;

use alloc::vec::Vec;

use esp_backtrace as _;
use esp_hal::{
    main,
    time::{Duration, Instant},
};
use log::info;
```

```
esp_bootloader_esp_idf::esp_app_desc!();

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());

    // Small internal-RAM heap + the PSRAM as a second heap region.
    esp_alloc::heap_allocator!(size: 32 * 1024);
    esp_alloc::psram_allocator!(peripherals.PSRAM, esp_hal::psram);

    info!("heap after init: used {} B, free {} B", esp_alloc::HEAP.used(),
esp_alloc::HEAP.free());

    // 500 KiB cannot fit in the 32 KiB internal heap, so it must come from PSRAM.
    let words = 500 * 1024 / 4;
    let mut big: Vec<u32> = Vec::with_capacity(words);
    for i in 0..words {
        big.push((i & 0xFF) as u32);
    }
    let checksum: u32 = big.iter().fold(0u32, |a, &b| a.wrapping_add(b));
    info!("allocated {} KiB, checksum {}", words * 4 / 1024, checksum);
    info!("heap after alloc: used {} B, free {} B", esp_alloc::HEAP.used(),
esp_alloc::HEAP.free());

    loop {
        let t = Instant::now();
        while t.elapsed() < Duration::from_secs(5) {}
        info!("still alive, vector[1000] = {}", big[1000]);
    }
}
```

6. Explanation of the important functions, methods, types and parameters

Item	Explanation
`esp_alloc::heap_allocator!(size: 32 * 1024)`	Internal heap region.
`esp_alloc::psram_allocator!(peripherals.PSRAM, esp_hal::psram)`	Initialises the PSRAM and registers it as an external heap region.
`Vec::with_capacity(500 * 1024 / 4)`	500 KiB cannot fit the internal heap, so it must come from PSRAM.
`esp_alloc::HEAP.used()` / `.free()`	Heap statistics.
Release builds only	The PSRAM driver docs require `--release`.

Full signatures and stability of the esp-hal items are in Part III.

7. Expected output or behaviour

Heap statistics before and after, `allocated 500 KiB, checksum ...` and a periodic `still alive` message.

Representative log lines:

```
INFO - heap after init: used 0 B, free 33xxx B + PSRAM
INFO - allocated 500 KiB, checksum ...
```

8. Compilation and flashing

```
cd esp32s3-labs
. "$HOME/export-esp.sh"                      # once per shell
cargo build --release --bin lab24_psram     # compile only
cargo run   --release --bin lab24_psram     # compile, flash, open monitor
```

Equivalent manual flashing:

```
espflash flash --monitor --chip esp32s3 target/xtensa-esp32s3-none-
elf/release/lab24_psram
```

9. Common errors and troubleshooting

Symptom	Likely cause and remedy
Panic or hang at PSRAM init	Board has no PSRAM, or mode mismatch (octal vs quad); check the module name.
Build fails in debug	Build with `--release`.
Atomics misbehave	Do not allocate atomics in PSRAM.

These are the likely causes from the code and hardware principles; none was observed on a real board.

10. Documentation links

- esp_hal::psram
- esp-alloc

Part III - ESP-HAL 1.1.1 API reference

How this reference was produced

The reference documents the functions, methods and types **used in the laboratories and projects**, organised by peripheral. It is not a copy of the full rustdoc, which is available at https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/.

For every entry the *signature*, the *stability* and the *source line* were **extracted automatically from the tag `esp-hal-v1.1.1`**. The generator stops with an error if an API name or an enum variant cannot be found, so no function name or signature in this part is invented. The prose (purpose, mechanism, parameters, example, limitations) was written by the author and then reviewed against the extracted signatures; that review caught several errors in the first draft (for example a wrong receiver type, several wrong return types, and stability labels that were too optimistic or too pessimistic), which were corrected.

Each entry follows this pattern:

- **Signature**: verbatim from the source (generics trimmed of attribute noise). Builder methods that are generated by `#[derive(BuilderLite)]` are shown as `fn with_<field>(self, <field>: T) -> Self`.
- **Stability**: *Stable*, or *UNSTABLE* (needs the feature `unstable`) with the reason: the whole module is unstable, or the single item carries `#[instability::unstable]`. *Companion crate* marks APIs of `esp-rtos`, `esp-radio`, `esp-alloc`, `esp-println` and `esp-bootloader-esp-idf`, which are versioned separately (their versions are in Appendix E).
- **Purpose / How it works (conceptually) / Parameters and return / Example / Limits**.
- **Source**: file and line in the 1.1.1 tree, so you can read the implementation.

Where the source marks an API as chip-specific with `#[cfg(...)]`, the attribute is quoted. Items of third-party crates (`embedded-hal`, `embassy-*`, `mfrc522`, `trouble-host`) are *not* signature-checked here; they are listed in Appendix E with versions and links.

Stable versus unstable modules

Module	Status	Provides	Used in
`clock`	**Stable**	CPU clock presets and queries	Lab 03
`efuse`	**Stable**	eFuse access	-
`gpio`	**Stable**	Digital I/O, interrupts, async pins	Labs 01, 02, 08
`i2c`	**Stable**	I2C master	Labs 10, 11
`interrupt`	**Stable**	Interrupt handler types	Labs 07, 08
`peripherals`	**Stable**	Peripherals singleton struct	all
`rng`	**Stable**	Random numbers	Labs 20, Project 3
`spi`	**Stable**	SPI master	Labs 12, 13, 14b
`system`	**Stable**	Reset, reset reason	Lab 15
`time`	**Stable**	Instant, Duration, Rate	all
`uart`	**Stable**	UART	Lab 09
`aes`	UNSTABLE	AES accelerator	-
`analog`	UNSTABLE	ADC (and DAC on other chips)	Lab 04, Projects 1, 3
`delay`	UNSTABLE	Blocking Delay	Labs 05, 06, 11
`dma`	UNSTABLE	GDMA, buffers, Mem2Mem	Lab 14
`ecc`	UNSTABLE	ECC accelerator	-

Module	Status	Provides	Used in
`etm`	UNSTABLE	Event task matrix	-
`hmac`	UNSTABLE	HMAC accelerator	-
`i2s`	UNSTABLE	I2S audio bus	-
`lcd_cam`	UNSTABLE	LCD and camera interface	camera (not covered)
`ledc`	UNSTABLE	LED PWM controller	Labs 05, 06, Projects 1, 2
`mcpwm`	UNSTABLE	Motor-control PWM	Lab 18
`otg_fs`	UNSTABLE	USB OTG full speed	Lab 22b
`parl_io`	UNSTABLE	Parallel IO	-
`pcnt`	UNSTABLE	Pulse counter	Lab 17
`psram`	UNSTABLE	External PSRAM	Lab 24
`rmt`	UNSTABLE	Remote-control transceiver	Lab 16
`rsa`	UNSTABLE	RSA accelerator	-
`rtc_cntl`	UNSTABLE	RTC control, sleep	-
`sha`	UNSTABLE	SHA accelerator	-
`timer`	UNSTABLE	Timer groups, SYSTIMER, watchdog	Labs 07, 15
`touch`	UNSTABLE	Capacitive touch	-
`tsens`	UNSTABLE	Temperature sensor: no ESP32-S3 driver in 1.1.1	Project 3 note
`twai`	UNSTABLE	CAN 2.0 controller	Lab 19
`usb_serial_jtag`	UNSTABLE	Built-in USB serial/JTAG	Lab 22a

Other unstable internals (`asynch`, `debugger`, `assist_debug`, `rom`, `sync`, `trace`) are not used. Unstable modules need `esp-hal = { features = ["unstable"] }`.

Inside these modules individual items can also be unstable. Examples confirmed in the source: `esp_hal::system::reset_reason`, `esp_hal::gpio::Io::new`, `esp_hal::gpio::Input::listen`, `esp_hal::spi::master::Spi::with_cs`, `esp_hal::uart::Uart::split`, and the attribute macro `#[handler]`. The `main` and `ram` attribute macros are stable.

1. Initialisation, clocks, time and delay

`esp_hal::init`

Stability: Stable (crate-root item) | **Source:** `esp-hal/src/lib.rs:718` | **Conditional on:** `feature = "rt"`

```
pub fn init(config: Config) -> Peripherals
```

Purpose. Brings the chip into a known state and returns the one-and-only `Peripherals` value.

How it works. Applies the CPU-clock request, initialises internal clock/system state, DISABLES the RWDT, SWD, TIMG0 and TIMG1 watchdogs, and hands out ownership of every peripheral and GPIO as a separate zero-sized token.

Parameters and return. `config`: `esp_hal::Config` (use `Config::default()` and `with_*` builders). Returns `Peripherals`.

Example.

```
let peripherals = esp_hal::init(esp_hal::Config::default());
```

Limits. Call exactly once, first. A second call is a bug: ownership of all peripherals would be duplicated.

esp_hal::Config::with_cpu_clock

Stability: Stable (crate-root item) | **Source:** esp-hal/src/lib.rs:652

```
fn with_cpu_clock(self, cpu_clock: ClockConfig) -> Self   // generated by
#[derive(BuilderLite)]
```

Purpose. Selects the CPU clock preset applied during `init`.

How it works. Stored in `Config` and applied by `init` when it configures the PLL/CPU clock tree.

Parameters and return. `cpu_clock`: `ClockConfig`; a `CpuClock` preset converts into it (so `with_cpu_clock(CpuClock::max())` works).

Example.

```
esp_hal::Config::default().with_cpu_clock(CpuClock::max())
```

Limits. ESP32-S3 default is 80 MHz (see Lab 03). The clock cannot be changed after `init` with the stable API.

esp_hal::clock::CpuClock

Stability: Stable | **Source:** esp-hal/src/soc/esp32s3/clocks.rs:35

```
pub enum CpuClock
```

Purpose. Enumerates the CPU clock presets of the ESP32-S3.

How it works. Each variant is a frozen PLL/divider configuration; `CpuClock::max()` returns the highest one.

Parameters and return. Variants for ESP32-S3: `_80MHz` (default), `_160MHz`, `_240MHz`.

Example.

```
CpuClock::_240MHz
```

Limits. Higher clock = more current and heat; flash/PSRAM timing is re-derived by the HAL.

esp_hal::clock::cpu_clock

Stability: Stable | **Source:** esp-hal/src/clock/mod.rs:523

```
pub fn cpu_clock() -> Rate
```

Purpose. Reports the CPU clock frequency that is currently configured.

How it works. Reads the HAL's recorded clock-tree state, it does not measure the clock.

Parameters and return. No parameters. Returns `esp_hal::time::Rate`.

Example.

```
info!("{} MHz", cpu_clock().as_mhz());
```

esp_hal::clock::xtal_clock

Stability: Stable | **Source:** esp-hal/src/clock/mod.rs:528

```
pub fn xtal_clock() -> Rate
```

Purpose. Reports the crystal frequency (40 MHz on ESP32-S3).

How it works. Reads the recorded XTAL frequency.

Parameters and return. No parameters. Returns `Rate`.

Example.

```
let x = xtal_clock();
```

esp_hal::system::reset_reason

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/system.rs:381`

```
pub fn reset_reason() -> Option<SocResetReason>
```

Purpose. Tells why the CPU last reset (power-on, software, watchdog, …).

How it works. Reads the reset-cause field from the RTC control block for the current core.

Parameters and return. No parameters. Returns `Option<SocResetReason>`.

Example.

```
info!("{:?}", reset_reason());
```

Limits. UNSTABLE item inside the stable `system` module (marked `#[instability::unstable]`). Returns `None` if the cause is not recognised.

esp_hal::system::software_reset

Stability: Stable | **Source:** `esp-hal/src/system.rs:366`

```
pub fn software_reset() -> !
```

Purpose. Resets the whole chip from software.

How it works. Calls the ROM software-reset routine; never returns.

Parameters and return. No parameters. Returns `!`.

Example.

```
esp_hal::system::software_reset();
```

esp_hal::time::Rate::from_mhz

Stability: Stable | **Source:** `esp-hal/src/time.rs:91`

```
pub const fn from_mhz(val: u32) -> Self
```

Purpose. Constructs a frequency value in MHz (`from_hz` and `from_khz` are siblings).

How it works. `Rate` wraps a `u32` Hz count; the constructors are `const fn`.

Parameters and return. `val: u32`. Returns `Rate`.

Example.

```
Rate::from_khz(100)
```

Limits. from_hz / from_khz verified in the same file (lines 59 and 75).

esp_hal::time::Rate::as_mhz

Stability: Stable | **Source:** `esp-hal/src/time.rs:142`

```
pub const fn as_mhz(&self) -> u32
```

Purpose. Converts a `Rate` to whole megahertz.

How it works. Integer division of the stored Hz value.

Parameters and return. `&self`. Returns `u32`.

Example.

```
cpu_clock().as_mhz()
```

esp_hal::time::Instant::now

Stability: Stable | **Source:** esp-hal/src/time.rs:266

```
pub fn now() -> Self
```

Purpose. Reads the free-running 64-bit microsecond system timer.

How it works. Reads the SYSTIMER counter; monotonic since boot, no overflow in practice.

Parameters and return. No parameters. Returns `Instant`.

Example.

```
let t = Instant::now();
```

esp_hal::time::Instant::elapsed

Stability: Stable | **Source:** esp-hal/src/time.rs:305

```
pub fn elapsed(&self) -> Duration
```

Purpose. Time elapsed since this `Instant` was taken.

How it works. Subtracts the stored timestamp from `Instant::now()`.

Parameters and return. `&self`. Returns `Duration`.

Example.

```
while t.elapsed() < Duration::from_millis(500) {}
```

Limits. A busy-wait: burns CPU. Prefer timers/async for long waits.

esp_hal::time::Instant::duration_since_epoch

Stability: Stable | **Source:** esp-hal/src/time.rs:288

```
pub fn duration_since_epoch(&self) -> Duration
```

Purpose. Time since boot as a `Duration`.

How it works. Converts the raw tick count.

Parameters and return. `&self`. Returns `Duration`.

Example.

```
Instant::now().duration_since_epoch().as_millis()
```

esp_hal::time::Duration::from_millis

Stability: Stable | **Source:** esp-hal/src/time.rs:420

```
pub const fn from_millis(val: u64) -> Self
```

Purpose. Creates a `Duration` from milliseconds.

How it works. `const fn` over a microsecond-resolution integer.

Parameters and return. `val: u64`. Returns `Duration`.

Example.

`Duration::from_millis(250)`

Limits. `from_micros`, `from_secs`, `from_minutes`, `from_hours` are siblings in the same file.

esp_hal::time::Duration::as_millis

Stability: Stable | **Source:** `esp-hal/src/time.rs:503`

```
pub const fn as_millis(&self) -> u64
```

Purpose. Converts a `Duration` to whole milliseconds.

How it works. Integer division.

Parameters and return. `&self`. Returns `u64`.

Example.

```
d.as_millis()
```

esp_hal::time::Duration::as_micros

Stability: Stable | **Source:** `esp-hal/src/time.rs:488`

```
pub const fn as_micros(&self) -> u64
```

Purpose. Converts a `Duration` to whole microseconds.

How it works. Identity on the internal unit.

Parameters and return. `&self`. Returns `u64`.

Example.

```
t0.elapsed().as_micros()
```

esp_hal::delay::Delay::new

Stability: UNSTABLE (module `delay` requires feature `unstable`) | **Source:** `esp-hal/src/delay.rs:48`

```
pub const fn new() -> Self
```

Purpose. Creates a blocking delay provider.

How it works. Stateless wrapper over the system timer.

Parameters and return. No parameters. Returns `Delay`.

Example.

```
let delay = Delay::new();
```

Limits. UNSTABLE module (`esp_hal::delay`). Stable code can use `Instant` busy-waits instead.

esp_hal::delay::Delay::delay_millis

Stability: UNSTABLE (module `delay` requires feature `unstable`) | **Source:** `esp-hal/src/delay.rs:60`

```
pub fn delay_millis(&self, ms: u32)
```

Purpose. Blocks the CPU for N milliseconds.

How it works. Spin-waits on `Instant`.

Parameters and return. `&self`, `ms: u32`. Returns `()`.

Example.

```
delay.delay_millis(500);
```

Limits. Blocking: do not call it inside async tasks.

esp_hal::delay::Delay::delay_micros

Stability: UNSTABLE (module `delay` requires feature `unstable`) | **Source:** `esp-hal/src/delay.rs:65`

```
pub fn delay_micros(&self, us: u32)
```

Purpose. Blocks for N microseconds.

How it works. Spin-waits.

Parameters and return. `&self`, `us: u32`. Returns `()`.

Example.

```
delay.delay_micros(50);
```

esp_hal::delay::Delay::delay_nanos

Stability: UNSTABLE (module `delay` requires feature `unstable`) | **Source:** `esp-hal/src/delay.rs:70`

```
pub fn delay_nanos(&self, ns: u32)
```

Purpose. Blocks for N nanoseconds (resolution limited by the timer).

How it works. Spin-waits.

Parameters and return. `&self`, `ns: u32`. Returns `()`.

Example.

```
delay.delay_nanos(200);
```

Limits. Real resolution is far coarser than 1 ns.

2. GPIO

esp_hal::gpio::Level

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:188`

```
pub enum Level
```

Purpose. Logic level of a pin.

How it works. `Low` = 0, `High` = 1; implements `Not`.

Parameters and return. Variants: `Low`, `High`.

Example.

```
Level::High
```

esp_hal::gpio::Pull

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:271`

```
pub enum Pull
```

Purpose. Selects the internal pull resistor of an input.

How it works. Enables the pad's ~45 kOhm pull-up or pull-down in the IO MUX.

Parameters and return. Variants: `None`, `Up`, `Down`.

Example.

```
Pull::Up
```

Limits. Internal resistors are weak; use external ones for long wires or I2C.

esp_hal::gpio::Event

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/gpio/mod.rs:144`

```
pub enum Event
```

Purpose. Edge/level condition that triggers a GPIO interrupt.

How it works. Programmed into the pad's interrupt-type field.

Parameters and return. Variants: `RisingEdge`, `FallingEdge`, `AnyEdge`, `LowLevel`, `HighLevel`.

Example.

```
Event::FallingEdge
```

esp_hal::gpio::DriveMode

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:722`

```
pub enum DriveMode
```

Purpose. Output driver type.

How it works. `PushPull` drives both levels; `OpenDrain` only pulls low (needs a pull-up).

Parameters and return. Variants: `PushPull`, `OpenDrain`.

Example.

```
DriveMode::PushPull
```

esp_hal::gpio::Output::new

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:820`

```
pub fn new(pin: impl OutputPin + 'd, initial_level: Level, config: OutputConfig) -> Self
```

Purpose. Turns a GPIO token into a digital output.

How it works. Configures the pad as output through the GPIO matrix, applies the config and sets the initial level BEFORE enabling the driver (no glitch). Consumes the token.

Parameters and return. `pin`: GPIO token (e.g. `peripherals.GPIO21`); `initial_level: Level`; `config: OutputConfig`. Returns `Output`.

Example.

```
Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default())
```

esp_hal::gpio::Output::set_high

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:890`

```
pub fn set_high(&mut self)
```

Purpose. Drives the pin high.

How it works. Writes the GPIO set register (atomic).

Parameters and return. `&mut self`. Returns `()`.

Example.

```
led.set_high();
```

esp_hal::gpio::Output::set_low

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:908`

```
pub fn set_low(&mut self)
```

Purpose. Drives the pin low.

How it works. Writes the GPIO clear register.

Parameters and return. `&mut self`. Returns `()`.

Example.

```
led.set_low();
```

esp_hal::gpio::Output::set_level

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:926`

```
pub fn set_level(&mut self, level: Level)
```

Purpose. Drives the pin to a given `Level`.

How it works. Dispatches to set/clear.

Parameters and return. `level: Level`.

Example.

```
alarm.set_level(Level::High);
```

esp_hal::gpio::Output::toggle

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:1010`

```
pub fn toggle(&mut self)
```

Purpose. Inverts the output level.

How it works. Reads the output register and writes the opposite.

Parameters and return. `&mut self`. Returns `()`.

Example.

```
led.toggle();
```

esp_hal::gpio::Output::is_set_high

Stability: Stable | **Source:** `esp-hal/src/gpio/mod.rs:947`

```
pub fn is_set_high(&self) -> bool
```

Purpose. Reports the level the driver is currently commanding.

How it works. Reads the output register (not the pad).

Parameters and return. `&self`. Returns `bool`.

Example.

```
led.is_set_high()
```

esp_hal::gpio::Input::new

Stability: Stable | **Source:** esp-hal/src/gpio/mod.rs:1090

```
pub fn new(pin: impl InputPin + 'd, config: InputConfig) -> Self
```

Purpose. Turns a GPIO token into a digital input.

How it works. Configures the pad as input with the requested pull; the pin is high-impedance otherwise.

Parameters and return. `pin`: GPIO token; `config: InputConfig`. Returns `Input`.

Example.

```
Input::new(peripherals.GPIO14, InputConfig::default().with_pull(Pull::Up))
```

esp_hal::gpio::InputConfig::with_pull

Stability: Stable | **Source:** esp-hal/src/gpio/mod.rs:1028

```
fn with_pull(self, pull: Pull) -> Self   // generated by #[derive(BuilderLite)]
```

Purpose. Sets the pull resistor in an `InputConfig`.

How it works. Builder (generated); `InputConfig` is `#[non_exhaustive]`, so builders are the only way to construct it.

Parameters and return. `pull: Pull`.

Example.

```
InputConfig::default().with_pull(Pull::Up)
```

esp_hal::gpio::Input::level

Stability: Stable | **Source:** esp-hal/src/gpio/mod.rs:1177

```
pub fn level(&self) -> Level
```

Purpose. Samples the pin.

How it works. Reads the GPIO input register (one APB-clock synchroniser delay).

Parameters and return. `&self`. Returns `Level`.

Example.

```
if button.level() == Level::Low { }
```

esp_hal::gpio::Input::is_high

Stability: Stable | **Source:** esp-hal/src/gpio/mod.rs:1141

```
pub fn is_high(&self) -> bool
```

Purpose. True when the pin reads high.

How it works. Wrapper over `level()`.

Parameters and return. `&self`. Returns `bool`.

Example.

```
button.is_high()
```

esp_hal::gpio::Input::is_low

Stability: Stable | **Source:** esp-hal/src/gpio/mod.rs:1159

```
pub fn is_low(&self) -> bool
```

Purpose. True when the pin reads low.

How it works. Wrapper over `level()`.

Parameters and return. `&self`. Returns `bool`.

Example.

```
button.is_low()
```

esp_hal::gpio::Input::listen

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** esp-hal/src/gpio/mod.rs:1271

```
pub fn listen(&mut self, event: Event)
```

Purpose. Arms a GPIO interrupt for the given `Event`.

How it works. Programs the pad interrupt type and enables it; the shared GPIO interrupt then fires the registered handler.

Parameters and return. `event: Event`.

Example.

```
button.listen(Event::FallingEdge);
```

Limits. UNSTABLE method. Requires a handler registered with `Io::set_interrupt_handler`.

esp_hal::gpio::Input::clear_interrupt

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** esp-hal/src/gpio/mod.rs:1285

```
pub fn clear_interrupt(&mut self)
```

Purpose. Acknowledges a pending GPIO interrupt.

How it works. Writes the interrupt-status clear bit for this pin.

Parameters and return. `&mut self`.

Example.

```
button.clear_interrupt();
```

Limits. UNSTABLE. Forgetting this makes the ISR re-fire forever.

esp_hal::gpio::Input::is_interrupt_set

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** esp-hal/src/gpio/mod.rs:1292

```
pub fn is_interrupt_set(&self) -> bool
```

Purpose. Tells whether THIS pin raised the interrupt.

How it works. Reads the pin's bit in the GPIO interrupt-status register. All pins share one handler, so check before acting.

Parameters and return. `&self`. Returns `bool`.

Example.

```
if button.is_interrupt_set() { ... }
```

Limits. UNSTABLE.

esp_hal::gpio::Input::wait_for_falling_edge

Stability: Stable | **Source:** `esp-hal/src/gpio/asynch.rs:201`

```
pub async fn wait_for_falling_edge(&mut self)
```

Purpose. Async: suspends the task until a falling edge occurs.

How it works. Registers a waker and enables the pin interrupt; the future completes in the GPIO interrupt.

Parameters and return. `&mut self`. Returns a future (`.await`).

Example.

```
button.wait_for_falling_edge().await;
```

Limits. The `Input` method is STABLE in 1.1.1 (only the generic `wait_for` and the `Flex` variants are unstable). It still needs an executor and time driver, e.g. esp-rtos + Embassy.

esp_hal::gpio::Io::new

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/gpio/mod.rs:633`

```
pub fn new(_io_mux: IO_MUX<'d>) -> Self
```

Purpose. Creates the GPIO/IO_MUX interrupt manager.

How it works. Wraps the `IO_MUX` peripheral token so a GPIO interrupt handler can be registered.

Parameters and return. `io_mux: IO_MUX` token. Returns `Io`.

Example.

```
let mut io = Io::new(peripherals.IO_MUX);
```

esp_hal::gpio::Io::set_interrupt_handler

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/gpio/mod.rs:671`

```
pub fn set_interrupt_handler(&mut self, handler: InterruptHandler)
```

Purpose. Registers THE handler for all GPIO interrupts.

How it works. Binds the function to the GPIO CPU interrupt line and enables it.

Parameters and return. `handler: InterruptHandler` (a function annotated with `#[handler]`).

Example.

```
io.set_interrupt_handler(gpio_isr);
```

Limits. UNSTABLE. One handler for all pins: dispatch inside it.

3. ADC (analog input)

esp_hal::analog::adc::Attenuation

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/mod.rs:77`

```
pub enum Attenuation
```

Purpose. Input attenuation selects the measurable voltage range.

How it works. Inserts an analog divider before the SAR converter: more attenuation = higher full-scale input.

Parameters and return. ESP32-S3 variants: `_0dB`, `_2p5dB`, `_6dB`, `_11dB` (about 0.9 V, 1.25 V, 1.75 V, 3.1 V full scale, indicative).

Example.

```
Attenuation::_11dB
```

Limits. Ranges are approximate and unit-specific; see the TRM and calibration.

esp_hal::analog::adc::AdcConfig::new

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/mod.rs:141`

```
pub fn new() -> Self
```

Purpose. Creates an empty ADC configuration.

How it works. Holds the per-pin attenuation choices until the ADC unit is created.

Parameters and return. No parameters. Returns `AdcConfig<ADCI>`.

Example.

```
let mut cfg = AdcConfig::new();
```

esp_hal::analog::adc::AdcConfig::enable_pin

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/mod.rs:146`

```
pub fn enable_pin<PIN>(&mut self, pin: PIN, attenuation: Attenuation) -> AdcPin<PIN, ADCX> where PIN: AdcChannel + AnalogPin
```

Purpose. Registers an analog-capable pin with an attenuation and returns a handle for reading it.

How it works. Records the attenuation for the pin's ADC channel and consumes the pin token (it becomes an analog pad).

Parameters and return. `pin`: analog-capable GPIO token; `attenuation: Attenuation`. Returns `AdcPin`.

Example.

```
cfg.enable_pin(peripherals.GPIO2, Attenuation::_11dB)
```

Limits. Pin must belong to the ADC unit being configured (ADC1: GPIO1..10).

esp_hal::analog::adc::AdcConfig::enable_pin_with_cal

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/mod.rs:165` | **Conditional on:** `feature = "unstable"`; `not(esp32)`

```
pub fn enable_pin_with_cal<PIN, CS>(&mut self, pin: PIN, attenuation: Attenuation) -> AdcPin<PIN, ADCX, CS> where ADCX: CalibrationAccess, PIN: AdcChannel + AnalogPin, CS: AdcCalScheme<ADCX>
```

Purpose. Like `enable_pin` but with a calibration scheme so readings are in millivolts.

How it works. Uses eFuse factory data (curve/line/basic scheme, e.g. `AdcCalCurve<ADC1>`) to correct each raw sample.

Parameters and return. Turbofish `::<_, AdcCalCurve<ADC1>>`; `pin`; `attenuation`. Returns `AdcPin` that yields mV.

Example.

```
cfg.enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(peripherals.GPIO1, Attenuation::_11dB)
```

Limits. Accuracy depends on the chip's eFuse calibration data.

esp_hal::analog::adc::Adc::new

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/xtensa.rs:338`

```
pub fn new(adc_instance: ADCX, config: AdcConfig<ADCX>) -> Self
```

Purpose. Creates the ADC driver for ADC1 (or ADC2).

How it works. Powers and configures the SAR unit with the collected `AdcConfig`.

Parameters and return. `adc_instance: ADC1`/`ADC2` token; `config: AdcConfig`. Returns `Adc` (blocking).

Example.

```
let mut adc = Adc::new(peripherals.ADC1, cfg);
```

Limits. ADC2 cannot be used while Wi-Fi is active.

esp_hal::analog::adc::Adc::read_oneshot

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/xtensa.rs:427`

```
pub fn read_oneshot<PIN, CS>(&mut self, pin: &mut super::AdcPin<PIN, ADCX, CS>) -> nb::Result<u16, ()> where PIN: super::AdcChannel, CS: super::AdcCalScheme<ADCX>
```

Purpose. Starts a conversion and returns the result when ready (non-blocking API).

How it works. Triggers one SAR conversion; returns `WouldBlock` while converting.

Parameters and return. `pin: &mut AdcPin`. Returns `nb::Result<u16, ()>` (raw counts, or mV with a calibrated pin).

Example.

```
nb::block!(adc.read_oneshot(&mut pin)).unwrap()
```

Limits. Wrap in `nb::block!` to spin until ready.

esp_hal::analog::adc::Adc::read_blocking

Stability: UNSTABLE (module `analog` requires feature `unstable`) | **Source:** `esp-hal/src/analog/adc/xtensa.rs:404`

```
pub fn read_blocking<PIN, CS>(&mut self, pin: &mut AdcPin<PIN, ADCX, CS>) -> u16 where PIN: AdcChannel, CS: AdcCalScheme<ADCX>
```

Purpose. Blocking convenience read.

How it works. Loops on `read_oneshot` internally.

Parameters and return. `pin: &mut AdcPin`. Returns `u16`.

Example.

```
adc.read_blocking(&mut pin)
```

4. LEDC (PWM) and servo control

esp_hal::ledc::Ledc::new

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/mod.rs:123`

```
pub fn new(_instance: LEDC<'d>) -> Self
```

Purpose. Creates the LEDC PWM controller.

How it works. Enables the peripheral clock and resets the block.

Parameters and return. `ledc: LEDC` token. Returns `Ledc`.

Example.

```
let mut ledc = Ledc::new(peripherals.LEDC);
```

Limits. UNSTABLE module.

esp_hal::ledc::Ledc::set_global_slow_clock

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/mod.rs:148` | **Conditional on:** `not(esp32)`

```
pub fn set_global_slow_clock(&mut self, clock_source: LSGlobalClkSource)
```

Purpose. Chooses the clock feeding the low-speed timers.

How it works. Writes the LEDC global clock-source field.

Parameters and return. `clock_source: LSGlobalClkSource` (e.g. `APBClk`).

Example.

```
ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);
```

esp_hal::ledc::Ledc::timer

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/mod.rs:176`

```
pub fn timer<S: TimerSpeed>(&self, number: timer::Number) -> Timer<'d, S>
```

Purpose. Returns a handle to one of the LEDC timers.

How it works. Timers define PWM frequency/resolution and are shared by channels.

Parameters and return. `number: timer::Number`. Returns `Timer<'_, S>`.

Example.

```
ledc.timer::<LowSpeed>(timer::Number::Timer0)
```

Limits. ESP32-S3: low-speed mode only, 4 timers, 8 channels.

esp_hal::ledc::Ledc::channel

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/mod.rs:181`

```
pub fn channel<S: TimerSpeed>(&self, number: channel::Number, output_pin: impl PeripheralOutput<'d>) -> Channel<'d, S>
```

Purpose. Creates a PWM channel bound to an output pin.

How it works. Routes the channel's output signal to the pin through the GPIO matrix.

Parameters and return. `number: channel::Number`; `output_pin`: GPIO token. Returns `Channel`.

Example.

```
ledc.channel(channel::Number::Channel0, peripherals.GPIO21)
```

esp_hal::ledc::timer::TimerIFace::configure

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/timer.rs:243`

```
fn configure(&mut self, config: config::Config<S::ClockSourceType>) -> Result<(), Error>
```

Purpose. Applies frequency, resolution and clock source to a timer.

How it works. Computes the 18-bit divider (10.8 fixed point) from source clock, frequency and duty resolution; errors if unreachable.

Parameters and return. `config: timer::config::Config { duty, clock_source, frequency }`. Returns `Result<(), timer::Error>`.

Example.

```
timer0.configure(Config { duty: Duty10Bit, clock_source: LSClockSource::APBClk, frequency: Rate::from_khz(5) })?
```

Limits. Frequency x 2^bits must fit the divider range (50 Hz needs <= 14 bits at 80 MHz APB).

esp_hal::ledc::channel::ChannelIFace::configure

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/channel.rs:170`

```
fn configure(&mut self, config: config::Config<'a, S>) -> Result<(), Error>
```

Purpose. Binds a channel to a timer and sets its initial duty/drive mode.

How it works. Writes the timer selection and initial duty into the channel registers and enables the output.

Parameters and return. `config: channel::config::Config { timer, duty_pct, drive_mode }`. Returns `Result<(), channel::Error>`.

Example.

```
channel0.configure(Config { timer: &timer0, duty_pct: 0, drive_mode: DriveMode::PushPull })?
```

Limits. `duty_pct` must be 0..=100.

esp_hal::ledc::channel::ChannelIFace::set_duty

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/channel.rs:180`

```
fn set_duty(&self, duty_pct: u8) -> Result<(), Error>
```

Purpose. Sets the duty cycle in percent.

How it works. Scales the percentage to the timer's resolution and latches it.

Parameters and return. `duty_pct: u8` (0..=100). Returns `Result<(), channel::Error>`.

Example.

```
channel0.set_duty(50)?
```

Limits. Percent resolution only: use `SetDutyCycle`/`set_duty_hw` for fine control.

esp_hal::ledc::channel::ChannelIFace::start_duty_fade

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/channel.rs:217`

```
fn start_duty_fade(&self, start_duty_pct: u8, end_duty_pct: u8, duration_ms: u16) -> Result<(), Error>
```

Purpose. Starts a hardware-timed linear fade between two duty values.

How it works. Programs the LEDC fade engine (step size and per-step cycle count); no CPU involvement while running.

Parameters and return. `start_duty_pct: u8`, `end_duty_pct: u8`, `duration_ms: u16`. Returns `Result<(), channel::Error>`.

Example.

```
channel0.start_duty_fade(0, 100, 1500)?
```

Limits. Check `is_duty_fade_running()` before starting another fade.

esp_hal::ledc::channel::ChannelIFace::is_duty_fade_running

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/channel.rs:289`

```
fn is_duty_fade_running(&self) -> bool
```

Purpose. Tells whether a fade is still in progress.

How it works. Reads the fade-done flag.

Parameters and return. `&self`. Returns `bool`.

Example.

```
while ch.is_duty_fade_running() {}
```

esp_hal::ledc::timer::config::Duty

Stability: UNSTABLE (module `ledc` requires feature `unstable`) | **Source:** `esp-hal/src/ledc/timer.rs:72`

```
pub enum Duty
```

Purpose. Duty-cycle resolution in bits.

How it works. Number of counter bits: 2^bits duty steps per period.

Parameters and return. Variants include `Duty5Bit` … `Duty14Bit` (see source for the chip's full list).

Example.

```
Duty::Duty14Bit
```

Limits. Resolution trades against maximum frequency.

embedded_hal::pwm::SetDutyCycle::set_duty_cycle (on ledc Channel)

Stability: Stable (crate-root item) | **Source:** esp-hal/src/ledc/channel.rs:328

```
fn set_duty_cycle(&mut self, mut duty: u16) -> Result<(), Self::Error>
```

Purpose. embedded-hal 1.0 duty-cycle setter implemented by ledc::channel::Channel.

How it works. Writes the raw hardware duty value (0..=max_duty_cycle()).

Parameters and return. duty: u16 in timer ticks. Returns Result<(), _>.

Example.

```
servo.set_duty_cycle(ticks)?; // Lab 06
```

Limits. The trait lives in the embedded-hal crate (needs use embedded_hal::pwm::SetDutyCycle).

5. Hardware timers and watchdog

esp_hal::timer::timg::TimerGroup::new

Stability: UNSTABLE (module timer requires feature unstable) | **Source:** esp-hal/src/timer/timg.rs:187 | *(the source has 2 variants; the one for ESP32-S3 / the primary impl is shown)*

```
pub fn new(_timer_group: T) -> Self
```

Purpose. Creates a timer group (TIMG0/TIMG1) giving two general-purpose timers plus a watchdog.

How it works. Enables the group clock and splits it into timer0, timer1 and wdt fields.

Parameters and return. timer_group: TIMG0/TIMG1 token. Returns TimerGroup.

Example.

```
let timg0 = TimerGroup::new(peripherals.TIMG0);
```

Limits. TIMG0 is also needed by esp-rtos for the OS tick (esp_rtos::start(timg0.timer0, ..)).

esp_hal::timer::PeriodicTimer::new

Stability: UNSTABLE (module timer requires feature unstable) | **Source:** esp-hal/src/timer/mod.rs:349

```
pub fn new(inner: impl Timer + Into<AnyTimer<'d>>) -> PeriodicTimer<'d, Blocking>
```

Purpose. Wraps any hardware timer as an auto-reloading periodic timer.

How it works. Stores the timer; start programs the alarm and auto-reload.

Parameters and return. inner: a timer (timg0.timer0, a SYSTIMER alarm). Returns PeriodicTimer<Blocking>.

Example.

```
PeriodicTimer::new(timg0.timer0)
```

esp_hal::timer::PeriodicTimer::start

Stability: UNSTABLE (module timer requires feature unstable) | **Source:** esp-hal/src/timer/mod.rs:362

```
pub fn start(&mut self, period: Duration) -> Result<(), Error>
```

Purpose. Starts the periodic timer with the given period.

How it works. Sets alarm value and auto-reload, enables the counter.

Parameters and return. `period: Duration`. Returns `Result<(), Error>`.

Example.

```
periodic.start(Duration::from_millis(250)).unwrap();
```

Limits. Error if the period is out of range for the timer.

esp_hal::timer::PeriodicTimer::wait

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/mod.rs:378`

```
pub fn wait(&mut self)
```

Purpose. Blocks until the current period expires.

How it works. Spins on the interrupt-raw status, then clears it.

Parameters and return. `&mut self`. Returns `()`.

Example.

```
periodic.wait();
```

esp_hal::timer::PeriodicTimer::listen

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/mod.rs:403`

```
pub fn listen(&mut self)
```

Purpose. Enables the timer interrupt.

How it works. Unmasks the alarm interrupt source.

Parameters and return. `&mut self`.

Example.

```
periodic.listen();
```

esp_hal::timer::PeriodicTimer::clear_interrupt

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/mod.rs:413`

```
pub fn clear_interrupt(&mut self)
```

Purpose. Acknowledges the timer interrupt (call in the ISR).

How it works. Writes the interrupt-clear bit.

Parameters and return. `&mut self`.

Example.

```
timer.clear_interrupt();
```

esp_hal::timer::PeriodicTimer::set_interrupt_handler

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/timer/mod.rs:398` | *(the source has 2 variants; the one for ESP32-S3 / the primary impl is shown)*

```
pub fn set_interrupt_handler(&mut self, handler: InterruptHandler)
```

Purpose. Registers the interrupt service routine for this timer.

How it works. Binds the `#[handler]` function to the timer's interrupt and enables CPU delivery.

Parameters and return. `handler: InterruptHandler`.

Example.

```
periodic.set_interrupt_handler(on_timer);
```

Limits. UNSTABLE. Also available through the `InterruptConfigurable` trait.

esp_hal::timer::timg::Wdt::set_timeout

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/timg.rs:692`

```
pub fn set_timeout(&mut self, stage: MwdtStage, timeout: Duration)
```

Purpose. Sets how long a watchdog stage may run without feeding.

How it works. Converts the duration to timer ticks and writes the stage hold register.

Parameters and return. `stage: MwdtStage`, `timeout: Duration`.

Example.

```
wdt.set_timeout(MwdtStage::Stage0, Duration::from_millis(3000));
```

esp_hal::timer::timg::Wdt::enable

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/timg.rs:612`

```
pub fn enable(&mut self)
```

Purpose. Arms the main-system watchdog (stage 0 = reset the system).

How it works. Unlocks the write-protect, sets stage-0 action to system reset and enables it.

Parameters and return. `&mut self`.

Example.

```
wdt.enable();
```

Limits. `esp_hal::init` leaves all watchdogs DISABLED; you must enable it yourself.

esp_hal::timer::timg::Wdt::feed

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/timg.rs:671`

```
pub fn feed(&mut self)
```

Purpose. Restarts the watchdog countdown.

How it works. Writes the feed register under write-protect.

Parameters and return. `&mut self`.

Example.

```
wdt.feed();
```

esp_hal::timer::timg::Wdt::disable

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/timg.rs:619`

```
pub fn disable(&mut self)
```

Purpose. Disables the watchdog.

How it works. Clears the enable bit under write-protect.

Parameters and return. `&mut self`.

Example.

```
wdt.disable();
```

esp_hal::timer::timg::MwdtStage

Stability: UNSTABLE (module `timer` requires feature `unstable`) | **Source:** `esp-hal/src/timer/timg.rs:574`

```
pub enum MwdtStage
```

Purpose. Selects one of the four watchdog stages.

How it works. Each stage has its own timeout and action; stage 0 is used by `enable()`.

Parameters and return. Variants: `Stage0`..`Stage3`.

Example.

```
MwdtStage::Stage0
```

6. Interrupt handlers, entry point and macros

esp_hal::main (attribute macro)

Stability: Stable attribute macro | **Source:** `esp-hal-procmacros/src/rtos_main.rs`

Purpose. Marks the blocking firmware entry point and generates the `no_mangle` entry glue.

How it works. Expands to a function the runtime calls after startup; the function must have signature `fn() -> !`.

Parameters and return. No arguments.

Example.

```
#[main]
fn main() -> ! { loop {} }
```

Limits. Not used with Embassy: use `#[esp_rtos::main]` there.

esp_hal::handler (attribute macro)

Stability: UNSTABLE attribute macro (marked unstable in the 1.1.1 docs) | **Source:** `esp-hal-procmacros/src/lib.rs`

Purpose. Turns a plain function into an `InterruptHandler` that can be registered.

How it works. Wraps the function with the interrupt-entry prologue/epilogue and returns a static `InterruptHandler` value under the same name; optional `priority = ...`.

Parameters and return. Optional `priority = Priority::...`.

Example.

```
#[handler]
#[ram]
fn on_timer() { }
```

Limits. Keep handlers short, never block, acknowledge the source.

esp_hal::ram (attribute macro)

Stability: Stable attribute macro | **Source:** esp-hal-procmacros/src/ram.rs

Purpose. Places a function or static in internal RAM (IRAM/DRAM) instead of flash.

How it works. Adds a link-section attribute; options like `reclaimed`, `zeroed`, `persistent`, `rtc_fast`.

Parameters and return. Optional placement keywords.

Example.

```
#[ram]
fn fast() {}
```

Limits. ISRs that must run while flash cache is disabled need `#[ram]`.

esp_hal::interrupt::InterruptConfigurable (trait)

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** esp-hal/src/interrupt/mod.rs:96

```
pub trait InterruptConfigurable: crate::private::Sealed
```

Purpose. Trait implemented by drivers that own an interrupt (timers, PCNT, UART, …); its method `set_interrupt_handler(&mut self, handler: InterruptHandler)` registers the handler.

How it works. Replaces any previously registered handler for that peripheral.

Parameters and return. Method: `set_interrupt_handler(&mut self, handler: InterruptHandler)`.

Example.

```
use esp_hal::interrupt::InterruptConfigurable;
```

Limits. UNSTABLE trait. Bring it into scope to call the trait method; many drivers also expose an inherent method of the same name (Lab 07 needs no import).

esp_hal::dma_buffers! (macro_rules)

Stability: Macro is not marked unstable, but it feeds the UNSTABLE `dma` driver | **Source:** esp-hal/src/dma/mod.rs

Purpose. Allocates static DMA buffers and descriptors.

How it works. Creates `static mut` byte arrays plus the right number of DMA descriptors (one per 4092 bytes) and returns `(&'static mut [u8] rx, &'static mut [DmaDescriptor] rx_desc, tx, tx_desc)`.

Parameters and return. Size in bytes (one size for both directions, or `(rx_size, tx_size)`).

Example.

```
let (rx, rx_d, tx, tx_d) = dma_buffers!(256);
```

Limits. Each call site may be executed only once (static storage).

7. UART

esp_hal::uart::Uart::new

Stability: Stable | **Source:** esp-hal/src/uart/mod.rs:1389

```
pub fn new(uart: impl Instance + 'd, config: Config) -> Result<Self, ConfigError>
```

Purpose. Creates a blocking UART driver.

How it works. Enables the UART clock, applies baud/format and prepares FIFOs; pins are attached afterwards.

Parameters and return. `uart`: `UART0`/`UART1`/`UART2` token; `config: uart::Config`. Returns `Result<Uart<Blocking>, ConfigError>`.

Example.

```
Uart::new(peripherals.UART1, Config::default().with_baudrate(115_200))?
```

Limits. UART0 normally carries the boot console on GPIO43/44.

esp_hal::uart::Config::with_baudrate

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:308`

```
fn with_baudrate(self, baudrate: u32) -> Self   // generated by #[derive(BuilderLite)]
```

Purpose. Sets the baud rate.

How it works. Divider computed from the UART clock source.

Parameters and return. `baudrate: u32` in bit/s.

Example.

```
Config::default().with_baudrate(115_200)
```

Limits. Tolerance setter `with_baudrate_tolerance` is UNSTABLE.

esp_hal::uart::Uart::with_tx

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1757`

```
pub fn with_tx(mut self, tx: impl PeripheralOutput<'d>) -> Self
```

Purpose. Attaches the TX pin.

How it works. Routes the UART TX signal to the pin via the GPIO matrix.

Parameters and return. `tx`: output-capable GPIO token. Returns `Self`.

Example.

```
.with_tx(peripherals.GPIO47)
```

esp_hal::uart::Uart::with_rx

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1737`

```
pub fn with_rx(mut self, rx: impl PeripheralInput<'d>) -> Self
```

Purpose. Attaches the RX pin.

How it works. Routes the pin to the UART RX signal.

Parameters and return. `rx`: input-capable GPIO token. Returns `Self`.

Example.

```
.with_rx(peripherals.GPIO42)
```

esp_hal::uart::Uart::write

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1860`

```
pub fn write(&mut self, data: &[u8]) -> Result<usize, TxError>
```

Purpose. Writes bytes to the TX FIFO.

How it works. Pushes into the 128-byte hardware FIFO, blocking while it is full.

Parameters and return. `data: &[u8]`. Returns `Result<usize, TxError>` (bytes accepted).

Example.

```
uart.write(b"hello")?
```

esp_hal::uart::Uart::flush

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1879`

```
pub fn flush(&mut self) -> Result<(), TxError>
```

Purpose. Waits until the FIFO and shift register are empty.

How it works. Polls the TX-idle status.

Parameters and return. `&mut self`. Returns `Result<(), TxError>`.

Example.

```
uart.flush()?
```

esp_hal::uart::Uart::read_ready

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1914`

```
pub fn read_ready(&self) -> bool
```

Purpose. True if at least one byte is waiting in the RX FIFO.

How it works. Reads the RX FIFO count.

Parameters and return. `&self`. Returns `bool`.

Example.

```
if uart.read_ready() { }
```

esp_hal::uart::Uart::read

Stability: Stable | **Source:** `esp-hal/src/uart/mod.rs:1955`

```
pub fn read(&mut self, buf: &mut [u8]) -> Result<usize, RxError>
```

Purpose. Reads bytes from the RX FIFO.

How it works. Copies available bytes; blocks until at least one byte is available.

Parameters and return. `buf: &mut [u8]`. Returns `Result<usize, RxError>`.

Example.

```
let n = uart.read(&mut buf)?;
```

Limits. Check `read_ready()` first to avoid blocking; FIFO overflow reports `RxError`.

esp_hal::uart::Uart::split

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/uart/mod.rs:2012`

```
pub fn split(self) -> (UartRx<'d, Dm>, UartTx<'d, Dm>)
```

Purpose. Splits into independent RX and TX halves.

How it works. Moves the two halves out so different tasks/ISRs can own them.

Parameters and return. `self`. Returns `(UartRx, UartTx)`.

Example.

```
let (rx, tx) = uart.split();
```

8. I2C master

esp_hal::i2c::master::I2c::new

Stability: Stable | **Source:** `esp-hal/src/i2c/master/mod.rs:714`

```
pub fn new(i2c: impl Instance + 'd, config: Config) -> Result<Self, ConfigError>
```

Purpose. Creates a blocking I2C master.

How it works. Enables I2C0/I2C1, applies timing derived from the requested frequency.

Parameters and return. `i2c`: `I2C0`/`I2C1` token; `config: Config`. Returns `Result<I2c<Blocking>, ConfigError>`.

Example.

```
I2c::new(peripherals.I2C0, Config::default())?
```

Limits. ESP32-S3 has two I2C controllers.

esp_hal::i2c::master::Config::with_frequency

Stability: Stable | **Source:** `esp-hal/src/i2c/master/mod.rs:582`

```
fn with_frequency(self, frequency: Rate) -> Self   // generated by #[derive(BuilderLite)]
```

Purpose. Sets the SCL frequency.

How it works. Computes SCL high/low periods from the clock source.

Parameters and return. `frequency: Rate`.

Example.

```
Config::default().with_frequency(Rate::from_khz(100))
```

Limits. Standard mode 100 kHz, fast mode 400 kHz.

esp_hal::i2c::master::I2c::with_sda

Stability: Stable | **Source:** `esp-hal/src/i2c/master/mod.rs:1193`

```
pub fn with_sda(mut self, sda: impl PeripheralInput<'d> + PeripheralOutput<'d>) -> Self
```

Purpose. Attaches the SDA pin (open-drain, internal pull-up enabled).

How it works. Routes SDA through the GPIO matrix in open-drain mode.

Parameters and return. `sda`: input+output GPIO token.

Example.

```
.with_sda(peripherals.GPIO41)
```

Limits. Internal pull-ups are weak: use external 2.2-4.7 kOhm.

esp_hal::i2c::master::I2c::with_scl

Stability: Stable | **Source:** esp-hal/src/i2c/master/mod.rs:1245

```
pub fn with_scl(mut self, scl: impl PeripheralInput<'d> + PeripheralOutput<'d>) -> Self
```

Purpose. Attaches the SCL pin.

How it works. Same as SDA.

Parameters and return. `scl`: GPIO token.

Example.

```
.with_scl(peripherals.GPIO40)
```

esp_hal::i2c::master::I2c::write

Stability: Stable | **Source:** esp-hal/src/i2c/master/mod.rs:1270

```
pub fn write<A: Into<I2cAddress>>(&mut self, address: A, buffer: &[u8]) -> Result<(),
Error>
```

Purpose. START, address+W, data, STOP.

How it works. Programs the I2C command list and waits for completion.

Parameters and return. `address: impl Into<I2cAddress>` (7-bit `u8`), `buffer: &[u8]`. Returns `Result<(), Error>`.

Example.

```
i2c.write(0x27, &[0x08])?
```

Limits. An empty buffer is a valid address probe (used by Lab 10); `Err` = no ACK.

esp_hal::i2c::master::I2c::read

Stability: Stable | **Source:** esp-hal/src/i2c/master/mod.rs:1296

```
pub fn read<A: Into<I2cAddress>>(&mut self, address: A, buffer: &mut [u8]) -> Result<(),
Error>
```

Purpose. START, address+R, read bytes, NACK last, STOP.

How it works. Queues read commands for `buffer.len()` bytes.

Parameters and return. `address`, `buffer: &mut [u8]`. Returns `Result<(), Error>`.

Example.

```
i2c.read(0x38, &mut buf)?
```

esp_hal::i2c::master::I2c::write_read

Stability: Stable | **Source:** esp-hal/src/i2c/master/mod.rs:1327

```
pub fn write_read<A: Into<I2cAddress>>(&mut self, address: A, write_buffer: &[u8],
read_buffer: &mut [u8]) -> Result<(), Error>
```

Purpose. Write then repeated-START read in one transaction.

How it works. Typical register read: write register address, then read.

Parameters and return. `address`, `write: &[u8]`, `read: &mut [u8]`. Returns `Result<(), Error>`.

Example.

```
i2c.write_read(0x68, &[0x75], &mut id)?
```

esp_hal::i2c::master::I2c::into_async

Stability: Stable | **Source:** esp-hal/src/i2c/master/mod.rs:744

```
pub fn into_async(mut self) -> I2c<'d, Async>
```

Purpose. Converts the driver to interrupt-driven async mode.

How it works. Enables the I2C interrupt and implements `embedded_hal_async::i2c::I2c`.

Parameters and return. `self`. Returns `I2c<Async>`.

Example.

```
let i2c = i2c.into_async();
```

9. SPI master

esp_hal::spi::Mode

Stability: Stable | **Source:** esp-hal/src/spi/mod.rs:77

```
pub enum Mode
```

Purpose. SPI clock polarity/phase.

How it works. Mode 0: CPOL=0/CPHA=0 (clock idles low, sample on rising edge) … mode 3: CPOL=1/CPHA=1.

Parameters and return. Variants `_0`, `_1`, `_2`, `_3`.

Example.

```
Mode::_0
```

esp_hal::spi::master::Spi::new

Stability: Stable | **Source:** esp-hal/src/spi/master/mod.rs:787

```
pub fn new(spi: impl Instance + 'd, config: Config) -> Result<Self, ConfigError>
```

Purpose. Creates a blocking SPI master.

How it works. Enables SPI2/SPI3 and programs clock divider and mode.

Parameters and return. `spi`: `SPI2`/`SPI3` token; `config: Config`. Returns `Result<Spi<Blocking>, ConfigError>`.

Example.

```
Spi::new(peripherals.SPI2, Config::default().with_frequency(Rate::from_mhz(1)))?
```

Limits. SPI0/SPI1 are reserved for flash/PSRAM.

esp_hal::spi::master::Config::with_frequency

Stability: Stable | **Source:** esp-hal/src/spi/master/mod.rs:461

```
fn with_frequency(self, frequency: Rate) -> Self   // generated by #[derive(BuilderLite)]
```

Purpose. Sets SCK frequency.

How it works. Divider from the 80 MHz APB clock.

Parameters and return. `frequency: Rate`.

Example.

```
Config::default().with_frequency(Rate::from_mhz(4))
```

esp_hal::spi::master::Config::with_mode

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:469`

```
fn with_mode(self, mode: Mode) -> Self   // generated by #[derive(BuilderLite)]
```

Purpose. Sets SPI mode (CPOL/CPHA).

How it works. Stored in the mode bits of the user register.

Parameters and return. `mode: spi::Mode`.

Example.

```
Config::default().with_mode(Mode::_0)
```

esp_hal::spi::master::Spi::with_sck

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1024`

```
pub fn with_sck(mut self, sclk: impl PeripheralOutput<'d>) -> Self
```

Purpose. Attaches the clock pin.

How it works. Routes the SPI clock output via the GPIO matrix.

Parameters and return. `sclk`: output GPIO token.

Example.

```
.with_sck(peripherals.GPIO39)
```

esp_hal::spi::master::Spi::with_mosi

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1053`

```
pub fn with_mosi(mut self, mosi: impl PeripheralOutput<'d>) -> Self
```

Purpose. Attaches MOSI (master out).

How it works. Routes the data-out signal.

Parameters and return. `mosi`: output GPIO token.

Example.

```
.with_mosi(peripherals.GPIO38)
```

esp_hal::spi::master::Spi::with_miso

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1079`

```
pub fn with_miso(self, miso: impl PeripheralInput<'d>) -> Self
```

Purpose. Attaches MISO (master in).

How it works. Routes the pin to the data-in signal.

Parameters and return. `miso`: input GPIO token.

Example.

```
.with_miso(peripherals.GPIO42)
```

esp_hal::spi::master::Spi::with_cs

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/spi/master/mod.rs:1154`

```
pub fn with_cs(mut self, cs: impl PeripheralOutput<'d>) -> Self
```

Purpose. Attaches a hardware-driven chip-select.

How it works. The peripheral asserts CS around each transfer.

Parameters and return. `cs`: output GPIO token.

Example.

```
.with_cs(peripherals.GPIO21)
```

Limits. UNSTABLE (`#[instability::unstable]`): needs the `unstable` feature. Only one hardware CS; the stable alternative is a software CS: use `embedded-hal-bus::ExclusiveDevice` with a GPIO for software CS (Lab 13).

esp_hal::spi::master::Spi::transfer

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1282`

```
pub fn transfer(&mut self, words: &mut [u8]) -> Result<(), Error>
```

Purpose. Full-duplex in-place transfer.

How it works. Shifts `words` out while shifting the received bytes back into the same buffer.

Parameters and return. `words: &mut [u8]`. Returns `Result<(), Error>`.

Example.

```
spi.transfer(&mut buf)?
```

esp_hal::spi::master::Spi::write

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1215`

```
pub fn write(&mut self, words: &[u8]) -> Result<(), Error>
```

Purpose. Write-only transfer (received data discarded).

How it works. Same engine, RX ignored.

Parameters and return. `words: &[u8]`. Returns `Result<(), Error>`.

Example.

```
spi.write(&[0x9F])?
```

esp_hal::spi::master::Spi::read

Stability: Stable | **Source:** `esp-hal/src/spi/master/mod.rs:1253`

```
pub fn read(&mut self, words: &mut [u8]) -> Result<(), Error>
```

Purpose. Read-only transfer (sends dummy bytes).

How it works. Clocks out dummy data to produce SCK.

Parameters and return. `words: &mut [u8]`. Returns `Result<(), Error>`.

Example.

```
spi.read(&mut buf)?
```

esp_hal::spi::master::Spi::with_dma

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/spi/master/dma.rs:69`

```
pub fn with_dma(self, channel: impl DmaChannelFor<AnySpi<'d>>) -> SpiDma<'d, Blocking>
```

Purpose. Attaches a DMA channel and returns `SpiDma`.

How it works. Binds the SPI to the DMA channel's descriptor engine.

Parameters and return. `channel`: a DMA channel token such as `peripherals.DMA_CH0`. Returns `SpiDma<Blocking>`.

Example.

```
.with_dma(peripherals.DMA_CH0)
```

Limits. UNSTABLE method. ESP32-S3 has 5 GDMA channels shared by all GDMA users.

10. DMA

esp_hal::dma::DmaRxBuf::new

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/buffers.rs:699`

```
pub fn new(descriptors: &'static mut [DmaDescriptor], buffer: &'static mut [u8]) -> Result<Self, DmaBufError>
```

Purpose. Wraps a byte buffer + descriptor array as a DMA receive buffer.

How it works. Links descriptors over the buffer; validates alignment and size.

Parameters and return. `descriptors: &'static mut [DmaDescriptor]`,`buffer: &'static mut [u8]`. Returns `Result<DmaRxBuf, DmaBufError>`.

Example.

```
DmaRxBuf::new(rx_desc, rx_buf)?
```

esp_hal::dma::DmaTxBuf::new

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/buffers.rs:513`

```
pub fn new(descriptors: &'static mut [DmaDescriptor], buffer: &'static mut [u8]) -> Result<Self, DmaBufError>
```

Purpose. Wraps a byte buffer + descriptors as a DMA transmit buffer.

How it works. Same linking as RX.

Parameters and return. `descriptors`, `buffer`. Returns `Result<DmaTxBuf, DmaBufError>`.

Example.

```
DmaTxBuf::new(tx_desc, tx_buf)?
```

esp_hal::dma::DmaTxBuf::as_mut_slice

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/buffers.rs:629`

```
pub fn as_mut_slice(&mut self) -> &mut [u8]
```

Purpose. Mutable view of the data bytes.

How it works. Direct access to the buffer memory.

Parameters and return. `&mut self`. Returns `&mut [u8]`.

Example.

```
buf.as_mut_slice()[0] = 1;
```

esp_hal::dma::DmaRxBuf::as_slice

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/buffers.rs:795`

```
pub fn as_slice(&self) -> &[u8]
```

Purpose. Read view of the received bytes.

How it works. Direct access.

Parameters and return. `&self`. Returns `&[u8]`.

Example.

```
rx.as_slice()
```

esp_hal::dma::Mem2Mem::new

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/m2m.rs:64` | *(the source has 2 variants; the one for ESP32-S3 / the primary impl is shown)*

```
pub fn new(channel: impl DmaChannelConvert<Mem2MemChannel<'d>>, peripheral: impl
DmaEligible) -> Self
```

Purpose. Creates a memory-to-memory DMA engine.

How it works. Uses a GDMA channel and (on ESP32-S3) the SPI2 DMA peripheral as the data path.

Parameters and return. `channel`: `DMA_CH0` token; `peripheral`: `SPI2` token. Returns `Mem2Mem`.

Example.

```
Mem2Mem::new(peripherals.DMA_CH0, peripherals.SPI2)
```

Limits. Consumes SPI2 on ESP32-S3.

esp_hal::dma::Mem2Mem::with_descriptors

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/m2m.rs:113`

```
pub fn with_descriptors(self, rx_descriptors: &'static mut [DmaDescriptor],
tx_descriptors: &'static mut [DmaDescriptor], config: BurstConfig) ->
Result<SimpleMem2Mem<'d, Blocking>, DmaError>
```

Purpose. Supplies the descriptor lists.

How it works. Stores rx/tx descriptors and burst config.

Parameters and return. `rx_descriptors`, `tx_descriptors` (`&'static mut [DmaDescriptor]`), `config: BurstConfig`. Returns `Result<SimpleMem2Mem<Blocking>, DmaError>`.

Example.

```
.with_descriptors(rx_d, tx_d, BurstConfig::default())?
```

esp_hal::dma::Mem2Mem::start_transfer

Stability: UNSTABLE (module `dma` requires feature `unstable`) | **Source:** `esp-hal/src/dma/m2m.rs:424`

```
pub fn start_transfer(&mut self, rx_buffer: &mut [u8], tx_buffer: &[u8]) ->
Result<SimpleMem2MemTransfer<'_, 'd, Dm>, DmaError>
```

Purpose. Starts the copy.

How it works. Links the buffers and starts both DMA directions.

Parameters and return. `rx_buffer: &mut [u8]`, `tx_buffer: &[u8]`. Returns `Result<SimpleMem2MemTransfer, DmaError>`; call `.wait()` on it.

Example.

```
mem2mem.start_transfer(rx, tx)?.wait()?
```

11. RMT

esp_hal::rmt::Rmt::new

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:912`

```
pub fn new(peripheral: RMT<'rmt>, frequency: Rate) -> Result<Self, ConfigError>
```

Purpose. Creates the RMT driver with a tick source frequency.

How it works. Validates that the requested rate is reachable from the clock source and programs the divider.

Parameters and return. `peripheral: RMT` token; `frequency: Rate`. Returns `Result<Rmt, ConfigError>`.

Example.

```
Rmt::new(peripherals.RMT, Rate::from_mhz(80))?
```

Limits. UNSTABLE module. ESP32-S3: 4 TX + 4 RX channels.

esp_hal::rmt::PulseCode

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:323`

```
pub struct PulseCode
```

Purpose. One RMT symbol: two (level, duration) pairs.

How it works. Packed into a 32-bit RAM word.

Parameters and return. `PulseCode::new(level1, length1, level2, length2)`; `PulseCode::end_marker()` terminates a sequence.

Example.

```
PulseCode::new(Level::High, 32, Level::Low, 68)
```

esp_hal::rmt::PulseCode::new

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:347`

```
pub const fn new(level1: Level, length1: u16, level2: Level, length2: u16) -> Self
```

Purpose. Builds a symbol.

How it works. Packs levels and 15-bit lengths.

Parameters and return. `level1: Level, length1: u16, level2: Level, length2: u16`. Returns `PulseCode`.

Example.

```
PulseCode::new(Level::High, 32, Level::Low, 68)
```

Limits. Lengths are in RMT ticks; `new_clamped` clamps out-of-range lengths.

esp_hal::rmt::PulseCode::end_marker

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:436`

```
pub const fn end_marker() -> Self
```

Purpose. The terminating symbol.

How it works. A zero-length pair that ends transmission.

Parameters and return. No parameters.

Example.

```
PulseCode::end_marker()
```

esp_hal::rmt::TxChannelConfig::with_clk_divider

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:659`

```
fn with_clk_divider(self, clk_divider: u8) -> Self   // generated by
#[derive(BuilderLite)]
```

Purpose. Divides the RMT tick clock for this channel.

How it works. Tick = source / divider.

Parameters and return. `clk_divider: u8`.

Example.

```
TxChannelConfig::default().with_clk_divider(1)
```

esp_hal::rmt::Channel::transmit

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:1737`

```
pub fn transmit<'data>(self, mut data: &'data [PulseCode]) -> Result<TxTransaction<'ch,
'data>, (Error, Self)>
```

Purpose. Starts transmitting a symbol sequence.

How it works. Copies symbols to RMT RAM (refilling if longer) and starts the channel; consumes the channel into a transaction.

Parameters and return. `self`, `data: &[PulseCode]`. Returns `Result<TxTransaction, (Error, Self)>`.

Example.

```
let tx = channel.transmit(&data).unwrap();
```

Limits. Sequence must end with `end_marker()`.

esp_hal::rmt::TxTransaction::wait

Stability: UNSTABLE (module `rmt` requires feature `unstable`) | **Source:** `esp-hal/src/rmt.rs:1514`

```
pub fn wait(mut self) -> Result<Channel<'ch, Blocking, Tx>, (Error, Channel<'ch, Blocking, Tx>)>
```

Purpose. Blocks until the transmission finishes and returns the channel.

How it works. Polls the end/error event.

Parameters and return. `self`. Returns `Result<Channel, (Error, Channel)>`.

Example.

```
channel = tx.wait().unwrap();
```

12. PCNT (pulse counter)

esp_hal::pcnt::Pcnt::new

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/mod.rs:137`

```
pub fn new(_instance: PCNT<'d>) -> Self
```

Purpose. Creates the pulse-counter peripheral and its units.

How it works. Enables PCNT and exposes `unit0..unit3` (4 units, 2 channels each on ESP32-S3).

Parameters and return. `pcnt: PCNT` token. Returns `Pcnt`.

Example.

```
let mut pcnt = Pcnt::new(peripherals.PCNT);
```

esp_hal::pcnt::Pcnt::set_interrupt_handler

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/pcnt/mod.rs:183`

```
pub fn set_interrupt_handler(&mut self, handler: InterruptHandler)
```

Purpose. Registers the PCNT interrupt handler.

How it works. Binds the shared PCNT interrupt.

Parameters and return. `handler: InterruptHandler`.

Example.

```
pcnt.set_interrupt_handler(on_pcnt);
```

esp_hal::pcnt::unit::Unit::set_high_limit

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:140`

```
pub fn set_high_limit(&self, value: Option<i16>) -> Result<(), InvalidHighLimit>
```

Purpose. Sets the upper limit event threshold.

How it works. Programs the threshold register; `None` disables.

Parameters and return. `value: Option<i16>`. Returns `Result<(), InvalidHighLimit>`.

Example.

```
u0.set_high_limit(Some(100))?
```

esp_hal::pcnt::unit::Unit::set_low_limit

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:109`

```
pub fn set_low_limit(&self, value: Option<i16>) -> Result<(), InvalidLowLimit>
```

Purpose. Sets the lower limit threshold.

How it works. Same as high limit.

Parameters and return. `value: Option<i16>`. Returns `Result<(), InvalidLowLimit>`.

Example.

```
u0.set_low_limit(Some(-100))?
```

esp_hal::pcnt::unit::Unit::set_filter

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:198`

```
pub fn set_filter(&self, threshold: Option<u16>) -> Result<(), InvalidFilterThreshold>
```

Purpose. Sets the glitch filter.

How it works. Pulses shorter than the threshold (in APB cycles) are ignored.

Parameters and return. `threshold: Option<u16>` (max 1023). Returns `Result<(), InvalidFilterThreshold>`.

Example.

```
u0.set_filter(Some(1023))?
```

esp_hal::pcnt::unit::Unit::clear

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:220`

```
pub fn clear(&self)
```

Purpose. Resets the counter to zero.

How it works. Pulses the counter-clear bit.

Parameters and return. `&self`.

Example.

```
u0.clear();
```

esp_hal::pcnt::unit::Unit::resume

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:244`

```
pub fn resume(&self)
```

Purpose. Starts/continues counting.

How it works. Clears the pause bit.

Parameters and return. `&self`.

Example.

```
u0.resume();
```

esp_hal::pcnt::unit::Unit::listen

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:271`

```
pub fn listen(&self)
```

Purpose. Enables the unit's limit/threshold events as interrupts.

How it works. Sets the interrupt-enable bits.

Parameters and return. `&self`.

Example.

```
u0.listen();
```

esp_hal::pcnt::unit::Unit::interrupt_is_set

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:289`

```
pub fn interrupt_is_set(&self) -> bool
```

Purpose. Checks whether this unit raised an interrupt.

How it works. Reads the unit's interrupt-raw bit.

Parameters and return. `&self`. Returns `bool`.

Example.

```
u0.interrupt_is_set()
```

esp_hal::pcnt::unit::Unit::reset_interrupt

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:298`

```
pub fn reset_interrupt(&self)
```

Purpose. Clears the unit's interrupt.

How it works. Writes the clear bit.

Parameters and return. `&self`.

Example.

```
u0.reset_interrupt();
```

esp_hal::pcnt::unit::Unit::events

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/unit.rs:253`

```
pub fn events(&self) -> Events
```

Purpose. Reports which events (high/low limit, thresholds, zero) occurred.

How it works. Reads the event status flags.

Parameters and return. `&self`. Returns `Events`.

Example.

```
let ev = u0.events(); ev.high_limit
```

esp_hal::pcnt::channel::Channel::set_ctrl_signal

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/channel.rs:70`

```
pub fn set_ctrl_signal<'d>(&self, source: impl PeripheralInput<'d>) -> &Self
```

Purpose. Chooses the control (direction) input.

How it works. Routes the signal into the channel's ctrl input.

Parameters and return. `source: impl PeripheralInput` (e.g. `pin.peripheral_input()`). Returns `&Self` (chainable).

Example.

```
ch0.set_ctrl_signal(b.clone());
```

esp_hal::pcnt::channel::Channel::set_edge_signal

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/channel.rs:130`

```
pub fn set_edge_signal<'d>(&self, source: impl PeripheralInput<'d>) -> &Self
```

Purpose. Chooses the pulse (edge) input.

How it works. Routes the signal into the channel's pulse input.

Parameters and return. `source: impl PeripheralInput`.

Example.

```
ch0.set_edge_signal(a.clone());
```

esp_hal::pcnt::channel::Channel::set_input_mode

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/channel.rs:59`

```
pub fn set_input_mode(&self, neg_edge: EdgeMode, pos_edge: EdgeMode)
```

Purpose. Defines what a rising/falling edge does to the count.

How it works. Programs the positive/negative edge action.

Parameters and return. `neg_edge: EdgeMode`, `pos_edge: EdgeMode`.

Example.

```
ch0.set_input_mode(EdgeMode::Increment, EdgeMode::Decrement);
```

esp_hal::pcnt::channel::Channel::set_ctrl_mode

Stability: UNSTABLE (module `pcnt` requires feature `unstable`) | **Source:** `esp-hal/src/pcnt/channel.rs:44`

```
pub fn set_ctrl_mode(&self, low: CtrlMode, high: CtrlMode)
```

Purpose. Defines how the control level modifies counting.

How it works. Programs the low/high control-level behaviour.

Parameters and return. `low: CtrlMode`, `high: CtrlMode`.

Example.

```
ch0.set_ctrl_mode(CtrlMode::Reverse, CtrlMode::Keep);
```

esp_hal::gpio::Input::peripheral_input

Stability: UNSTABLE item (marked `#[instability::unstable]`, needs feature `unstable`) | **Source:** `esp-hal/src/gpio/mod.rs:1123`

```
pub fn peripheral_input(&self) -> interconnect::InputSignal<'d>
```

Purpose. Returns a signal handle that peripherals (PCNT, …) can consume.

How it works. Exposes the pad's input signal through the GPIO matrix without consuming the `Input`.

Parameters and return. `&self`. Returns `interconnect::InputSignal`.

Example.

```
let a = pin_a.peripheral_input();
```

Limits. UNSTABLE.

13. MCPWM

esp_hal::mcpwm::McPwm::new

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/mod.rs:148`

```
pub fn new(peripheral: PWM, peripheral_clock: PeripheralClockConfig) -> Self
```

Purpose. Creates the motor-control PWM block (3 timers, 3 operators).

How it works. Enables the peripheral and sets the shared prescaled clock.

Parameters and return. `peripheral: MCPWM0`/`MCPWM1`; `peripheral_clock: PeripheralClockConfig`. Returns `McPwm`.

Example.

```
McPwm::new(peripherals.MCPWM0, clock_cfg)
```

Limits. MCPWM source clock on ESP32-S3 is 160 MHz.

esp_hal::mcpwm::PeripheralClockConfig::with_frequency

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/mod.rs:219`

```
pub fn with_frequency(target_freq: Rate) -> Result<Self, FrequencyError>
```

Purpose. Derives the peripheral clock from the source clock.

How it works. Finds an 8-bit prescaler for the requested frequency.

Parameters and return. `target_freq: Rate`. Returns `Result<Self, FrequencyError>`.

Example.

```
PeripheralClockConfig::with_frequency(Rate::from_mhz(32))?
```

esp_hal::mcpwm::PeripheralClockConfig::timer_clock_with_frequency

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/mod.rs:269`

```
pub fn timer_clock_with_frequency(&self, period: u16, mode: timer::PwmWorkingMode,
target_freq: Rate) -> Result<timer::TimerClockConfig, FrequencyError>
```

Purpose. Computes timer prescaler/period for a period count and PWM frequency.

How it works. Chooses prescaler so that `period + 1` counts take 1/frequency.

Parameters and return. `period: u16`, `mode: PwmWorkingMode`, `target_freq: Rate`. Returns `Result<TimerClockConfig, FrequencyError>`.

Example.

```
clock_cfg.timer_clock_with_frequency(99, PwmWorkingMode::Increase, Rate::from_khz(20))?
```

esp_hal::mcpwm::timer::Timer::start

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/timer.rs:54`

```
pub fn start(&mut self, timer_config: TimerClockConfig)
```

Purpose. Starts the PWM timer.

How it works. Writes prescaler/period and sets run mode.

Parameters and return. `timer_config: TimerClockConfig`.

Example.

```
mcpwm.timer0.start(timer_cfg);
```

esp_hal::mcpwm::operator::Operator::set_timer

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/operator.rs:195`

```
pub fn set_timer<const TIM: u8>(&mut self, timer: &Timer<TIM, PWM>)
```

Purpose. Connects an operator to a timer.

How it works. Writes the operator-timer select field.

Parameters and return. `timer: &Timer<TIM, PWM>`.

Example.

```
mcpwm.operator0.set_timer(&mcpwm.timer0);
```

esp_hal::mcpwm::operator::Operator::with_pins

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/operator.rs:229`

```
pub fn with_pins(self, pin_a: impl PeripheralOutput<'d>, config_a: PwmPinConfig<true>,
pin_b: impl PeripheralOutput<'d>, config_b: PwmPinConfig<false>) -> (PwmPin<'d, PWM, OP,
true>, PwmPin<'d, PWM, OP, false>)
```

Purpose. Attaches output pins to both PWM outputs (A and B) of an operator.

How it works. Creates two `PwmPin`s with their own configuration.

Parameters and return. `pin_a`, `config_a`, `pin_b`, `config_b`. Returns `(PwmPin<A>, PwmPin<B>)`.

Example.

```
.with_pins(p1, PwmPinConfig::UP_ACTIVE_HIGH, p2, PwmPinConfig::UP_ACTIVE_HIGH)
```

esp_hal::mcpwm::operator::Operator::with_linked_pins

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/operator.rs:243`

```
pub fn with_linked_pins(self, pin_a: impl PeripheralOutput<'d>, config_a:
PwmPinConfig<true>, pin_b: impl PeripheralOutput<'d>, config_b: PwmPinConfig<false>,
config_dt: DeadTimeCfg) -> LinkedPins<'d, PWM, OP>
```

Purpose. Attaches A/B outputs with a dead-time module (complementary drive).

How it works. Configures rising/falling-edge delay between the two outputs.

Parameters and return. `pin_a, config_a, pin_b, config_b, config_dt: DeadTimeCfg`. Returns `LinkedPins`.

Example.

```
// see operator.rs
```

Limits. The 1.1.1 module docs list dead time as 'not yet implemented'; verify on a scope before relying on it.

esp_hal::mcpwm::operator::PwmPin::set_timestamp

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/operator.rs:349`

```
pub fn set_timestamp(&mut self, value: u16)
```

Purpose. Sets the compare value (duty) of an output.

How it works. Writes the timestamp register (shadowed).

Parameters and return. `value: u16` in timer counts (0..=period).

Example.

```
pwm_a.set_timestamp(50);
```

esp_hal::mcpwm::operator::PwmPinConfig::UP_ACTIVE_HIGH

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/operator.rs:265`

```
pub const UP_ACTIVE_HIGH: Self
```

Purpose. Pin configuration: output high from period start until the timestamp.

How it works. Sets generator actions on timer-equals-zero (set) and timer-equals-compare (clear).

Parameters and return. Associated constant of `PwmPinConfig<IS_A>`.

Example.

```
PwmPinConfig::UP_ACTIVE_HIGH
```

esp_hal::mcpwm::timer::PwmWorkingMode

Stability: UNSTABLE (module `mcpwm` requires feature `unstable`) | **Source:** `esp-hal/src/mcpwm/timer.rs:221`

```
pub enum PwmWorkingMode
```

Purpose. Counting direction of the PWM timer.

How it works. Increase = sawtooth up; Decrease = down; UpDown = triangle.

Parameters and return. Variants: `Increase`, `Decrease`, `UpDown`.

Example.

```
PwmWorkingMode::Increase
```

14. TWAI (CAN 2.0)

esp_hal::twai::TwaiConfiguration::new

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:940`

```
pub fn new(peripheral: impl Instance + 'd, rx_pin: impl PeripheralInput<'d>, tx_pin: impl PeripheralOutput<'d>, baud_rate: BaudRate, mode: TwaiMode) -> Self
```

Purpose. Creates a TWAI configuration for use WITH an external transceiver.

How it works. Routes TX/RX through the GPIO matrix and sets bit timing for the baud rate; controller stays in reset-like config mode.

Parameters and return. `peripheral: TWAI0`; `rx_pin`; `tx_pin`; `baud_rate: BaudRate`; `mode: TwaiMode`. Returns `TwaiConfiguration<Blocking>`.

Example.

```
TwaiConfiguration::new(peripherals.TWAI0, rx, tx, BaudRate::B125K, TwaiMode::Normal)
```

Limits. ESP32-S3 has one TWAI controller; a transceiver (e.g. SN65HVD230) is needed on a real bus.

esp_hal::twai::TwaiConfiguration::new_no_transceiver

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:955`

```
pub fn new_no_transceiver(peripheral: impl Instance + 'd, rx_pin: impl PeripheralInput<'d>, tx_pin: impl PeripheralOutput<'d>, baud_rate: BaudRate, mode: TwaiMode) -> Self
```

Purpose. Variant for direct ESP-to-ESP wiring or self-test without a transceiver.

How it works. Makes TX open-drain with pull-up.

Parameters and return. same parameters as `new`.

Example.

```
TwaiConfiguration::new_no_transceiver(...)
```

esp_hal::twai::TwaiConfiguration::set_filter

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:844`

```
pub fn set_filter(&mut self, filter: impl Filter)
```

Purpose. Programs the acceptance filter.

How it works. Stores code/mask registers applied at `start()`.

Parameters and return. `filter: impl Filter` (e.g. `SingleStandardFilter`).

Example.

```
cfg.set_filter(const { SingleStandardFilter::new(..) });
```

esp_hal::twai::TwaiConfiguration::start

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:889`

```
pub fn start(self) -> Twai<'d, Dm>
```

Purpose. Leaves configuration mode and returns the operational driver.

How it works. Applies mode/filter and enables the controller.

Parameters and return. `self`. Returns `Twai`.

Example.

```
let mut can = cfg.start();
```

Limits. Bit rate and filter are locked after start.

esp_hal::twai::Twai::transmit

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:1106`

```
pub fn transmit(&mut self, frame: &EspTwaiFrame) -> nb::Result<(), EspTwaiError>
```

Purpose. Queues a frame for transmission.

How it works. Writes the TX buffer and requests transmission; `WouldBlock` while busy.

Parameters and return. `frame: &EspTwaiFrame`. Returns `nb::Result<(), EspTwaiError>`.

Example.

```
nb::block!(can.transmit(&frame))?
```

esp_hal::twai::Twai::receive

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:1111`

```
pub fn receive(&mut self) -> nb::Result<EspTwaiFrame, EspTwaiError>
```

Purpose. Reads one received frame.

How it works. Reads the RX FIFO; `WouldBlock` if empty.

Parameters and return. `&mut self`. Returns `nb::Result<EspTwaiFrame, EspTwaiError>`.

Example.

```
let f = nb::block!(can.receive())?;
```

esp_hal::twai::EspTwaiFrame::new

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:422` | *(the source has 2 variants; the one for ESP32-S3 / the primary impl is shown)*

```
pub fn new(id: impl Into<Id>, data: &[u8]) -> Option<Self>
```

Purpose. Creates a data frame.

How it works. Copies up to 8 bytes.

Parameters and return. `id: impl Into<Id>`, `data: &[u8]` (<= 8). Returns `Option<EspTwaiFrame>`.

Example.

```
EspTwaiFrame::new(StandardId::ZERO, &[1,2,3])
```

esp_hal::twai::EspTwaiFrame::new_self_reception

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:459`

```
pub fn new_self_reception(id: impl Into<Id>, data: &[u8]) -> Option<Self>
```

Purpose. Creates a frame flagged for self-reception (self-test mode).

How it works. Sets the self-reception request bit.

Parameters and return. `id`, `data`. Returns `Option<EspTwaiFrame>`.

Example.

```
EspTwaiFrame::new_self_reception(id, &[n])
```

esp_hal::twai::StandardId::new

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:264`

```
pub fn new(raw: u16) -> Option<Self>
```

Purpose. Creates an 11-bit identifier.

How it works. Range-checked (<= 0x7FF).

Parameters and return. `raw: u16`. Returns `Option<StandardId>`.

Example.

```
StandardId::new(0x123)
```

esp_hal::twai::Twai::transmit_error_count

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:1070`

```
pub fn transmit_error_count(&self) -> u8
```

Purpose. Current TX error counter.

How it works. Reads the TEC register.

Parameters and return. `&self`. Returns `u8`.

Example.

```
can.transmit_error_count()
```

esp_hal::twai::TwaiMode

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:237`

```
pub enum TwaiMode
```

Purpose. Operating mode.

How it works. Normal = ACKs and participates; SelfTest = no ACK needed; ListenOnly = silent.

Parameters and return. Variants: `Normal`, `SelfTest`, `ListenOnly`.

Example.

```
TwaiMode::SelfTest
```

esp_hal::twai::BaudRate

Stability: UNSTABLE (module `twai` requires feature `unstable`) | **Source:** `esp-hal/src/twai/mod.rs:568`

```
pub enum BaudRate
```

Purpose. Preset bit rates.

How it works. Each variant carries precomputed timing parameters.

Parameters and return. e.g. `B125K`, `B250K`, `B500K`, `B1000K` (see source for all).

Example.

`BaudRate::B125K`

15. USB Serial/JTAG and USB OTG

esp_hal::usb_serial_jtag::UsbSerialJtag::new

Stability: UNSTABLE (module `usb_serial_jtag` requires feature `unstable`) | **Source:** `esp-hal/src/usb_serial_jtag.rs:316`

```
pub fn new(usb_device: USB_DEVICE<'d>) -> Self
```

Purpose. Creates the built-in USB CDC-ACM serial driver.

How it works. Uses the fixed USB PHY on GPIO19/20; no driver setup needed on the host.

Parameters and return. `usb_device: USB_DEVICE` token. Returns `UsbSerialJtag<Blocking>`.

Example.

```
UsbSerialJtag::new(peripherals.USB_DEVICE)
```

Limits. UNSTABLE. Same peripheral esp-println `auto`/`jtag-serial` writes to.

esp_hal::usb_serial_jtag::UsbSerialJtag::write

Stability: UNSTABLE (module `usb_serial_jtag` requires feature `unstable`) | **Source:** `esp-hal/src/usb_serial_jtag.rs:393`

```
pub fn write(&mut self, data: &[u8]) -> Result<(), Error>
```

Purpose. Writes bytes to the host.

How it works. Fills the 64-byte endpoint FIFO and flushes.

Parameters and return. `data: &[u8]`. Returns `Result<(), Error>`.

Example.

```
usb.write(b"hi")?
```

esp_hal::usb_serial_jtag::UsbSerialJtag::read_byte

Stability: UNSTABLE (module `usb_serial_jtag` requires feature `unstable`) | **Source:** `esp-hal/src/usb_serial_jtag.rs:414`

```
pub fn read_byte(&mut self) -> nb::Result<u8, Error>
```

Purpose. Reads one byte if available.

How it works. Reads the RX FIFO.

Parameters and return. `&mut self`. Returns `nb::Result<u8, Error>`.

Example.

```
nb::block!(usb.read_byte())?
```

esp_hal::otg_fs::Usb::new

Stability: UNSTABLE (module `otg_fs` requires feature `unstable`) | **Source:** `esp-hal/src/otg_fs.rs:73`

```
pub fn new(usb0: peripherals::USB0<'d>, _usb_dp: impl UsbDp + 'd, _usb_dm: impl UsbDm + 'd) -> Self
```

Purpose. Creates the USB OTG full-speed core handle.

How it works. Takes the USB0 peripheral and the D+/D- pins.

Parameters and return. `usb0: USB0`, `dp: GPIO20`, `dm: GPIO19`. Returns `Usb`.

Example.

```
Usb::new(peripherals.USB0, peripherals.GPIO20, peripherals.GPIO19)
```

Limits. UNSTABLE. Pass to `otg_fs::asynch::Driver::new` for `embassy-usb`.

esp_hal::otg_fs::asynch::Driver::new

Stability: UNSTABLE (module `otg_fs` requires feature `unstable`) | **Source:** `esp-hal/src/otg_fs.rs:184`

```
pub fn new(peri: Usb<'d>, ep_out_buffer: &'d mut [u8], config: Config) -> Self
```

Purpose. Creates the embassy-usb driver.

How it works. Allocates endpoint buffers and enables the USB interrupt.

Parameters and return. `peripheral: Usb`, `ep_out_buffer: &mut [u8]`, `config: Config`. Returns `Driver`.

Example.

```
Driver::new(usb, &mut ep_out_buffer, Config::default())
```

16. PSRAM, heap allocation and logging (companion crates)

esp_alloc::heap_allocator! (macro_rules, esp-alloc 0.10.0)

Stability: Companion crate API | **Source:** `esp-alloc/src/macros.rs`

Purpose. Creates a global heap in internal RAM.

How it works. Reserves a static buffer (`size: N`) and registers it as a heap region; `#[ram(reclaimed)]` places it in reclaimed IRAM.

Parameters and return. `size: expr` (and optional attributes).

Example.

```
esp_alloc::heap_allocator!(size: 72 * 1024);
```

Limits. Required by esp-radio (Wi-Fi/BLE).

esp_alloc::psram_allocator! (macro_rules, esp-alloc 0.10.0)

Stability: Companion crate API | **Source:** `esp-alloc/src/macros.rs`

Purpose. Adds external PSRAM as a heap region.

How it works. Creates `Psram::new(peripheral, config)` and registers `raw_parts()` as an External-capability region.

Parameters and return. `peripheral: PSRAM` token, `psram_module` path (`esp_hal::psram`), optional config.

Example.

```
esp_alloc::psram_allocator!(peripherals.PSRAM, esp_hal::psram);
```

Limits. Atomics do not work in PSRAM: never allocate `Atomic*`/`Arc` there. Release builds only.

esp_println::logger::init_logger_from_env

Stability: Companion crate API | **Source:** `esp-println/src/logger.rs:20`

```
pub fn init_logger_from_env()
```

Purpose. Installs the `log` backend that prints via esp-println.

How it works. Reads the `ESP_LOG` environment variable at build time for the level.

Parameters and return. No parameters.

Example.

```
esp_println::logger::init_logger_from_env();
```

Limits. Needs esp-println feature `log-04`.

esp_bootloader_esp_idf::esp_app_desc! (macro)

Stability: Companion crate API | **Source:** `esp-bootloader-esp-idf/src/lib.rs`

Purpose. Embeds the ESP-IDF application descriptor required by the 2nd-stage bootloader.

How it works. Defines a static `esp_app_desc_t` in a dedicated section.

Parameters and return. No arguments.

Example.

```
esp_bootloader_esp_idf::esp_app_desc!();
```

Limits. Missing it makes the bootloader reject the image.

17. esp-rtos, Embassy glue and wireless (companion crates)

esp_rtos::start

Stability: Companion crate API | **Source:** `esp-rtos/src/lib.rs:254`

```
pub fn start(timer: impl TimerSource, int0: SoftwareInterrupt<'static, 0>)
```

Purpose. Starts the esp-rtos scheduler and the Embassy time driver.

How it works. Converts the current context into the main task and configures the tick timer and a software interrupt used for context switching.

Parameters and return. `timer: impl TimerSource` (e.g. `timg0.timer0`), `int0: SoftwareInterrupt<'static, 0>`.

Example.

```
esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);
```

Limits. Call once after `esp_hal::init`, before any Embassy/Wi-Fi/BLE use.

esp_rtos::main (attribute macro)

Stability: Companion crate API | **Source:** `esp-hal-procmacros/src/rtos_main.rs`

Purpose. Async entry point: `#[esp_rtos::main] async fn main(spawner: Spawner)`.

How it works. Creates the Embassy executor and runs `main` as its first task.

Parameters and return. The function receives an `embassy_executor::Spawner`.

Example.

```
#[esp_rtos::main]
async fn main(spawner: Spawner) -> ! { loop {} }
```

Limits. Requires esp-rtos feature `embassy`.

esp_radio::wifi::new

Stability: Companion crate API | **Source:** `esp-radio/src/wifi/mod.rs:2205`

```
pub fn new<'d>(device: crate::hal::peripherals::WIFI<'d>, config: ControllerConfig) ->
Result<(WifiController<'d>, Interfaces<'d>), WifiError>
```

Purpose. Initialises the Wi-Fi driver and returns its controller and network interfaces.

How it works. Starts the Wi-Fi firmware blob, allocates from the heap and creates `Interfaces` (station/access-point).

Parameters and return. `device: WIFI` token, `config: ControllerConfig`. Returns `Result<(WifiController, Interfaces), WifiError>`.

Example.

```
esp_radio::wifi::new(peripherals.WIFI, ControllerConfig::default())?
```

Limits. Requires a heap (`esp-alloc`) and `esp_rtos::start` first. ADC2 unusable while Wi-Fi is on.

esp_radio::wifi::WifiController::connect_async

Stability: Companion crate API | **Source:** `esp-radio/src/wifi/mod.rs:2860`

```
pub async fn connect_async(&mut self) -> Result<ConnectedStationInfo, WifiError>
```

Purpose. Connects the station to the configured access point.

How it works. Starts association and 4-way handshake; resolves on success/failure.

Parameters and return. `&mut self`. Returns a future resolving to `Result<ConnectedStationInfo, WifiError>`.

Example.

```
controller.connect_async().await
```

Limits. 2.4 GHz only.

esp_radio::wifi::WifiController::wait_for_disconnect_async

Stability: Companion crate API | **Source:** `esp-radio/src/wifi/mod.rs:2967`

```
pub async fn wait_for_disconnect_async(&self) -> Result<DisconnectedStationInfo,
WifiError>
```

Purpose. Waits until the link drops.

How it works. Awaits the disconnect event.

Parameters and return. `&self`. Returns a future resolving to `Result<DisconnectedStationInfo, WifiError>`.

Example.

```
controller.wait_for_disconnect_async().await
```

esp_radio::wifi::WifiController::scan_async

Stability: Companion crate API | **Source:** `esp-radio/src/wifi/mod.rs:2801`

```
pub async fn scan_async(&mut self, config: &ScanConfig) -> Result<Vec<AccessPointInfo>,
WifiError>
```

Purpose. Scans for access points.

How it works. Runs an active scan and returns the results.

Parameters and return. `config: &ScanConfig`. Returns a future of `Result<Vec<AccessPointInfo>, WifiError>`.

Example.

```
controller.scan_async(&ScanConfig::default().with_max(10)).await?
```

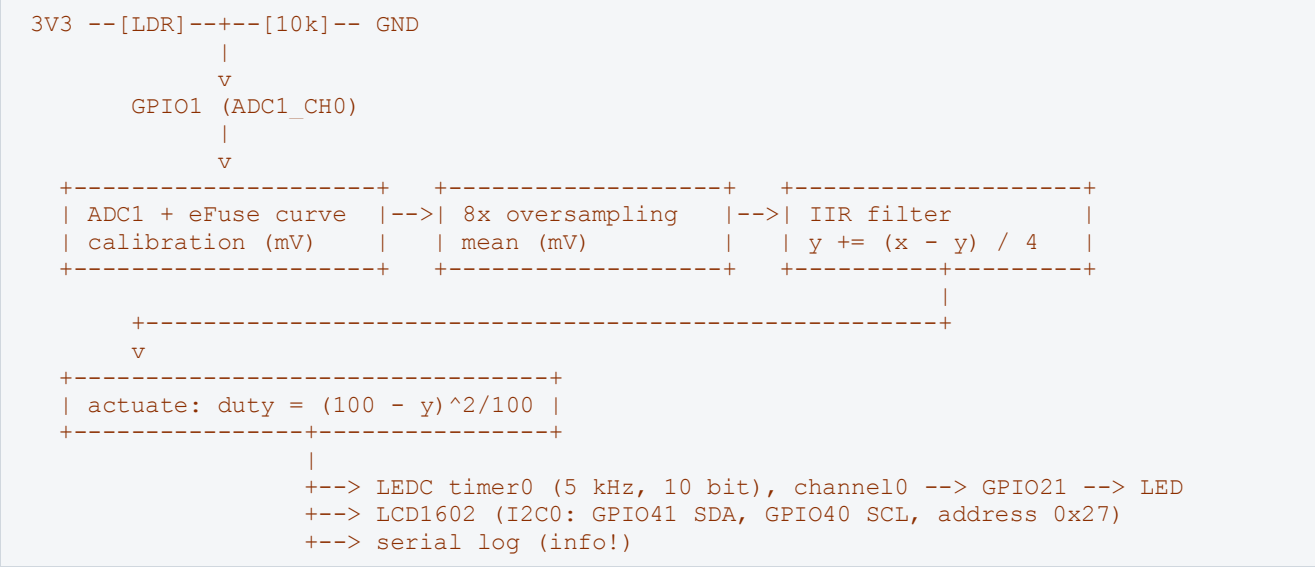
Part IV - Integrated projects

The three projects combine several laboratories. They are ordered by difficulty. All three compile (see Appendix D) and none has been run on hardware. Each project calls `reserve_camera_pins!` so that the OV2640 pins stay protected, and each uses every GPIO at most once; the pin audit in section 7.4 checks this from the source code.

Project 1 - Basic monitoring system

Goal. Read an analog sensor (an LDR light sensor), control an LED with PWM, and display measurements on the LCD1602. Builds on Labs 04, 05 and 11. File: `src/bin/proj1_basic_monitor.rs`. Status: **BUILD-VERIFIED, not hardware-tested.**

Architecture



Components

- ESP32-S3 camera board (OV2640 mounted but unused)
- LCD1602 with PCF8574 I2C backpack (0x27) behind a 3.3 V-safe bus (level shifter, chapter 7 and Lab 11)
- LDR (photoresistor, e.g. GL5528)
- 10 kohm resistor (divider)
- LED + 330 ohm resistor
- Breadboard and jumper wires

Pin allocation

GPIO	Function	Connected to
1	ADC1_CH0 (analog in)	LDR divider output
21	LEDC ch0 PWM out	LED anode via 330 ohm
40	I2C0 SCL	LCD1602 backpack SCL
41	I2C0 SDA	LCD1602 backpack SDA
4-13, 15-18	OV2640 camera	Reserved by `reserve_camera_pins!`, untouched

Cargo dependencies

Crate	Locked version	Role
`esp-backtrace`	0.19.0	Panic handler
`esp-bootloader-esp-idf`	0.5.0	Application descriptor (`esp_app_desc!`)
`esp-hal`	1.1.1	Hardware drivers (GPIO, ADC, LEDC, …)
`esp-println`	0.17.0	Console output and `log` backend
`esp32s3-labs`	path	This package (library with LCD driver and camera guard)
`log`	0.4.35	Logging macros
`nb`	1.1.0	`nb::block!` for non-blocking calls

Versions are those in `Cargo.lock`; `esp-hal` is pinned with `=1.1.1`. A crate that is a dependency of the package but only used indirectly (for example `esp-radio` through `esp_rtos`) may be absent from this list.

All dependencies come from the shared `Cargo.toml` (Appendix A); the feature flags are `esp-hal = { version = "=1.1.1", features = ["esp32s3", "unstable", "log-04"] }` (ADC and LEDC are unstable).

Complete source code

```
//! Project 1 - Basic monitoring system: analog sensor -> PWM LED -> LCD1602
//!
//! A light-dependent resistor (LDR) is read with the ADC. A "night light" LED gets
BRIGHTER
//! as the room gets DARKER (PWM duty = 100 % - light %), and the LCD shows the
measurements.
//!
//! Pin allocation (no pin is used twice; the camera pins are reserved and untouched):
//!   GPIO1  (ADC1_CH0)  LDR divider output      3V3 --[LDR]--+--[10 k]-- GND, node ->
GPIO1
//!   GPIO21 (LEDC ch0)  PWM LED                 GPIO21 --[330 ohm]--> LED --> GND
//!   GPIO41 (I2C0 SDA)  LCD1602 backpack SDA    GPIO40 (I2C0 SCL)  LCD1602 backpack SCL
//!
//! Software architecture: one cooperative super-loop
//!   sample (8x oversampling, factory-calibrated mV)  ->  filter (exponential average)
//!   ->  actuate (LEDC duty with square-law gamma)    ->  display (LCD) + log (serial)
//! API level: ADC/LEDC/Delay UNSTABLE; I2C STABLE. LCD driver: src/lcd1602.rs (not
hardware-tested).
#![no_std]
#![no_main]

use core::fmt::Write;

use esp_backtrace as _;
use esp_hal::{
    analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
    delay::Delay,
    gpio::DriveMode,
    i2c::master::{Config as I2cConfig, I2c},
    ledc::{
        LSGlobalClkSource, Ledc, LowSpeed,
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
    },
    main,
    peripherals::ADC1,
    time::Rate,
};
use esp32s3_labs::{lcd1602::Lcd1602, reserve_camera_pins};
use log::{info, warn};

esp_bootloader_esp_idf::esp_app_desc!();
```

```
const LCD_ADDR: u8 = 0x27;
const OVERSAMPLE: u32 = 8;
/// With 11 dB attenuation the calibrated ADC reads up to about 3.1 V.
const FULL_SCALE_MV: u32 = 3100;

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let _camera = reserve_camera_pins!(peripherals);
    let delay = Delay::new();

    // --- ADC1 on GPIO1 ----------------------------------------------------------
----
    let mut adc_cfg = AdcConfig::new();
    let mut ldr = adc_cfg
        .enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(peripherals.GPIO1,
Attenuation::_11dB);
    let mut adc = Adc::new(peripherals.ADC1, adc_cfg);

    // --- LEDC PWM on GPIO21 (5 kHz, 10-bit) -------------------------------------
----
    let mut ledc = Ledc::new(peripherals.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);
    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            duty: timer::config::Duty::Duty10Bit,
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_khz(5),
        })
        .unwrap();
    let mut led = ledc.channel(channel::Number::Channel0, peripherals.GPIO21);
    led.configure(channel::config::Config {
        timer: &timer0,
        duty_pct: 0,
        drive_mode: DriveMode::PushPull,
    })
    .unwrap();

    // --- LCD1602 on I2C0 (GPIO41 = SDA, GPIO40 = SCL) ---------------------------
----
    let i2c = I2c::new(peripherals.I2C0,
I2cConfig::default().with_frequency(Rate::from_khz(100)))
        .unwrap()
        .with_sda(peripherals.GPIO41)
        .with_scl(peripherals.GPIO40);
    let mut lcd = Lcd1602::new(i2c, delay, LCD_ADDR);
    let lcd_ok = lcd.init().is_ok();
    if !lcd_ok {
        warn!("LCD not found at 0x{:02X}: continuing with serial output only", LCD_ADDR);
    }

    let mut filtered: i32 = -1; // percent, -1 = no sample yet
    loop {
        // 1) sample ----------------------------------------------------------------
----
        let mut sum = 0u32;
        for   in 0..OVERSAMPLE {
            sum += nb::block!(adc.read_oneshot(&mut ldr)).unwrap() as u32;
        }
        let mv = sum / OVERSAMPLE;
        let light_pct = (mv * 100 / FULL_SCALE_MV).min(100) as i32;

        // 2) filter: first-order IIR, y += (x - y) / 4 ---------------------------
----
        filtered = if filtered < 0 { light_pct } else { filtered + (light_pct - filtered)
/ 4 };

        // 3) actuate: darker room -> brighter LED; square-law gamma for a perceptually
        //    smoother response (see Lab 05).
        let want = (100 - filtered).clamp(0, 100) as u32;
```

```
        let duty = (want * want / 100) as u8;
        led.set_duty(duty).unwrap();

        // 4) display + log ------------------------------------------------------------
------
        if lcd_ok {
            let _ = lcd.set_cursor(0, 0);
            let _ = write!(lcd, "Light{:3}%{:5}mV", filtered, mv);
            let _ = lcd.set_cursor(0, 1);
            let _ = write!(lcd, "LED duty {:3}%   ", duty);
        }
        info!("ldr = {} mV, light = {} %, led duty = {} %", mv, filtered, duty);

        delay.delay_millis(200);
    }
}
```

Software architecture

The program is a single cooperative super-loop with four stages per 200 ms cycle:

1. **Sample.** Eight calibrated ADC readings (mV) are averaged. Averaging N independent samples lowers random noise by about the square root of N.
2. **Normalise.** `light_pct = mv * 100 / 3100`, limited to 100. 3100 mV is the approximate full scale at 11 dB attenuation.
3. **Filter.** A first-order recursive filter `y += (x - y) / 4` smooths flicker. Because the division is an integer division that truncates toward zero, the filter stops moving when the difference is below 4, so the steady-state error is at most 3 percentage points; this is acceptable for a night light and avoids floating point.
4. **Actuate and display.** Brightness is inverted (dark room, bright LED) and mapped through a square-law curve to approximate gamma correction. The LCD is updated with `write!`; if the LCD does not answer on I2C at start-up the program continues with serial output only.

Ownership: `adc`, `ldr` (the `AdcPin`), `led` (the LEDC channel) and `lcd` are owned by `main`; the I2C bus was moved into the `Lcd1602` driver, so no other code can touch it.

Build, flash and test

```
cargo run --release --bin proj1_basic_monitor
```

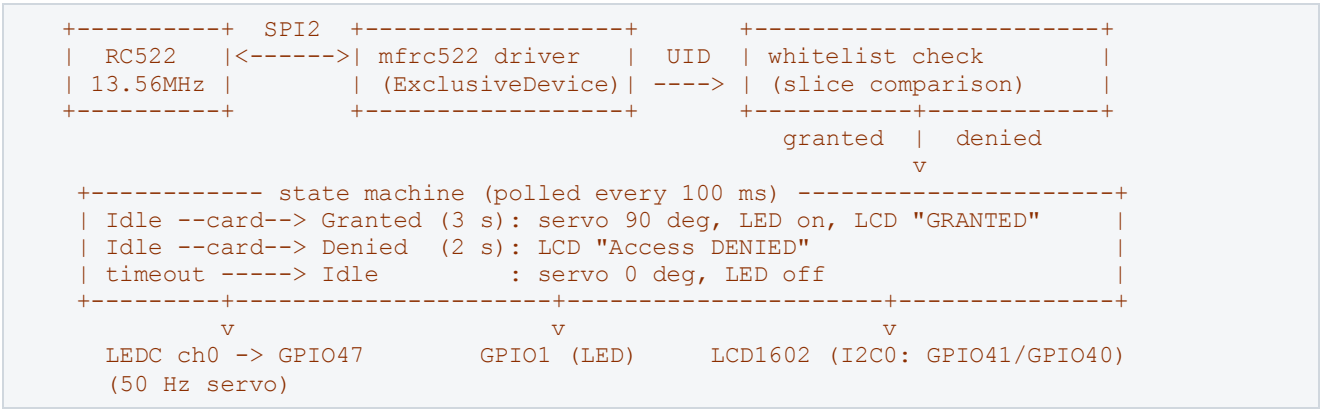
Step	Action	Expected
1	Power up	LCD shows `Light  nn%  mmmmV` and `LED duty nnn%`
2	Cover the LDR	`Light` falls toward 0 %, LED duty rises, LED brightens smoothly
3	Shine a lamp on it	`Light` rises, LED dims
4	Unplug the LCD (power off) and reset	Serial log warns `LCD not found` and continues
5	Watch the serial log	One `ldr = .. mV, light = .. %, led duty = .. %` line every 200 ms

If the LED reacts the wrong way, swap the LDR and the 10 kohm resistor (the divider direction) or change the mapping line `want = 100 - filtered`.

Project 2 - RFID access-control system

Goal. Read RFID cards with the RC522, check the UID against a whitelist, show the result on the LCD1602, and move a servo that acts as a door lock. Builds on Labs 06, 11 and 13. File: `src/bin/proj2_rfid_access.rs`. Status: **BUILD-VERIFIED, not hardware-tested.**

Architecture



Components

- ESP32-S3 camera board
- RC522 RFID module (3.3 V) and MIFARE Classic cards or tags
- SG90 / MG90S servo with an external 5 V supply (500 mA or more), common ground; optional 470 uF capacitor
- LCD1602 with PCF8574 I2C backpack (0x27), 3.3 V-safe bus
- LED + 330 ohm resistor
- Breadboard and jumper wires

Pin allocation

GPIO	Function	Connected to
1	GPIO out	Door-open LED via 330 ohm
14	GPIO out	RC522 RST
21	GPIO out (software CS)	RC522 SDA / NSS
38	SPI2 MOSI	RC522 MOSI
39	SPI2 SCK	RC522 SCK
40	I2C0 SCL	LCD1602 SCL
41	I2C0 SDA	LCD1602 SDA
42	SPI2 MISO	RC522 MISO
47	LEDC ch0, 50 Hz	Servo signal
4-13, 15-18	OV2640 camera	Reserved by `reserve_camera_pins!`, untouched

Cargo dependencies

Crate	Locked version	Role
`embedded-hal`	1.0.0	Hardware traits (I2C, SPI, PWM, delay)
`embedded-hal-bus`	0.3.0	SpiDevice from bus + CS
`esp-backtrace`	0.19.0	Panic handler

Crate	Locked version	Role
`esp-bootloader-esp-idf`	0.5.0	Application descriptor (`esp_app_desc!`)
`esp-hal`	1.1.1	Hardware drivers (GPIO, ADC, LEDC, …)
`esp-println`	0.17.0	Console output and `log` backend
`esp32s3-labs`	path	This package (library with LCD driver and camera guard)
`log`	0.4.35	Logging macros
`mfrc522`	0.8.0	RC522 RFID driver

Versions are those in `Cargo.lock`; `esp-hal` is pinned with `=1.1.1`. A crate that is a dependency of the package but only used indirectly (for example `esp-radio` through `esp_rtos`) may be absent from this list.

The RC522 driver is the `mfrc522` crate 0.8.0; `embedded-hal-bus` converts the esp-hal SPI bus plus a chip-select GPIO into the `SpiDevice` that the crate requires.

Complete source code

```
//! Project 2 - RFID access-control system: RC522 + LCD1602 + servo door lock
//!
//! A card is read by the RC522; if its UID is on the whitelist the servo "opens the
door"
//! for 3 s and a green LED lights, otherwise "ACCESS DENIED" is shown.
//!
//! Pin allocation (all distinct; camera pins reserved and untouched; LCD bus =
GPIO40/41):
//!   RC522  SDA(CS)=GPIO21  SCK=GPIO39  MOSI=GPIO38  MISO=GPIO42  RST=GPIO14   3.3 V
supply!
//!   LCD    SDA=GPIO41  SCL=GPIO40  (I2C0, address 0x27)
//!   Servo  signal=GPIO47 (LEDC 50 Hz)  +5 V from an external supply, common GND
//!   LED    GPIO1 --[330 ohm]--> LED --> GND  (door-open indicator)
//!
//! Software architecture: a small finite-state machine polled every 100 ms
//!   Idle --(card seen)--> Granted | Denied --(3 s / 2 s elapsed)--> Idle
//! The reader is only polled in `Idle`, which also debounces repeated reads of the same
card.
//! Authorisation is a whitelist of UIDs below. Run once, read the UID from the serial
log of a
//! denied card, paste it into WHITELIST, rebuild.
//! SECURITY NOTE: MIFARE UIDs can be cloned; this is a teaching project, not a secure
lock.
//! API level: SPI/I2C STABLE; LEDC/Delay UNSTABLE; mfrc522 0.8.0 (not hardware-tested).
#![no_std]
#![no_main]


use embedded_hal::pwm::SetDutyCycle;
use embedded_hal_bus::spi::ExclusiveDevice;
use esp backtrace as _;
use esp_hal::{
    delay::Delay,
    gpio::{DriveMode, Level, Output, OutputConfig},
    i2c::master::{Config as I2cConfig, I2c},
    ledc::{
        LSGlobalClkSource, Ledc, LowSpeed,
        channel::{self, ChannelIFace},
        timer::{self, TimerIFace},
    },
    main,
    spi::{
        Mode,
        master::{Config as SpiConfig, Spi},
    },
    time::{Duration, Instant, Rate},
};
use esp32s3_labs::{lcd1602::Lcd1602, reserve_camera_pins};
```

```rust
use log::{info, warn};
use mfrc522::{Mfrc522, comm::blocking::spi::SpiInterface};

esp_bootloader_esp_idf::esp_app_desc!();

/// Authorised card UIDs (4, 7 or 10 bytes each). Replace with your own cards.
const WHITELIST: &[&[u8]] = &[
    &[0xDE, 0xAD, 0xBE, 0xEF],
    &[0x01, 0x23, 0x45, 0x67],
];

const LCD_ADDR: u8 = 0x27;
const OPEN_FOR: Duration = Duration::from_millis(3000);
const DENIED_FOR: Duration = Duration::from_millis(2000);

enum State {
    Idle,
    Granted(Instant),
    Denied(Instant),
}

/// 0..=180 degrees -> 0.5..2.5 ms pulse in a 20 ms period.
fn set_angle<S: SetDutyCycle>(servo: &mut S, deg: u32) {
    let pulse_us = 500 + 2000 * deg.min(180) / 180;
    let ticks = pulse_us * servo.max_duty_cycle() as u32 / 20_000;
    let _ = servo.set_duty_cycle(ticks as u16);
}

#[main]
fn main() -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals = esp_hal::init(esp_hal::Config::default());
    let _camera = reserve_camera_pins!(peripherals);
    let delay = Delay::new();

    // --- outputs: LED + servo ---------------------------------------------------
-------
    let mut led = Output::new(peripherals.GPIO1, Level::Low, OutputConfig::default());

    let mut ledc = Ledc::new(peripherals.LEDC);
    ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);
    let mut timer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
    timer0
        .configure(timer::config::Config {
            duty: timer::config::Duty::Duty14Bit,
            clock_source: timer::LSClockSource::APBClk,
            frequency: Rate::from_hz(50),
        })
        .unwrap();
    let mut servo = ledc.channel(channel::Number::Channel0, peripherals.GPIO47);
    servo
        .configure(channel::config::Config {
            timer: &timer0,
            duty_pct: 0,
            drive_mode: DriveMode::PushPull,
        })
        .unwrap();
    set_angle(&mut servo, 0); // locked

    // --- LCD --------------------------------------------------------------------
----------
    let i2c = I2c::new(peripherals.I2C0,
I2cConfig::default().with_frequency(Rate::from_khz(100)))
        .unwrap()
        .with_sda(peripherals.GPIO41)
        .with_scl(peripherals.GPIO40);
    let mut lcd = Lcd1602::new(i2c, delay, LCD_ADDR);
    let lcd_ok = lcd.init().is_ok();
    if !lcd_ok {
        warn!("LCD not found: serial output only");
    }
    let mut show = |top: &str, bottom: &str| {
```

```
        if lcd_ok {
            let _ = lcd.clear();
            let _ = lcd.print(top);
            let _ = lcd.set_cursor(0, 1);
            let _ = lcd.print(bottom);
        }
    };

    // --- RC522 ------------------------------------------------------------------
----------
    let mut rst = Output::new(peripherals.GPIO14, Level::Low, OutputConfig::default());
    delay.delay_millis(50);
    rst.set_high();
    delay.delay_millis(50);

    let spi_bus = Spi::new(
        peripherals.SPI2,
        SpiConfig::default().with_frequency(Rate::from_mhz(4)).with_mode(Mode::_0),
    )
    .unwrap()
    .with_sck(peripherals.GPIO39)
    .with_mosi(peripherals.GPIO38)
    .with_miso(peripherals.GPIO42);
    let cs = Output::new(peripherals.GPIO21, Level::High, OutputConfig::default());
    let spi_dev = ExclusiveDevice::new(spi_bus, cs, Delay::new()).unwrap();
    let nss_delay = Delay::new();
    let itf = SpiInterface::new(spi_dev).with_delay(move || nss_delay.delay_nanos(200));
    let mut rfid = Mfrc522::new(itf).init().expect("RC522 init failed - check SPI wiring
and 3.3 V");
    info!("RC522 version register: {:?}", rfid.version().ok());

    show("Access control", "Present card...");
    let mut state = State::Idle;

    loop {
        match state {
            State::Idle => {
                if let Ok(atqa) = rfid.new_card_present() {
                    if let Ok(uid) = rfid.select(&atqa) {
                        let bytes = uid.as_bytes();
                        let _ = rfid.hlta(); // card sleeps until it leaves and re-enters
the field
                        if WHITELIST.iter().any(|allowed| *allowed == bytes) {
                            info!("ACCESS GRANTED, UID {:02X?}", bytes);
                            show("Access GRANTED", "Welcome!");
                            set_angle(&mut servo, 90); // unlock
                            led.set_high();
                            state = State::Granted(Instant::now());
                        } else {
                            warn!("ACCESS DENIED, UID {:02X?}", bytes);
                            show("Access DENIED", "Unknown card");
                            state = State::Denied(Instant::now());
                        }
                    }
                }
            }
            State::Granted(since) => {
                if since.elapsed() >= OPEN_FOR {
                    set_angle(&mut servo, 0); // lock again
                    led.set_low();
                    show("Access control", "Present card...");
                    state = State::Idle;
                }
            }
            State::Denied(since) => {
                if since.elapsed() >= DENIED_FOR {
                    show("Access control", "Present card...");
                    state = State::Idle;
                }
            }
        }
```

```
            delay.delay_millis(100);
        }
    }
```

Software architecture

- **Peripherals**: SPI2 (RC522), I2C0 (LCD), LEDC timer 0 / channel 0 (servo at 50 Hz, 14-bit resolution), and two GPIO outputs (LED, RC522 reset). Each is created once in `main`; the camera pins are reserved by `_camera`.
- **State machine**: the `State` enum has three states. Only `Idle` polls the reader. After a read the card is put to sleep with `hlta()`, so the same card is not read repeatedly while it stays on the antenna.
- **Whitelist**: `WHITELIST: &[&[u8]]` holds UIDs of any length (4, 7 or 10 bytes); a card is accepted if its UID bytes equal one entry (`*allowed == bytes`).
- **Servo helper**: `set_angle` is generic over the embedded-hal `SetDutyCycle` trait, so it works with any PWM channel.
- **LCD**: the closure `show` clears the display and prints two lines; if the LCD is absent, `lcd_ok` is false and `show` does nothing.
- **Timing**: the loop period is 100 ms, so door timing is accurate to about that value; `Instant` measures the 3 s and 2 s holds without blocking.

Security note. A UID is an identifier, not a secret, and many cards can be cloned. MIFARE Classic's cryptography is known to be broken. This is a teaching project. A real lock needs mutual authentication (for example DESFire EV2/EV3 or a challenge-response with a secure element), a tamper-resistant mechanical design, rate limiting, logging and a fail-secure policy.

Build, flash and test

```
cargo run --release --bin proj2_rfid_access
```

Step	Action	Expected
1	Power up	LCD `Access control / Present card...`; servo at 0 degrees; serial prints the RC522 version register
2	Present an *unknown* card	Serial `ACCESS DENIED, UID [..]`; LCD `Access DENIED`; servo does not move; back to idle after 2 s
3	Copy the UID bytes from the log into `WHITELIST`, rebuild and flash	
4	Present that card	`ACCESS GRANTED`; servo to 90 degrees; LED on; after 3 s servo returns to 0 and the LED turns off
5	Hold the card on the reader	Only one event, not a stream
6	Disconnect the RC522 and reset	`RC522 init failed` panic message from `expect`

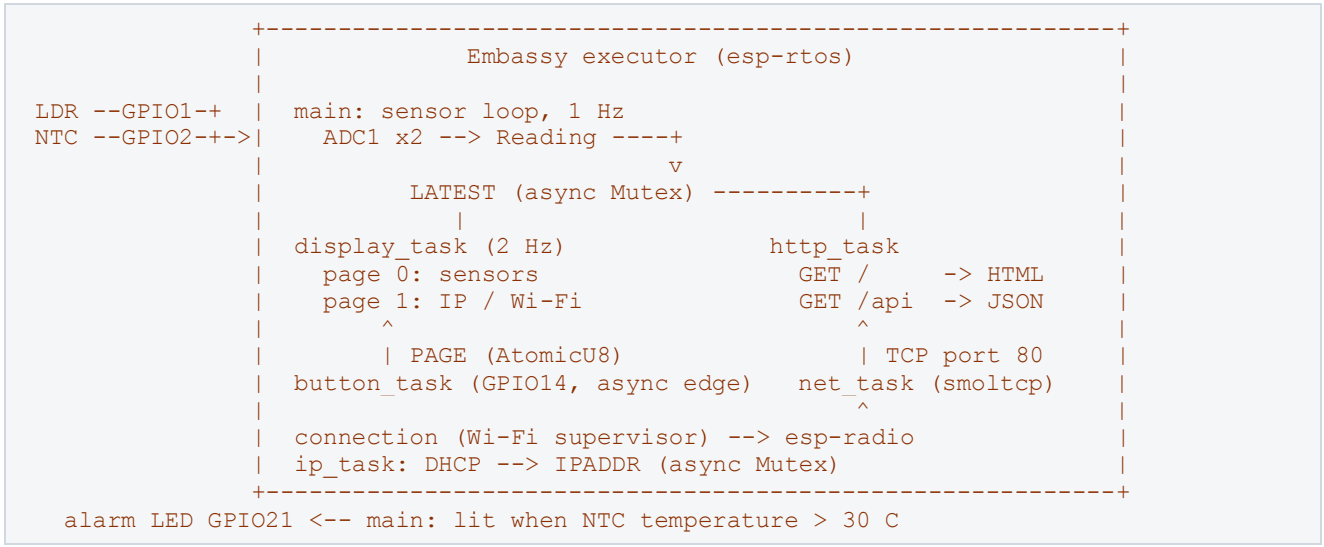
Power the servo from a separate 5 V supply with a common ground (Lab 06). Keep the RC522 on 3.3 V.

Project 3 - Advanced IoT monitoring system

Goal. Measure light and temperature, show them on the LCD, publish them over Wi-Fi as a web page and a JSON API, and do all this concurrently with Embassy tasks. Builds on Labs 04, 11, 20 and 23. File: `src/bin/proj3_iot_monitor.rs`. Status: **BUILD-VERIFIED, not hardware-tested, not run against a router.**

A note on sensors. The on-die temperature sensor would have been a convenient third sensor, but esp-hal 1.1.1 has no `tsens` driver for the ESP32-S3, so the project uses an external **NTC thermistor** and an **LDR**, both read by ADC1, and derives the session minimum and maximum. An I2C sensor such as an AHT20 could be added on the LCD bus (address 0x38, no clash with 0x27); it is left as an exercise because it needs bus sharing between tasks.

Architecture



Components

- ESP32-S3 camera board
- LCD1602 with PCF8574 I2C backpack (0x27), 3.3 V-safe bus
- LDR + 10 kohm resistor (light divider)
- NTC thermistor 10 kohm, B = 3950, + 10 kohm resistor (temperature divider)
- LED + 330 ohm resistor (alarm)
- Push button
- 2.4 GHz Wi-Fi network; a PC or phone with a browser / curl

Pin allocation

GPIO	Function	Connected to
1	ADC1_CH0 (analog in)	LDR divider output
2	ADC1_CH1 (analog in)	NTC divider output
14	GPIO in, pull-up	Page button to GND
21	GPIO out	Alarm LED via 330 ohm
40	I2C0 SCL	LCD1602 SCL
41	I2C0 SDA	LCD1602 SDA
4-13, 15-18	OV2640 camera	Reserved by `reserve_camera_pins!`, untouched

Cargo dependencies

Crate	Locked version	Role

Crate	Locked version	Role
`embassy-executor`	0.10.0	Async executor and `#[task]`
`embassy-net`	0.9.1	TCP/IP stack (smoltcp) for sockets and DHCP
`embassy-sync`	0.7.2	Mutex, Signal, channels
`embassy-time`	0.5.1	Timers, Ticker
`embedded-io-async`	0.7.0	Async `Write` for sockets
`esp-alloc`	0.10.0	Global heap (and PSRAM heap)
`esp-backtrace`	0.19.0	Panic handler
`esp-bootloader-esp-idf`	0.5.0	Application descriptor (`esp_app_desc!`)
`esp-hal`	1.1.1	Hardware drivers (GPIO, ADC, LEDC, …)
`esp-println`	0.17.0	Console output and `log` backend
`esp-radio`	0.18.0	Wi-Fi and BLE
`esp-rtos`	0.3.0	Scheduler, Embassy time driver, `#[esp_rtos::main]`
`esp32s3-labs`	path	This package (library with LCD driver and camera guard)
`heapless`	0.9.3	Fixed-capacity strings and vectors
`libm`	0.2.16	`logf` for the thermistor
`log`	0.4.35	Logging macros
`nb`	1.1.0	`nb::block!` for non-blocking calls
`static-cell`	?	

Versions are those in `Cargo.lock`; `esp-hal` is pinned with `=1.1.1`. A crate that is a dependency of the package but only used indirectly (for example `esp-radio` through `esp_rtos`) may be absent from this list.

Features that matter: `esp-rtos` with `embassy`, `esp-radio` and `esp-alloc`; `esp-radio` with `wifi`, `ble`, `coex`, `esp-alloc`, `unstable`; `embassy-net` with `tcp`, `udp`, `dhcpv4`, `medium-ethernet`; `libm` for the natural logarithm in the thermistor equation.

Complete source code

```
//! Project 3 - Advanced IoT monitoring system: sensors + LCD + Wi-Fi + Embassy tasks
//!
//! Build:  SSID="MyWifi" PASSWORD="secret" cargo run --release --bin proj3_iot_monitor
//! Then browse to  http://<ip printed on the LCD / serial>/   (auto-refreshing
dashboard)
//!                 http://<ip>/api                              (JSON for scripts / Home
Assistant)
//!
//! Sensors (all chosen so that they need no I2C address and no Wi-Fi-conflicting ADC2
pin):
//!   LDR light sensor  3V3 --[LDR]--+--[10 k]-- GND,  node -> GPIO1 (ADC1_CH0)
//!   NTC 10 k beta 3950   3V3 --[10 k]--+--[NTC]-- GND,  node -> GPIO2 (ADC1 CH1)
//!   (The on-die temperature sensor is NOT used: esp-hal 1.1.1 has no `tsens` driver for
ESP32-S3.)
//! Other pins:
//!   LCD1602 I2C SDA=GPIO41 SCL=GPIO40 (0x27)     alarm LED GPIO21 --[330 ohm]--> GND
//!   page button GPIO14 --> GND (internal pull-up)
//! Camera pins are reserved (moved into `_camera`) and never touched.
//!
//! Task architecture (one cooperative Embassy executor, see ASCII diagram in the
manual):
//!   main          sensor loop, 1 Hz: ADC x2 -> LATEST (shared state)
//!   display_task  LCD refresh, 2 Hz, two pages (sensors / network)
//!   button task   awaits button edge -> toggles PAGE
//!   connection    keeps Wi-Fi associated        net_task  runs the TCP/IP stack
//!   ip_task       waits for DHCP, stores the IP  http_task serves "/" (HTML) and "/api"
```

```
(JSON)
//! Shared data: `LATEST` (async Mutex), `IPADDR` (async Mutex), `PAGE` (AtomicU8).
//! Not hardware-tested; the NTC/LDR formulas are standard but need calibration for real
use.
#![no_std]
#![no_main]

use core::{
    fmt::Write as _,
    sync::atomic::{AtomicU8, Ordering},
};

use embassy_executor::Spawner;
use embassy_net::{Runner, Stack, StackResources, tcp::TcpSocket};
use embassy_sync::{blocking_mutex::raw::CriticalSectionRawMutex, mutex::Mutex};
use embassy_time::{Duration, Instant, Ticker, Timer};
use embedded_io_async::Write as _;
use esp_alloc as _;
use esp_backtrace as _;
use esp_hal::{
    Blocking,
    analog::adc::{Adc, AdcCalCurve, AdcConfig, Attenuation},
    clock::CpuClock,
    delay::Delay,
    gpio::{Input, InputConfig, Level, Output, OutputConfig, Pull},
    i2c::master::{Config as I2cConfig, I2c},
    interrupt::software::SoftwareInterruptControl,
    peripherals::ADC1,
    ram,
    rng::Rng,
    time::Rate,
    timer::timg::TimerGroup,
};
use esp_radio::wifi::{Config, ControllerConfig, Interface, WifiController,
sta::StationConfig};
use esp32s3_labs::{lcd1602::Lcd1602, reserve_camera_pins};
use log::{info, warn};

esp_bootloader_esp_idf::esp_app_desc!();

macro_rules! mk_static {
    ($t:ty, $val:expr) => {{
        static STATIC_CELL: static_cell::StaticCell<$t> = static_cell::StaticCell::new();
        #[deny(unused_attributes)]
        let x = STATIC_CELL.uninit().write($val);
        x
    }};
}

const SSID: &str = match option_env!("SSID") {
    Some(s) => s,
    None => "YOUR_SSID",
};
const PASSWORD: &str = match option_env!("PASSWORD") {
    Some(s) => s,
    None => "YOUR_PASSWORD",
};
const LCD_ADDR: u8 = 0x27;
const ALARM_C: f32 = 30.0; // alarm LED lights above this NTC temperature
const FULL_SCALE_MV: u32 = 3100;

// ----------------------------------------------------------------------------
----
// shared state
// ----------------------------------------------------------------------------
----
#[derive(Clone, Copy)]
struct Reading {
    ntc_c: f32,
    ntc_min: f32,
    ntc_max: f32,
```

```
    light_pct: u8,
    light_mv: u16,
    uptime_s: u32,
}

impl Reading {
    const ZERO: Reading = Reading { ntc_c: 0.0, ntc_min: 0.0, ntc_max: 0.0, light_pct: 0,
light_mv: 0, uptime_s: 0 };
}

static LATEST: Mutex<CriticalSectionRawMutex, Reading> = Mutex::new(Reading::ZERO);
static IPADDR: Mutex<CriticalSectionRawMutex, Option<[u8; 4]>> = Mutex::new(None);
static PAGE: AtomicU8 = AtomicU8::new(0);

/// Convert the NTC divider voltage (mV, 3V3 -- 10k -- node -- NTC -- GND) to degrees
Celsius
/// with the beta equation  1/T = 1/T0 + ln(R/R0)/B.
fn ntc_celsius(mv: u32) -> f32 {
    const VCC_MV: f32 = 3300.0;
    const R_FIXED: f32 = 10_000.0;
    const R0: f32 = 10_000.0; // NTC resistance at T0
    const BETA: f32 = 3950.0;
    const T0_K: f32 = 298.15; // 25 degC
    let v = mv as f32;
    if v < 1.0 || v > VCC_MV - 1.0 {
        return f32::NAN; // open or shorted sensor
    }
    let r_ntc = R_FIXED * v / (VCC_MV - v);
    let inv_t = 1.0 / T0_K + libm::logf(r_ntc / R0) / BETA;
    1.0 / inv_t - 273.15
}

// ----------------------------------------------------------------------------------------
----
// main = hardware bring-up + sensor loop
// ----------------------------------------------------------------------------------------
----
#[esp_rtos::main]
async fn main(spawner: Spawner) -> ! {
    esp_println::logger::init_logger_from_env();
    let peripherals =
esp_hal::init(esp_hal::Config::default().with_cpu_clock(CpuClock::max()));
    let _camera = reserve_camera_pins!(peripherals);

    esp_alloc::heap_allocator!(#[ram(reclaimed)] size: 64 * 1024);
    esp_alloc::heap_allocator!(size: 36 * 1024);

    let timg0 = TimerGroup::new(peripherals.TIMG0);
    let sw_int = SoftwareInterruptControl::new(peripherals.SW_INTERRUPT);
    esp_rtos::start(timg0.timer0, sw_int.software_interrupt0);

    // --- peripherals owned by tasks ---------------------------------------------------
-----
    let i2c = I2c::new(peripherals.I2C0,
I2cConfig::default().with_frequency(Rate::from_khz(100)))
        .unwrap()
        .with_sda(peripherals.GPIO41)
        .with_scl(peripherals.GPIO40);
    let lcd = Lcd1602::new(i2c, Delay::new(), LCD_ADDR);
    let button = Input::new(peripherals.GPIO14,
InputConfig::default().with_pull(Pull::Up));
    let mut alarm = Output::new(peripherals.GPIO21, Level::Low, OutputConfig::default());

    // --- peripherals owned by main (the sensor loop) ----------------------------------
-------
    let mut adc_cfg = AdcConfig::new();
    let mut ldr = adc_cfg.enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(peripherals.GPIO1,
Attenuation:: 11dB);
    let mut ntc = adc_cfg.enable_pin_with_cal::<_, AdcCalCurve<ADC1>>(peripherals.GPIO2,
Attenuation::_11dB);
    let mut adc = Adc::new(peripherals.ADC1, adc_cfg);
```

```
    // --- Wi-Fi station + network stack ------------------------------------------------
    let station = Config::Station(StationConfig::default().with_ssid(SSID).with_password(PASSWORD.into()));
    let (controller, interfaces) = esp_radio::wifi::new(peripherals.WIFI, ControllerConfig::default().with_initial_config(station)).unwrap();
    let rng = Rng::new();
    let seed = (rng.random() as u64) << 32 | rng.random() as u64;
    let (stack, runner) = embassy_net::new(
        interfaces.station,
        embassy_net::Config::dhcpv4(Default::default()),
        mk_static!(StackResources<4>, StackResources::<4>::new()),
        seed,
    );

    spawner.spawn(connection(controller).unwrap());
    spawner.spawn(net_task(runner).unwrap());
    spawner.spawn(ip_task(stack).unwrap());
    spawner.spawn(http_task(stack).unwrap());
    spawner.spawn(display_task(lcd).unwrap());
    spawner.spawn(button_task(button).unwrap());

    // --- sensor loop ------------------------------------------------------------------
    let boot = Instant::now();
    let (mut t_min, mut t_max) = (f32::MAX, f32::MIN);
    let mut ticker = Ticker::every(Duration::from_secs(1));
    loop {
        let (mut l, mut n) = (0u32, 0u32);
        for   in 0..8 {
            l += nb::block!(adc.read_oneshot(&mut ldr)).unwrap() as u32;
            n += nb::block!(adc.read_oneshot(&mut ntc)).unwrap() as u32;
        }
        let (light_mv, ntc_mv) = (l / 8, n / 8);
        let t = ntc_celsius(ntc_mv);
        if t.is_finite() {
            t_min = t_min.min(t);
            t_max = t_max.max(t);
        }
        let reading = Reading {
            ntc_c: t,
            ntc_min: if t_min <= t_max { t_min } else { 0.0 },
            ntc_max: if t_min <= t_max { t_max } else { 0.0 },
            light_pct: (light_mv * 100 / FULL_SCALE_MV).min(100) as u8,
            light_mv: light_mv as u16,
            uptime_s: boot.elapsed().as_secs() as u32,
        };
        *LATEST.lock().await = reading;
        alarm.set_level(if reading.ntc_c > ALARM_C { Level::High } else { Level::Low });
        info!(
            "ntc {:.1} C (min {:.1}, max {:.1}), light {} % ({} mV)",
            reading.ntc_c, reading.ntc_min, reading.ntc_max, reading.light_pct, reading.light_mv
        );
        ticker.next().await;
    }
}

// ---------------------------------------------------------------------------------------
// tasks
// ---------------------------------------------------------------------------------------
fn pad16(s: &mut heapless::String<24>) {
    while s.len() < 16 {
        let _ = s.push(' ');
    }
    s.truncate(16);
}
```

```rust
#[embassy_executor::task]
async fn display_task(mut lcd: Lcd1602<I2c<'static, Blocking>, Delay>) {
    if lcd.init().is_err() {
        warn!("LCD not found at 0x{:02X}: display task ends (the rest keeps running)",
LCD_ADDR);
        return;
    }
    let mut ticker = Ticker::every(Duration::from_millis(500));
    loop {
        let r = *LATEST.lock().await;
        let ip = *IPADDR.lock().await;
        let mut top = heapless::String::<24>::new();
        let mut bottom = heapless::String::<24>::new();
        if PAGE.load(Ordering::Relaxed) % 2 == 0 {
            let _ = write!(top, "T{:5.1}C L{:3}%", r.ntc_c, r.light_pct);
            let _ = write!(bottom, "{:3.0}..{:3.0}C {:5}s", r.ntc_min, r.ntc_max,
r.uptime_s); // 16 columns
        } else {
            match ip {
                Some([a, b, c, d]) => {
                    let _ = write!(top, "{}.{}.{}.{}", a, b, c, d);
                    let _ = write!(bottom, "Wi-Fi OK  :80");
                }
                None => {
                    let _ = write!(top, "No IP yet");
                    let _ = write!(bottom, "Wi-Fi connecting");
                }
            }
        }
        pad16(&mut top);
        pad16(&mut bottom);
        let _ = lcd.set_cursor(0, 0);
        let _ = lcd.print(&top);
        let _ = lcd.set_cursor(0, 1);
        let _ = lcd.print(&bottom);
        ticker.next().await;
    }
}

#[embassy_executor::task]
async fn button_task(mut button: Input<'static>) {
    loop {
        button.wait_for_falling_edge().await;
        PAGE.fetch_xor(1, Ordering::Relaxed);
        Timer::after_millis(250).await; // debounce
    }
}

#[embassy_executor::task]
async fn connection(mut controller: WifiController<'static>) {
    loop {
        match controller.connect_async().await {
            Ok(info) => {
                info!("Wi-Fi connected: {:?}", info);
                let reason = controller.wait_for_disconnect_async().await.ok();
                warn!("Wi-Fi disconnected: {:?}", reason);
                *IPADDR.lock().await = None;
            }
            Err(e) => warn!("Wi-Fi connect failed: {:?}", e),
        }
        Timer::after(Duration::from_secs(5)).await;
    }
}

#[embassy_executor::task]
async fn net_task(mut runner: Runner<'static, Interface<'static>>) {
    runner.run().await
}

#[embassy_executor::task]
async fn ip_task(stack: Stack<'static>) {
```

```
    loop {
        stack.wait_config_up().await;
        if let Some(cfg) = stack.config_v4() {
            let o = cfg.address.address().octets();
            info!("Got IP {}.{}.{}.{}", o[0], o[1], o[2], o[3]);
            *IPADDR.lock().await = Some(o);
        }
        stack.wait_config_down().await; // loop again after the next (re)connect
    }
}

#[embassy_executor::task]
async fn http_task(stack: Stack<'static>) {
    let mut rx = [0u8; 512];
    let mut tx = [0u8; 1024];
    loop {
        let mut socket = TcpSocket::new(stack, &mut rx, &mut tx);
        socket.set_timeout(Some(Duration::from_secs(10)));
        if socket.accept(80).await.is_err() {
            continue;
        }

        let mut req = [0u8; 256];
        let n = socket.read(&mut req).await.unwrap_or(0);
        let r = *LATEST.lock().await;
        let ntc = if r.ntc_c.is_finite() { r.ntc_c } else { -999.0 };

        let wants_json = req[..n].starts_with(b"GET /api");
        let mut body = heapless::String::<400>::new();
        if wants_json {
            let _ = write!(
                body,

"{{\"ntc_c\":{:.1},\"ntc_min\":{:.1},\"ntc_max\":{:.1},\"light_pct\":{},\"light_mv\":{},\
"uptime_s\":{}}}",
                ntc, r.ntc_min, r.ntc_max, r.light_pct, r.light_mv, r.uptime_s
            );
        } else {
            let _ = write!(
                body,
                "<!doctype html><meta charset=utf-8><meta http-equiv=refresh content=2>\
                 <title>ESP32-S3 monitor</title><h1>ESP32-S3 monitor</h1>\
                 <p>Temperature (NTC): {:.1} &deg;C<p>Session min / max: {:.1} / {:.1}
&deg;C\
                 <p>Light: {} % ({} mV)<p>Uptime: {} s",
                ntc, r.ntc_min, r.ntc_max, r.light_pct, r.light_mv, r.uptime_s
            );
        }
        let mut resp = heapless::String::<640>::new();
        let _ = write!(
            resp,
            "HTTP/1.0 200 OK\r\nContent-Type: {}\r\nContent-Length: {}\r\nConnection:
close\r\n\r\n{}",
            if wants_json { "application/json" } else { "text/html" },
            body.len(),
            body
        );
        let _ = socket.write_all(resp.as_bytes()).await;
        let _ = socket.flush().await;
        socket.close();
        Timer::after_millis(50).await; // let the stack send the FIN before reusing
buffers
    }
}
```

Software architecture

Tasks. One executor runs seven futures cooperatively. Each `await` is a scheduling point.

Task	Rate / trigger	Responsibility

Task	Rate / trigger	Responsibility
`main`	`Ticker` 1 s	Owns ADC1 and the two `AdcPin`s; samples (8 x oversampling each), converts, updates `LATEST`, drives the alarm LED
`display_task`	`Ticker` 500 ms	Reads `LATEST` and `IPADDR`; writes the LCD page selected by `PAGE`
`button_task`	falling edge (async)	Toggles `PAGE`, debounces with a 250 ms sleep
`connection`	event-driven	`connect_async`, then `wait_for_disconnect_async`, retry after 5 s
`net_task`	continuous	`Runner::run()` drives the embassy-net/smoltcp stack
`ip_task`	DHCP event	Waits for `wait_config_up`, stores the address, waits for `wait_config_down`
`http_task`	TCP connection	Accepts on port 80, reads the request line, answers `/api` with JSON and anything else with an HTML page that refreshes every 2 s

Shared state. `LATEST` and `IPADDR` are `embassy_sync::mutex::Mutex<CriticalSectionRawMutex, _>`; `PAGE` is an `AtomicU8`. The `Reading` struct is `Copy`, so a reader copies it under the lock and releases the lock before doing slow work (formatting, I2C, sockets). No lock is held across an `await` that could block another task for long.

Ownership. `main` keeps the ADC and its two `AdcPin`s (naming their long generic types in a task signature would be awkward, and it keeps all analog work in one place), while the LCD, button and network handles move into the tasks that use them. The I2C bus belongs to `Lcd1602`, which belongs to `display_task`, so no other task can disturb the bus.

Blocking caveat. The LCD driver is blocking: updating both lines takes roughly tens of milliseconds at 100 kHz, during which the executor cannot run other tasks. This is acceptable here; for stricter latency, write an async LCD driver on `embedded_hal_async::i2c::I2c`.

Thermistor math. With the divider `3V3 - 10 k - node - NTC - GND`, `R_ntc = 10k * V / (3300 - V)` and, by the beta equation, `1/T = 1/298.15 + ln(R/10k)/3950`, then `T(C) = T - 273.15`. The values 10 kohm, B = 3950 are typical and must be replaced by your thermistor's datasheet. The ADC is calibrated at the factory, but the thermistor and resistor tolerances limit the accuracy to about +-1-2 degrees.

Memory. The firmware image is about 520 KB of code and the heap is 100 KB (64 KB reclaimed IRAM plus 36 KB DRAM). Wi-Fi needs a heap; increase it if you see allocation panics.

Build, flash and test

```
SSID="MyWifi" PASSWORD="secret" cargo run --release --bin proj3_iot_monitor
```

Step	Action	Expected
1	Boot	Serial: sensor lines each second; `Wi-Fi connected`; `Got IP a.b.c.d`
2	LCD page 0	`T 23.4C L 41%` and `min..max C  uptime s`
3	Press the button	LCD switches to the IP address and `Wi-Fi OK  :80`
4	Browse to `http://<ip>/`	A page that refreshes every 2 s
5	`curl -s http://<ip>/api`	JSON such as `{"ntc_c":23.4,"ntc_min":22.9,"ntc_max":23.6,"light_pct":41,"light_mv":1280,"uptime_s":57}`
6	Warm the NTC above 30 C	Alarm LED on GPIO21 lights
7	Power-cycle the router	`Wi-Fi disconnected` then automatic reconnect; LCD page 1 shows `No IP yet` meanwhile

If the build prints `SSID/PASSWORD not set` at run time, you did not pass the variables to cargo.

Appendices

Appendix A - Project directory structure and Cargo.toml

All source code of the manual is delivered as the Cargo package `esp32s3-labs` (also supplied as a zip archive next to this document).

```
esp32s3-labs/
|-- .cargo/
|   `-- config.toml   # target, runner, ESP_LOG, build-std
|-- docs/   # verification evidence (compiler outputs, build records)
|   |-- compile_fail/
|   |   `-- pin_conflict.rs
|   |-- ble_embassy_sync_mismatch_error.txt
|   |-- final_build_record.txt
|   |-- pin_conflict_compiler_output.txt
|   |-- stable_only_check_record.txt
|   `-- unstable_feature_missing_errors.txt
|-- src/
|   |-- bin/
|   |   |-- lab00_hello.rs   # Lab 00: Hello world, logging and the build pipeline
|   |   |-- lab01_gpio_output.rs   # Lab 01: GPIO digital output: blinking an LED
|   |   |-- lab02_gpio_input.rs   # Lab 02: GPIO digital input, pull-up / pull-down and
debouncing
|   |   |-- lab03_cpu_clock.rs   # Lab 03: CPU clock configuration
|   |   |-- lab04_adc.rs   # Lab 04: ADC analog-to-digital conversion
|   |   |-- lab05_ledc_pwm.rs   # Lab 05: LEDC PWM: LED brightness control
|   |   |-- lab06_servo.rs   # Lab 06: Servo motor control with PWM
|   |   |-- lab07_timer.rs   # Lab 07: Hardware timers and periodic events
|   |   |-- lab08_gpio_interrupt.rs   # Lab 08: GPIO interrupts and interrupt handlers
|   |   |-- lab09_uart.rs   # Lab 09: UART serial communication
|   |   |-- lab10_i2c_scan.rs   # Lab 10: I2C master: bus scan
|   |   |-- lab11_lcd1602.rs   # Lab 11: LCD1602 on I2C (PCF8574 backpack)
|   |   |-- lab12_spi.rs   # Lab 12: SPI master: full-duplex loopback
|   |   |-- lab13_rc522.rs   # Lab 13: RC522 RFID reader over SPI
|   |   |-- lab14_dma_mem2mem.rs   # Lab 14: DMA: memory-to-memory transfer
|   |   |-- lab14b_spi_dma.rs   # Lab 14B: DMA with SPI: full-duplex loopback
|   |   |-- lab15_watchdog.rs   # Lab 15: Watchdog timer
|   |   |-- lab16_rmt_ws2812.rs   # Lab 16: RMT peripheral: driving a WS2812 RGB LED
|   |   |-- lab17_pcnt_encoder.rs   # Lab 17: PCNT pulse counter and rotary encoder
|   |   |-- lab18_mcpwm.rs   # Lab 18: MCPWM: motor-control PWM signals
|   |   |-- lab19_twai.rs   # Lab 19: TWAI (CAN 2.0) communication
|   |   |-- lab20_wifi.rs   # Lab 20: Wi-Fi connectivity: station, DHCP and a TCP echo
server
|   |   |-- lab21_ble.rs   # Lab 21: Bluetooth Low Energy: GATT Battery Service
|   |   |-- lab22a_usb_serial_jtag.rs   # Lab 22A: USB Serial/JTAG: blocking echo
|   |   |-- lab22b_usb_otg_cdc.rs   # Lab 22B: USB OTG: CDC-ACM serial device with
embassy-usb
|   |   |-- lab23_embassy.rs   # Lab 23: Asynchronous programming with Embassy
|   |   |-- lab24_psram.rs   # Lab 24: Bonus: external PSRAM as heap
|   |   |-- proj1_basic_monitor.rs   # Project 1: basic monitoring system
|   |   |-- proj2_rfid_access.rs   # Project 2: RFID access control
|   |   `-- proj3_iot_monitor.rs   # Project 3: IoT monitor (Wi-Fi + Embassy)
|   |-- board.rs   # camera-pin reservation (type-level GPIO guard)
|   |-- lcd1602.rs   # HD44780/PCF8574 LCD driver
|   `-- lib.rs   # shared library root
|-- .clippy.toml
|-- .gitignore
|-- build.rs   # linker-error helper + linker script (generated)
|-- Cargo.lock
|-- Cargo.toml   # dependencies, features, profiles (esp-hal pinned =1.1.1)
|-- README.md   # how to build and use the project
|-- rust-toolchain.toml   # channel = "esp"
`-- VERIFICATION.md   # what was verified and how
```

The complete `Cargo.toml` of the project:

```toml
[package]
name = "esp32s3-labs"
version = "0.1.0"
edition = "2024"
rust-version = "1.88"
description = "ESP32-S3 + Rust + esp-hal 1.1.1 laboratory manual: all lab sources"
publish = false

# ---------------------------------------------------------------------------
# Core HAL stack. esp-hal is pinned with "=" so a newer 1.1.x/1.2.x release can
# never be picked up silently (the manual is written for exactly 1.1.1).
# ---------------------------------------------------------------------------
[dependencies]
esp-hal = { version = "=1.1.1", features = ["esp32s3", "unstable", "log-04"] }
esp-bootloader-esp-idf = { version = "0.5.0", features = ["esp32s3", "log-04"] }
esp-println = { version = "0.17.0", features = ["esp32s3", "log-04"] }
esp-backtrace = { version = "0.19.0", features = ["esp32s3", "panic-handler", "println"]
}
esp-alloc = "0.10.0"
log = "0.4.27"
critical-section = "1.2.0"
nb = "1.1.0"
static_cell = "2.1.1"
heapless = "0.9.0"

# embedded-hal ecosystem (traits implemented by esp-hal drivers)
embedded-hal = "1.0.0"
embedded-hal-async = "1.0.0"
embedded-hal-bus = "0.3.0"
embedded-io-async = "0.7.0"

# Embassy + RTOS glue (async labs, Wi-Fi, BLE, USB)
esp-rtos = { version = "0.3.0", features = ["esp32s3", "embassy", "esp-radio", "esp-
alloc", "log-04"] }
esp-radio = { version = "0.18.0", features = ["esp32s3", "wifi", "ble", "coex", "esp-
alloc", "unstable", "log-04"] }
embassy-executor = { version = "0.10.0", features = ["log"] }
embassy-time = { version = "0.5.0", features = ["log"] }
# trouble-host 0.6.0 is built against embassy-sync 0.7 and its #[gatt_server] macro
resolves
# the name through THIS crate: a different version here gives a NoopRawMutex trait error.
embassy-sync = "0.7.2"
embassy-futures = "0.1.2"
embassy-net = { version = "0.9.1", features = ["tcp", "udp", "dhcpv4", "medium-ethernet",
"log"] }
embassy-usb = { version = "0.6.0", default-features = false }
trouble-host = { version = "0.6.0", features = ["gatt", "default-packet-pool-mtu-255",
"derive", "scan"] }
bt-hci = "0.8.0"

# maths for the NTC thermistor (no_std ln)
libm = "0.2.11"

# RFID
mfrc522 = "0.8.0"

[features]
# Lab 03: select the CPU clock preset at build time.
clk-80 = []
clk-160 = []
clk-240 = []

[profile.dev]
# The default debug profile is too slow and too big for resource-constrained devices.
opt-level = "s"

[profile.release]
codegen-units = 1 # LLVM can perform better optimizations using a single thread
debug = 2         # prefer slower builds but better debugging experience
lto = 'fat'
opt-level = 's'
```

`src/lib.rs` (shared code root):

```
//! Shared helpers for the ESP32-S3 / esp-hal 1.1.1 laboratory manual.
//!
//! * [`board`]   - camera-pin reservation (type-level GPIO conflict protection)
//! * [`lcd1602`] - minimal HD44780 driver for the PCF8574 I²C "backpack"
#![no_std]

pub mod board;
pub mod lcd1602;
```

`src/board.rs` (camera-pin reservation):

```
//! Board pin map and camera-pin reservation.
//!
//! The OV2640 camera occupies fourteen GPIOs. In Rust the cleanest way to
//! guarantee that no other driver can claim them is to *move the pin tokens*
//! out of `Peripherals` into a struct that nobody else can access. The
//! compiler then rejects any later attempt to use the same pin
//! (`use of moved value: peripherals.GPIO15`).
//!
//! | Signal | GPIO | Signal | GPIO |
//! |--------|------|--------|------|
//! | XCLK   | 15   | D4     | 12   |
//! | SIOD   | 4    | D5     | 18   |
//! | SIOC   | 5    | D6     | 17   |
//! | D0     | 11   | D7     | 16   |
//! | D1     | 9    | VSYNC  | 6    |
//! | D2     | 8    | HREF   | 7    |
//! | D3     | 10   | PCLK   | 13   |
use esp_hal::peripherals::{
    GPIO4, GPIO5, GPIO6, GPIO7, GPIO8, GPIO9, GPIO10, GPIO11, GPIO12, GPIO13, GPIO15,
GPIO16,
    GPIO17, GPIO18,
};

/// All GPIO tokens that belong to the OV2640 camera interface.
///
/// Holding a value of this type for the whole program lifetime keeps the pins
/// reserved. Nothing here configures the pins; they stay in their reset state.
pub struct CameraPins<'d> {
    pub xclk: GPIO15<'d>,
    pub siod: GPIO4<'d>,
    pub sioc: GPIO5<'d>,
    pub d0: GPIO11<'d>,
    pub d1: GPIO9<'d>,
    pub d2: GPIO8<'d>,
    pub d3: GPIO10<'d>,
    pub d4: GPIO12<'d>,
    pub d5: GPIO18<'d>,
    pub d6: GPIO17<'d>,
    pub d7: GPIO16<'d>,
    pub vsync: GPIO6<'d>,
    pub href: GPIO7<'d>,
    pub pclk: GPIO13<'d>,
}

/// Move the camera GPIO tokens out of a `Peripherals` value.
///
/// This is a macro (not a function) so that only the *fields* are moved and
/// the rest of `peripherals` stays usable (Rust "partial move").
#[macro_export]
macro_rules! reserve_camera_pins {
    ($p:ident) => {
        $crate::board::CameraPins {
            xclk: $p.GPIO15,
            siod: $p.GPIO4,
            sioc: $p.GPIO5,
            d0: $p.GPIO11,
            d1: $p.GPIO9,
```

```
            d2: $p.GPIO8,
            d3: $p.GPIO10,
            d4: $p.GPIO12,
            d5: $p.GPIO18,
            d6: $p.GPIO17,
            d7: $p.GPIO16,
            vsync: $p.GPIO6,
            href: $p.GPIO7,
            pclk: $p.GPIO13,
        }
    };
}
```

`src/lcd1602.rs` (HD44780 over PCF8574 driver; build-verified, not hardware-tested):

```
//! Minimal HD44780 (LCD1602) driver for the common PCF8574 I²C backpack.
//!
//! PCF8574 → HD44780 wiring used by most backpacks (YwRobot / DFRobot style):
//!
//! ```text
//!  PCF8574 bit :  P7  P6  P5  P4  P3  P2  P1  P0
//!  LCD signal  :  D7  D6  D5  D4  BL  EN  RW  RS
//! ```
//!
//! The display is driven in 4-bit mode: each byte is sent as two nibbles, and
//! every nibble is latched with one pulse on EN. The driver is generic over the
//! `embedded-hal` 1.0 [`I2c`] and [`DelayNs`] traits, so it works with the
//! blocking esp-hal `I2c` driver and with `esp_hal::delay::Delay`.
//!
//! STATUS: build-verified only. It follows the standard HD44780 initialisation
//! sequence but has NOT been tested on real hardware by the author.
use core::fmt;
use embedded_hal::{delay::DelayNs, i2c::I2c};

const RS: u8 = 0x01;
const EN: u8 = 0x04;
const BACKLIGHT: u8 = 0x08;

/// HD44780 instruction codes used by this driver.
mod cmd {
    pub const CLEAR: u8 = 0x01;
    pub const HOME: u8 = 0x02;
    pub const ENTRY_MODE_INC: u8 = 0x06;
    pub const DISPLAY_ON: u8 = 0x0C; // display on, cursor off, blink off
    pub const FUNCTION_4BIT_2LINE: u8 = 0x28;
    pub const SET_DDRAM: u8 = 0x80;
}

/// LCD1602 behind a PCF8574 I²C expander.
pub struct Lcd1602<I2C, D> {
    i2c: I2C,
    delay: D,
    addr: u8,
    backlight: u8,
}

impl<I2C: I2c, D: DelayNs> Lcd1602<I2C, D> {
    /// Create the driver. Nothing is sent to the display until [`Self::init`].
    pub fn new(i2c: I2C, delay: D, addr: u8) -> Self {
        Self { i2c, delay, addr, backlight: BACKLIGHT }
    }

    /// Run the HD44780 power-on sequence for 4-bit mode (datasheet fig. 24).
    pub fn init(&mut self) -> Result<(), I2C::Error> {
        self.delay.delay_ms(50); // > 40 ms after Vcc reaches 2.7 V
        self.write_nibble(0x30, false)?; // 8-bit "function set", sent three
        self.delay.delay_ms(5); //          times to force a known state
        self.write_nibble(0x30, false)?;
        self.delay.delay_us(200);
        self.write_nibble(0x30, false)?;
        self.delay.delay_us(200);
```

```
        self.write_nibble(0x20, false)?; // switch to 4-bit interface
        self.delay.delay_us(200);
        self.command(cmd::FUNCTION_4BIT_2LINE)?;
        self.command(cmd::DISPLAY_ON)?;
        self.command(cmd::ENTRY_MODE_INC)?;
        self.clear()
    }

    /// Clear the display and return the cursor to (0, 0).
    pub fn clear(&mut self) -> Result<(), I2C::Error> {
        self.command(cmd::CLEAR)?;
        self.delay.delay_ms(2); // CLEAR needs ~1.52 ms
        Ok(())
    }

    /// Return the cursor to (0, 0) without clearing.
    pub fn home(&mut self) -> Result<(), I2C::Error> {
        self.command(cmd::HOME)?;
        self.delay.delay_ms(2);
        Ok(())
    }

    /// Move the cursor. `row` is 0 or 1, `col` is 0..=15.
    pub fn set_cursor(&mut self, col: u8, row: u8) -> Result<(), I2C::Error> {
        let offset = if row == 0 { 0x00 } else { 0x40 };
        self.command(cmd::SET_DDRAM | (offset + col.min(15)))
    }

    /// Turn the backlight on or off (takes effect on the next write).
    pub fn set_backlight(&mut self, on: bool) -> Result<(), I2C::Error> {
        self.backlight = if on { BACKLIGHT } else { 0 };
        self.expander_write(0)
    }

    /// Print an ASCII string at the current cursor position.
    pub fn print(&mut self, text: &str) -> Result<(), I2C::Error> {
        for b in text.bytes() {
            self.data(b)?;
        }
        Ok(())
    }

    /// Send an instruction byte (RS = 0).
    pub fn command(&mut self, value: u8) -> Result<(), I2C::Error> {
        self.send(value, false)
    }

    /// Send a data byte (RS = 1), i.e. one character.
    pub fn data(&mut self, value: u8) -> Result<(), I2C::Error> {
        self.send(value, true)
    }

    fn send(&mut self, value: u8, rs: bool) -> Result<(), I2C::Error> {
        self.write_nibble(value & 0xF0, rs)?; // high nibble first
        self.write_nibble((value << 4) & 0xF0, rs)?;
        self.delay.delay_us(50); // most instructions take < 40 µs
        Ok(())
    }

    /// Put one nibble (already in bits 7..4) on D7..D4 and pulse EN.
    fn write_nibble(&mut self, nibble: u8, rs: bool) -> Result<(), I2C::Error> {
        let base = nibble | if rs { RS } else { 0 };
        self.expander_write(base | EN)?; // EN high: LCD samples on falling edge
        self.delay.delay_us(1);
        self.expander_write(base)?; // EN low
        self.delay.delay_us(50);
        Ok(())
    }

    fn expander_write(&mut self, bits: u8) -> Result<(), I2C::Error> {
        self.i2c.write(self.addr, &[bits | self.backlight])
    }
```

```
}

/// Allows `write!(lcd, "T={:.1}C", t)`. I²C errors are mapped to `fmt::Error`.
impl<I2C: I2c, D: DelayNs> fmt::Write for Lcd1602<I2C, D> {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.print(s).map_err(|_| fmt::Error)
    }
}
```

`.cargo/config.toml`, `build.rs` and `rust-toolchain.toml` of the project are those of chapter 5 (differences: `build-std` includes `alloc`).

Appendix B - Final checklist for compiling, flashing and testing each laboratory

B.1 One-time preparation checklist

Done	Item
☐	System packages installed (chapter 3.1)
☐	Rust and `espup install --targets esp32s3` done; `. ~/export-esp.sh` sourced in this shell
☐	`espflash --version` works; `espflash board-info` identifies the chip
☐	User is in the `dialout` group; the serial port is listed by `espflash list-ports`
☐	Wiring checked against your board's schematic (chapter 7); the LCD bus is 3.3 V safe
☐	`cargo build --release --bin lab00_hello` succeeds and the monitor prints `Hello world!`

B.2 Per-laboratory checklist

For every row: build, flash and run with `cargo run --release --bin <bin>`, then tick the *Observed* box only after you have seen the behaviour on your board.

Lab	Command	Behaviour to observe	Observed
00	`cargo run --release --bin lab00_hello`	After flashing, the monitor shows the ROM and bootloader banner followed by one `Hello world!` line about every half second (INFO level).	☐
01	`cargo run --release --bin lab01_gpio_output`	The LED blinks at 1 Hz (500 ms on, 500 ms off) and the log prints `LED is ON` / `LED is OFF` alternately.	☐
02	`cargo run --release --bin lab02_gpio_input`	Pressing the button prints `button PRESSED` and lights the LED; releasing prints `button RELEASED`.	☐
03	`cargo run --release --bin lab03_cpu_clock --features clk-80`	Three log lines at start (requested preset, CPU MHz, XTAL MHz), then the time for 1,000,000 iterations (roughly three times shorter at 240 MHz than …	☐
04	`cargo run --release --bin lab04_adc`	Twice per second one line with the GPIO1 voltage in millivolts and the GPIO2 raw count out of 4095.	☐
05	`cargo run --release --bin lab05_ledc_pwm`	The LED breathes up and down over 3 s per cycle, then ramps up smoothly with a gamma curve.	☐
06	`cargo run --release --bin lab06_servo`	The servo horn moves in 30 degree steps from 0 to 180 degrees and back, pausing 600 ms at each position, while the log prints the angle.	☐
07	`cargo run --release --bin lab07_timer`	The LED toggles every 250 ms (2 Hz blink) with no blocking in `main`; once per second the log prints the number of ticks (about 4 per second).	☐
08	`cargo run --release --bin lab08_gpio_interrupt`	Each press increments `valid presses` once and toggles the LED, with no repeats caused by bounce.	☐

Lab	Command	Behaviour to observe	Observed
09	`cargo run --release --bin lab09_uart`	Every second: `loopback OK (n bytes): "Hello UART #k"`.	☐
10	`cargo run --release --bin lab10_i2c_scan`	Every 3 seconds the scan lists the devices found, e.g.	☐
11	`cargo run --release --bin lab11_lcd1602`	Line 1 shows `ESP32-S3 + Rust`; line 2 shows `uptime 0 s` increasing every second.	☐
12	`cargo run --release --bin lab12_spi`	Every 500 ms `SPI loopback OK: [A5, nn, 3C, FF, 00]` where `nn` increments.	☐
13	`cargo run --release --bin lab13_rc522`	At start the version register value is printed, then `Present a MIFARE card or tag to the reader ...`.	☐
14	`cargo run --release --bin lab14_dma_mem2mem`	`starting DMA copy of 10240 bytes` followed by `DMA copy verified OK in <n> us`.	☐
14B	`cargo run --release --bin lab14b_spi_dma`	Every second `SPI-DMA loopback OK, first bytes [..]`; without the jumper, `MISMATCH`.	☐
15	`cargo run --release --bin lab15_watchdog`	Boot 1: `reset reason: Some(ChipPowerOn)`, `watchdog armed: 3 s timeout`, five `fed the watchdog (n/5)` lines, the warning, then after about 3 s the …	☐
16	`cargo run --release --bin lab16_rmt_ws2812`	The LED slowly cycles through the colour wheel (rainbow) at low brightness.	☐
17	`cargo run --release --bin lab17_pcnt_encoder`	Turning the knob prints `encoder position: n (about n/4 detents)`: positive in one direction, negative in the other, continuing past +-100.	☐
18	`cargo run --release --bin lab18_mcpwm`	On a scope: 20 kHz on GPIO38 with duty ramping 0..100 % in 5 % steps every 200 ms while GPIO39 stays low; then the roles swap.	☐
19	`cargo run --release --bin lab19_twai`	Every second: `sent #n and received back: ...` showing the frame (ID 0x123, data `[n, DE, AD]`), and error counters `TX=0 RX=0`.	☐
20	`SSID="MyWifi" PASSWORD="secret" cargo run --release --bin lab20_wifi`	Log shows a scan list, `connecting to Wi-Fi ...`, `connected: ...`, `Got IP: 192.168.x.y/24`, then `echo: waiting for a client on port 7777`.	☐
21	`cargo run --release --bin lab21_ble`	Log: `Starting advertising and GATT service`.	☐
22A	`cargo run --release --bin lab22a_usb_serial_jtag`	Open `espflash monitor` or `picocom /dev/ttyACM0`: the banner appears and every typed lowercase letter returns in upper case.	☐
22B	`cargo run --release --bin lab22b_usb_otg_cdc`	The host enumerates a serial port named `USB-serial example`; typed lowercase letters are echoed in upper case.	☐
23	`cargo run --release --bin lab23_embassy`	The LED blinks at 1 Hz; each button press cycles the half-period 500 -> 250 -> 100 -> 1000 ms; the heartbeat line prints every 2 s even while the …	☐
24	`cargo run --release --bin lab24_psram`	Heap statistics before and after, `allocated 500 KiB, checksum ...` and a periodic `still alive` message.	☐
PROJ1	`cargo run --release --bin proj1_basic_monitor`	LED brightness follows the LDR; LCD shows light % and mV	☐
PROJ2	`cargo run --release --bin proj2_rfid_access`	Unknown card denied; whitelisted card opens the servo for 3 s	☐
PROJ3	`SSID=.. PASSWORD=.. cargo run --release --bin proj3_iot_monitor`	LCD pages switch with the button; `/` and `/api` answer over Wi-Fi	☐

Appendix C - Version pitfalls and migration notes

#	Pitfall (and where it was met while preparing this manual)	What to do
1	`esp-hal-embassy` no longer appears in the 1.1.x templates; Embassy support is `esp-rtos` with feature `embassy`	Use `#[esp_rtos::main]` and `esp_rtos::start(timg0.timer0, sw_int.software_interrupt0)`
2	Tasks are spawned with `spawner.spawn(task(..).unwrap())` in embassy-executor 0.10	The task function returns a `Result`
3	`embassy-sync 0.8` breaks trouble-host 0.6.0 macros (Lab 21 error)	Pin `0.7.2` until trouble-host moves on
4	`esp_hal::init` disables all watchdogs	Enable one explicitly (Lab 15)
5	`Spi::with_cs` is unstable	Drive CS with an `Output` (Lab 12)
6	No `tsens` driver for ESP32-S3	Use an external sensor (Project 3)
7	`Mem2Mem::new` takes `SPI2` on the S3	Do not use SPI2 elsewhere in that program
8	The `ESP_HAL_CONFIG_PSRAM_MODE` option was removed in the 1.1.0 line (esp-hal CHANGELOG, section `v1.1.0-rc.0`); PSRAM is initialised by `esp_alloc::psram_allocator!` (mode `Auto` by default)	Do not set the old environment variable
9	`esp-generate` creates projects for the esp-hal version current at *its* release	Install `esp-generate --version 1.3.0` for esp-hal 1.1.x templates
10	A looser requirement such as `~1.1` resolves to 1.1.2	Pin `esp-hal = "=1.1.1"`
11	The 1.1.1 doc example for `tsens` calls `to_celcius()`; the method is `to_celsius()`	Trust the source, not doc snippets
12	ADC2 cannot be used while Wi-Fi is active	Use ADC1 pins (GPIO1-10)
13	`espflash` from source needs rustc 1.95+	Use `cargo binstall espflash`
14	USB OTG and USB Serial/JTAG share GPIO19/20	Use BOOT+RESET to reflash after Lab 22b
15	PSRAM atomics do not work (esp-alloc docs)	Never allocate `Atomic*` or `Arc` in PSRAM

Appendix D - Verification record

Result: 30 of 30 binaries built successfully with 0 compiler warnings (final record `docs/final_build_record.txt`).

Binary	Build (`--release`)	Warnings	Code+data in flash (bytes)	Built without `unstable`
`lab00_hello`	OK	0	41,449	not tested
`lab01_gpio_output`	OK	0	42,025	yes, OK
`lab02_gpio_input`	OK	0	42,541	yes, OK
`lab03_cpu_clock`	OK	0	43,481	yes, OK
`lab04_adc`	OK	0	44,381	NO: fails with E0603 (negative control)
`lab05_ledc_pwm`	OK	0	46,493	NO: fails with E0603 (negative control)
`lab06_servo`	OK	0	44,993	not applicable (uses unstable modules)
`lab07_timer`	OK	0	45,097	not applicable (uses unstable modules)
`lab08_gpio_interrupt`	OK	0	43,389	not applicable (uses unstable modules)
`lab09_uart`	OK	0	58,765	yes, OK
`lab10_i2c_scan`	OK	0	62,321	yes, OK

Binary	Build (`--release`)	Warnings	Code+data in flash (bytes)	Built without `unstable`
`lab11_lcd1602`	OK	0	63,629	not applicable (uses unstable modules)
`lab12_spi`	OK	0	55,409	yes, OK
`lab13_rc522`	OK	0	63,685	not applicable (uses unstable modules)
`lab14_dma_mem2mem`	OK	0	57,745	not applicable (uses unstable modules)
`lab14b_spi_dma`	OK	0	69,673	not applicable (uses unstable modules)
`lab15_watchdog`	OK	0	44,141	not applicable (uses unstable modules)
`lab16_rmt_ws2812`	OK	0	53,013	not applicable (uses unstable modules)
`lab17_pcnt_encoder`	OK	0	45,625	not applicable (uses unstable modules)
`lab18_mcpwm`	OK	0	46,161	not applicable (uses unstable modules)
`lab19_twai`	OK	0	46,165	not applicable (uses unstable modules)
`lab20_wifi`	OK	0	504,273	not tested (companion crates)
`lab21_ble`	OK	0	335,141	not tested (companion crates)
`lab22a_usb_serial_jtag`	OK	0	34,605	not applicable (uses unstable modules)
`lab22b_usb_otg_cdc`	OK	0	75,397	not tested (companion crates)
`lab23_embassy`	OK	0	67,137	not tested (companion crates)
`lab24_psram`	OK	0	52,693	not applicable (uses unstable modules)
`proj1_basic_monitor`	OK	0	66,645	not applicable (uses unstable modules)
`proj2_rfid_access`	OK	0	76,665	not applicable (uses unstable modules)
`proj3_iot_monitor`	OK	0	538,725	not tested (companion crates)

Additional evidence runs

Test	Expected	Observed
Camera pin used twice (`docs/compile_fail/pin_conflict.rs`)	Compile error	`error[E0382]: use of moved value: peripherals.GPIO15`
Stable-labelled labs built with `unstable` off (labs 01, 02, 03, 09, 10, 12)	Build succeeds	All 6 succeeded
Unstable labs 04 and 05 built with `unstable` off	Build fails	`E0603: module analog is private` / `module ledc is private`
BLE lab with `embassy-sync 0.8`	(Predicted mismatch)	`E0277 NoopRawMutex ... RawMutex is not satisfied` + "multiple different versions of crate embassy_sync"
Every API in Part III exists in the tag	All resolve	171 of 171 resolved, 0 errors
Pin audit of all source files	No camera/flash/strapping/UART0 pin used; USB pins only in Lab 22b	Passed (section 7.4)

The raw records are included in the project archive under `docs/`: `final_build_record.txt`, `stable_only_check_record.txt`, `pin_conflict_compiler_output.txt`, `unstable_feature_missing_errors.txt`, `ble_embassy_sync_mismatch_error.txt`.

Toolchain used for verification: the Espressif Rust fork release v1.95.0.0 (`rustc 1.95.0-nightly`, host `x86_64-unknown-linux-gnu`, target `xtensa-esp32s3-none-elf`), Xtensa GCC 16.1.0 (crosstool-NG `esp-16.1.0_20260609`) as linker, `cargo build --release`. Dependencies were fetched from crates.io and are locked in `Cargo.lock`.

What this verification does not prove. Compilation proves that names, types, lifetimes and trait bounds are consistent with esp-hal 1.1.1. It does not prove correct timing, electrical behaviour, wireless interoperability, or correct pin choices for your board.

Appendix E - Resolved dependency versions

Crate	Locked version	Role
`bt-hci`	0.8.1	HCI types
`critical-section`	1.2.0	Critical sections for ISR-shared data
`embassy-executor`	0.10.0	Async executor and `#[task]`
`embassy-futures`	0.1.2	`select`, `join`
`embassy-net`	0.9.1	TCP/IP stack (smoltcp) for sockets and DHCP
`embassy-sync`	0.7.2	Mutex, Signal, channels
`embassy-time`	0.5.1	Timers, Ticker
`embassy-usb`	0.6.0	USB device stack
`embedded-hal`	1.0.0	Hardware traits (I2C, SPI, PWM, delay)
`embedded-hal-async`	1.0.0	Async hardware traits
`embedded-hal-bus`	0.3.0	SpiDevice from bus + CS
`embedded-io-async`	0.7.0	Async `Write` for sockets
`esp-alloc`	0.10.0	Global heap (and PSRAM heap)
`esp-backtrace`	0.19.0	Panic handler
`esp-bootloader-esp-idf`	0.5.0	Application descriptor (`esp_app_desc!`)
`esp-hal`	1.1.1	Hardware drivers (GPIO, ADC, LEDC, …)
`esp-println`	0.17.0	Console output and `log` backend
`esp-radio`	0.18.0	Wi-Fi and BLE
`esp-rtos`	0.3.0	Scheduler, Embassy time driver, `#[esp_rtos::main]`
`heapless`	0.9.3	Fixed-capacity strings and vectors
`libm`	0.2.16	`logf` for the thermistor
`log`	0.4.35	Logging macros
`mfrc522`	0.8.0	RC522 RFID driver
`nb`	1.1.0	`nb::block!` for non-blocking calls
`static_cell`	2.1.1	Static allocation of values
`trouble-host`	0.6.0	BLE host stack

Appendix F - References

- ESP-HAL 1.1.1 API documentation (ESP32-S3): https://docs.espressif.com/projects/rust/esp-hal/1.1.1/esp32s3/esp_hal/
- ESP-HAL source at the 1.1.1 tag: https://github.com/esp-rs/esp-hal/tree/esp-hal-v1.1.1 (examples in `/examples`, drivers in `/esp-hal/src`, device data in `/esp-metadata/devices/esp32s3.toml`)
- The Rust on ESP Book: https://esp-rs.github.io/book/ and https://docs.espressif.com/projects/rust/book/
- The Embedded Rust Book: https://docs.rust-embedded.org/book/
- Embassy: https://embassy.dev/
- esp-generate: https://github.com/esp-rs/esp-generate; espup: https://github.com/esp-rs/espup; espflash: https://github.com/esp-rs/espflash

- esp-radio 0.18.0: https://docs.rs/esp-radio/0.18.0/esp_radio/; esp-rtos, esp-alloc, esp-println on docs.rs
- trouble-host: https://docs.rs/trouble-host/0.6.0/trouble_host/; embassy-net: https://docs.rs/embassy-net/0.9.1/embassy_net/; embassy-usb: https://docs.rs/embassy-usb/0.6.0/embassy_usb/
- mfrc522: https://docs.rs/mfrc522/0.8.0/mfrc522/; embedded-hal-bus: https://docs.rs/embedded-hal-bus/0.3.0/embedded_hal_bus/
- ESP32-S3 datasheet and Technical Reference Manual: Espressif documentation portal, https://www.espressif.com/en/support/documents/technical-documents
- PCF8574 datasheet (Texas Instruments): https://www.ti.com/product/PCF8574; WS2812B datasheet: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf